

Psy ZK Circuit Journey: Tracing Proofs and Assumptions

This document provides a detailed walkthrough of the Zero-Knowledge proof lifecycle in Psy, starting from a user's local actions to the final, globally verifiable block proof. We meticulously track what each circuit proves and, critically, the assumptions it makes and how those assumptions are discharged by subsequent circuits.

Goal: Illustrate the flow of cryptographic guarantees and the progressive reduction of trust assumptions, culminating in a block proof dependent only on the previous block's established state.

Key:

- **Circuit:** The specific ZK circuit being executed.
 - **High-Level Purpose:** *Why* this circuit exists in the overall architecture.
 - **Proves (Technical Detail):** Specific cryptographic guarantees and state relations enforced by the circuit's constraints.
 - **How:** Key gadgets, hashing, and constraint mechanisms employed.
 - **Assumes** [A_] : Inputs or states treated as correct *before* this circuit's verification.
 - **Discharges** [R_] : Assumptions remaining from *previous* steps that are *verified* by this circuit.
 - **Remaining** [R_] : Assumptions still held true *after* this circuit's verification, passed to the next stage.
-

Phase 1: User Proving Session (UPS) - Local Execution

(User executes transactions and builds a local proof chain)

Step 1: Start Session

- **Circuit:** `UPSStartSessionCircuit`

- **High-Level Purpose:** To establish a cryptographically verified starting point for the user's transaction batch, ensuring it begins from a consistent and valid state relative to the last finalized global block. Prevents users from initiating proofs based on invalid or outdated personal states.
- **Proves:**
 - The provided `UserProvingSessionHeader` witness (`ups_header`) contains an internally consistent `session_start_context` and `current_state` .
 - `session_start_context.checkpoint_tree_root` matches the root of the verified `checkpoint_tree_proof` witness.
 - `session_start_context.checkpoint_leaf_hash` matches the value of the `checkpoint_tree_proof` .
 - `session_start_context.checkpoint_id` matches the index of the `checkpoint_tree_proof` .
 - The hash of the provided `checkpoint_leaf` witness matches `session_start_context.checkpoint_leaf_hash` .
 - The hash of the provided `state_roots` witness (`global_chain_root`) matches `checkpoint_leaf.global_chain_root` .
 - `state_roots.user_tree_root` matches the root of the verified `user_tree_proof` witness.
 - `session_start_context.start_session_user_leaf.user_id` matches the index of the `user_tree_proof` .
 - The hash of `session_start_context.start_session_user_leaf` matches the value of the `user_tree_proof` .
 - `current_state.user_leaf` matches `session_start_context.start_session_user_leaf` except `last_checkpoint_id` is updated to `session_start_context.checkpoint_id` .
 - `current_state.deferred_tx_debt_tree_root == EMPTY_TREE_ROOT` .
 - `current_state.inline_tx_debt_tree_root == EMPTY_TREE_ROOT` .
 - `current_state.tx_count == 0` .
 - `current_state.tx_hash_stack == ZERO_HASH` .
- **How:** `UPSSStartStepGadget` uses `MerkleProofGadget` s to verify paths, `Psy...Leaf/RootsGadget` s to hash witnesses and check consistency,

direct comparisons and constant checks. Public inputs calculated via `compute_tree_aware_proof_public_inputs`.

- **Assumes:**

- [A1.1] The *root hash* used in witness Merkle proofs (`checkpoint_tree_root` in `ups_header.session_start_context`) accurately reflects the globally finalized state of the previous block.
- [A1.4] The *constant* `empty_ups_proof_tree_root` used for the tree-aware public inputs is correct for this session's start.
- [A1.5] The *constant* `ups_step_circuit_whitelist_root` embedded in the output header is the correct root for allowed UPS circuits.
- (Initial correctness of witness data like proofs and leaves is assumed, then verified internally).

- **Discharges:** Internal consistency checks discharge assumptions about the relationships *between* the provided witness components (e.g., leaf data matches proof value).

- **Remaining:**

- [R1.1] = [A1.1] (Correctness of previous block's `CHKP` root).
- [R1.4] = [A1.4] (Correctness of session's `empty_ups_proof_tree_root`).
- [R1.5] = [A1.5] (Correctness of `ups_step_circuit_whitelist_root`).

Step 1.5: Execute Contract Function Circuit (CFC)

- **Circuit:** `DapenContractFunctionCircuit` (Specific instance per function)
- **High-Level Purpose:** To execute the specific smart contract logic defined by the developer for a single transaction call, locally generating a proof that *this execution instance* faithfully followed the code, given its specific inputs and assumed context. This decouples logic execution from state transition verification.

- **Proves:**

- The sequence of internal operations matches the compiled `DPNFunctionCircuitDefinition (fn_def)`.
- Given the assumed `tx_ctx_header` witness (containing start states like `start_contract_state_tree_root`, `start_deferred_tx_debt_tree_root`, call arguments hash/length) and circuit `inputs`:

- The simulated state commands (reads/writes to CST, debt tree interactions via `StateReaderGadget`) produce the `end_contract_state_tree_root` and `end_deferred_tx_debt_tree_root` recorded in `tx_ctx_header.transaction_end_ctx` .
 - The computed `outputs_hash` and `outputs_length` match those recorded in `tx_ctx_header.transaction_end_ctx` .
 - All `assertions` within the `fn_def` hold true.
- The public inputs hash (combining `session_proof_tree_root` and `tx_ctx_header` hash) is correctly computed.
- **How:** `PsyContractFunctionBuilderGadget` interprets `fn_def` , simulating execution using `SimpleDPNBuilder` and `StateReaderGadget` . Connects computed vs witnessed values in `tx_ctx_header` .
- **Assumes:**
 - [A1.5.1] The `DapenContractFunctionCircuitInput` witness (esp. `tx_input_ctx`) accurately reflects the state *before* this CFC execution (derived from the previous UPS step's output) and the correct function inputs/outputs.
 - [A1.5.3] The `session_proof_tree_root` witness correctly represents the root of the user's recursive proof tree *at this point* .
- **Discharges:** Internal consistency of the execution trace vs. the code definition (`fn_def`).
- **Remaining:**
 - [R1.5.1] Correctness of the assumed `tx_input_ctx` (start state, inputs/outputs).
 - [R1.5.3] Correctness of the assumed `session_proof_tree_root` .
- **Output:** A CFC Proof object.

Step 2: Verify CFC & Process UPS Delta

- **Circuit:** `UPSCFCStandardTransactionCircuit`
- **High-Level Purpose:** To integrate the result of a local CFC execution (Step 1.5) into the user's main proof chain. It verifies the CFC was executed correctly *and* that its claimed start/end states correctly link the previous UPS state to the next UPS state, while also proving the previous UPS step itself was valid.
- **Proves:**

- **Previous Step Validity:** Proof N-1 was valid & used a whitelisted UPS circuit ([R(N-1).5] discharged). Public inputs match header hash.
- **CFC Validity:** CFC proof (from Step 1.5) is valid & exists in the same UPS proof tree as Proof N-1 ([R1.5.3] discharged). CFC function is registered globally (GCON / CFT check linked via [R(N-1).1]).
- **Context Link:** The *inner public inputs hash* from the verified CFC proof matches the hash of the UPSCFCStandardStateDeltaInput witness used to calculate state changes ([R1.5.1] discharged). This is the critical link ensuring the state delta matches the verified computation.
- **State Delta Correctness:** The UPS header transition from step N-1 to N is valid:
 - UCON root updated correctly based on user_contract_tree_update_proof .
 - Debt tree roots updated correctly based on deferred/inline_tx_debt_pivot_proof s (starting from prev step's end state).
 - tx_count incremented; tx_hash_stack updated correctly.
- **How:** VerifyPreviousUPSStepProofInProofTreeGadget , UPSVerifyCFCStandardStepGadget connects cfc_inner_public_inputs_hash between UPSVerifyCFCProofExistsAndValidGadget and UPSCFCStandardStateDeltaGadget . Delta/pivot proofs verified.
- **Assumes:**
 - [A2.1] Witness data for *this* step (attestations, state delta proofs) is correct initially.
 - [R(N-1).1] (Prev Step) Last block's CHKP root correctness (used for CFC inclusion context).
 - [R(N-1).4] (Prev Step) Session's empty_ups_proof_tree_root correctness (defines proof tree base).
- **Discharges:** [R(N-1).5] (Prev UPS whitelist), [R1.5.1] (CFC Context), [R1.5.3] (CFC Proof Tree Root).
- **Remaining:**
 - [RN.1] = [R(N-1).1] (Last block's CHKP root).
 - [RN.4] = [R(N-1).4] (Session's empty_ups_proof_tree_root).

- [RN.5] (New) Current header's `ups_step_circuit_whitelist_root` correctness.

(Repeat Step 1.5 & 2, or deferred variant, for all user transactions)

Step 3: End Session

- **Circuit:** `UPStandardEndCapCircuit`
- **High-Level Purpose:** To securely conclude the user's local proving session, producing a single proof that attests to the validity of the entire sequence of transactions, authorized by the user's signature, and ready for network submission. It ensures the session ends in a clean state (no outstanding debts).
- **Proves:**
 - Last UPS step proof valid & used correct whitelisted circuit ([R_Last.5] discharged against *constant* `known_ups_circuit_whitelist_root`).
 - ZK Signature proof valid & in same proof tree ([R_Last.4] discharged).
 - Signature corresponds to user's key & authenticates `PsyUserProvingSessionSignatureDataCompact` derived from final UPS header state.
 - Nonce incremented correctly.
 - `last_checkpoint_id` updated correctly in final `UserLeaf` .
 - Final debt tree roots are empty.
 - (Optional) UPS proof tree aggregation used correct circuits (`known_proof_tree_circuit_whitelist_root`).
 - Public Outputs (`end_cap_result_hash` , `guta_stats_hash`) correctly computed.
- **How:** `UPSEndCapFromProofTreeGadget` orchestrates verification of last step & signature, `UPSEndCapCoreGadget` enforces final constraints. Optional `VerifyAggProofGadget` .
- **Assumes:**
 - [A3.1] Witness data correct initially.
 - [A3.2] Constant `known_ups_circuit_whitelist_root` .
 - [A3.3] Constant `known_proof_tree_circuit_whitelist_root` (if used).

- `[R_Last.1]` (Last tx step) Correctness of `CHKP` root used as session basis.
- **Discharges:** `[R_Last.5]` , `[R_Last.4]` . Potentially UPS tree agg assumptions.
- **Remaining:** `[R3.1] = [R_Last.1]` (Correctness of initial `CHKP` root).

Output of Phase 1: End Cap Proof + Public Inputs + State Deltas from user

Phase 2: Network Aggregation - Parallel Execution (Realms & Coordinators)

(The Proving Network receives End Cap proofs + state deltas from users and aggregate them to prove the change in the blockchain state root)

Step 4: Process End Cap Proof(s) (GUTA Entry - Realm)

- **Circuit(s):**
 - `GUTAVerifySingleEndCapCircuit` (Handles a single End Cap, e.g., an odd leaf in aggregation)
 - `GUTAVerifyTwoEndCapCircuit` (Handles pairs of End Cap proofs, typical base case)
- **High-Level Purpose:** To securely ingest user End Cap proofs into the GUTA process, verify their validity against the protocol and historical state, and translate them into the standard `GlobalUserTreeAggregatorHeader` format needed for recursive aggregation.
- **Proves:**
 - Input End Cap proof(s) (`proof_target` , `child_a/b_proof`) are valid ZK proofs.
 - The End Cap proof(s) were generated by the official End Cap circuit (fingerprint checked against `known_end_cap_fingerprint`).
 - The public inputs of the End Cap proof(s) correctly match the claimed result/stats (`end_cap_result` , `guta_stats`) provided as witness.
 - The `checkpoint_tree_root` claimed by the user(s) existed historically in the `CHKP` tree (verified via

- `checkpoint_historical_merkle_proof`).
 - (*For TwoEndCap*): The Nearest Common Ancestor (NCA) logic correctly combines the two individual user state transitions (`start_leaf` -> `end_leaf` at user indices) into a single state transition at their parent node in the `GUSR` tree. Statistics are correctly summed.
 - Outputs a `GlobalUserTreeAggregatorHeader` representing the state transition for the node processed (either a single user leaf or the NCA parent) and the combined stats.
- **How:**
 - `VerifyEndCapProofGadget` : Used internally (once or twice) to perform core End Cap proof verification, fingerprint check, public input matching, and historical checkpoint validation.
 - `TwoNCASStateTransitionGadget` (in `GUTAVerifyTwoEndCapCircuit`): Combines the two `GUSR` leaf transitions (derived from End Cap results) using an NCA proof witness.
 - `GUTASStatsGadget.combine_with` : Sums stats (in `GUTAVerifyTwoEndCapCircuit`).
 - Constructs the output `GlobalUserTreeAggregatorHeader`.
- **Assumes:**
 - [A4.1] Witness data (End Cap proof(s), results, stats, historical proofs, NCA proof if applicable) is correct initially.
 - [A4.2] Constant `known_end_cap_fingerprint_hash` is correct.
 - [A4.3] Public Input `guta_circuit_whitelist_root_hash` is correct.
 - [R3.1] (Implicit in End Cap) User session(s) based on valid past `CHKP` root.
- **Discharges:**
 - [R3.1] (via `VerifyEndCapProofGadget` 's historical proof check).
 - Validity of the input End Cap proof(s).
- **Remaining:**
 - [R4.1] (New) Correctness of the *current block's* `checkpoint_tree_root` (established by `checkpoint_historical_merkle_proof.current_root` and passed consistently upwards).
 - [R4.3] = [A4.3] (Correctness of `guta_circuit_whitelist` root).

Step 5: Aggregate GUTA Proofs (Recursive - Realm/Coordinator)

- **Circuit(s):**
 - `GUTAVerifyTwoGUTACircuit` (Aggregates two GUTA sub-proofs)
 - `GUTAVerifyLeftGUTARightEndCapCircuit` (Aggregates GUTA sub-proof and a new End Cap proof)
 - `GUTAVerifyLeftEndCapRightGUTACircuit` (Aggregates an End Cap proof and a GUTA sub-proof)
- **High-Level Purpose:** To recursively combine verified state transitions within the GUTA hierarchy. These circuits take proofs representing changes in subtrees (either previous GUTA aggregations or newly processed End Caps) and merge them into a proof for the parent node, typically using NCA logic.
- **Proves:**
 - Both input proofs (Type A and Type B, where Type can be GUTA or EndCap) are valid ZK proofs.
 - Input Proof A (if GUTA) used a whitelisted GUTA circuit (`[R(A).3]` discharged via `VerifyGUTAProofGadget`).
 - Input Proof A (if EndCap) used the whitelisted EndCap circuit (`[A5.EndCapFingerprint]` check via `VerifyEndCapProofGadget`).
 - Input Proof B processed similarly (`[R(B).3]` or `[A5.EndCapFingerprint]` discharged).
 - Both input proofs' headers/results reference the *same* `checkpoint_tree_root` (`[R(A).1]` and `[R(B).1]` verified to be equal, discharging one, remaining `[RN.1]`).
 - Both input proofs' headers reference the *same* `guta_circuit_whitelist` root (`[R(A).3]` and `[R(B).3]` verified to be equal, discharging one, remaining `[RN.3]`).
 - Public inputs of each input proof match their respective headers/results.
 - The NCA logic (`TwoNCASStateTransitionGadget`) correctly combines the state transitions (`SubTreeNodeStateTransition` from input GUTA headers or derived from EndCap results) based on the NCA proof witness.
 - Statistics are correctly summed (`GUTASStatsGadget.combine_with`).
 - Outputs a `GlobalUserTreeAggregatorHeader` for the parent NCA node.
- **How:**

- `VerifyGUTAProofGadget` (for GUTA inputs).
- `VerifyEndCapProofGadget` (for EndCap inputs).
- `TwoNCASStateTransitionGadget` : Core aggregation logic using NCA proof witness.
- Connections ensuring consistency of `checkpoint_tree_root` and `guta_circuit_whitelist` between inputs.
- **Assumes:**
 - `[A5.1]` Witness data (input proofs, headers/results, whitelist proofs, NCA proof) correct initially.
 - `[A5.EndCapFingerprint]` Constant `known_end_cap_fingerprint_hash` (if applicable).
 - `[R(A).1]` , `[R(B).1]` (from inputs) `CHKP` root correctness.
 - `[R(A).3]` , `[R(B).3]` (from inputs) `GUTA whitelist` correctness.
- **Discharges:** Validity/consistency of input proofs, their whitelist usage (`[R(A/B).3]`), and consistency of their assumed `CHKP` roots (`[R(A).1]` confirmed equal to `[R(B).1]`).
- **Remaining:**
 - `[RN.1]` = `[R(A).1]` (Common `CHKP` root correctness).
 - `[RN.3]` = `[R(A).3]` (Common `GUTA whitelist` correctness).

Step 5.5: Propagate GUTA Proof Upwards (Line Proof)

- **Circuit:** `GUTAVerifyGUTAToCapCircuit` (May be used within Realm or by Coordinator)
- **High-Level Purpose:** To efficiently propagate a verified state transition from a lower node up a direct path in the tree (where no merging is needed) to a higher level (e.g., the Realm root or the global root).
- **Proves:**
 - The input GUTA proof is valid and used a whitelisted GUTA circuit (`[R_In.3]` discharged).
 - The input proof references the correct `CHKP` root (`[R_In.1]`).
 - The state transition is correctly recalculated from the input proof's node level up to the target level (e.g., level 0) using the provided `top_line_siblings` witness.
 - Outputs a `GlobalUserTreeAggregatorHeader` with the state transition reflecting the change at the target level.

- **How:** `VerifyGUTAProofToLineGadget` (uses `VerifyGUTAProofGadget` and `GUTAHeaderLineProofGadget` which uses `SubTreeNodeTopLineGadget`).
- **Assumes:**
 - `[A5.5.1]` Witness data (input proof, header, whitelist proof, `top_line_siblings`) correct initially.
 - `[R_In.1]`, `[R_In.3]` (from input proof).
- **Discharges:** `[R_In.3]` (GUTA whitelist). Validity of input proof relative to `[R_In.1]`.
- **Remaining:**
 - `[R5.5.1]` = `[R_In.1]` (Common `CHKP` root correctness).
 - `[R5.5.3]` = `[R_In.3]` (Common `GUTA` whitelist correctness).

(Steps 5/5.5 repeat recursively until Realm roots are reached)

Step 5.N: Handle No GUTA Changes

- **Circuit:** `GUTANoChangeCircuit`
- **High-Level Purpose:** To allow the aggregation process to incorporate the latest checkpoint root even if no user activity (relevant to GUTA) occurred in a particular block or subtree. Maintains checkpoint consistency across the aggregation structure.
- **Proves:**
 - A specific `checkpoint_leaf` exists in the `checkpoint_tree` at the *previous* `checkpoint_id`.
 - The `GUSR` root remained unchanged between the previous and current checkpoint (`new_guta_header.state_transition` shows `old_node_value == new_node_value` at level 0).
 - Statistics are zero.
 - Outputs a `GlobalUserTreeAggregatorHeader` referencing the *current* `checkpoint_tree_root` but indicating no `GUSR` change.
- **How:** `GUTANoChangeGadget` (uses `MerkleProofGadget` for checkpoint proof, constructs no-op transition).
- **Assumes:**
 - `[A5.N.1]` Witness data correct initially.
 - `[A5.N.3]` Public Input `guta_circuit_whitelist` root is correct.
- **Discharges:** Internal consistency of checkpoint proof/leaf.
- **Remaining:**

- `[R5.N.1]` (New) Correctness of the *current* `checkpoint_tree_root` (from the witness proof).
- `[R5.N.3] = [A5.N.3]` (GUTA whitelist correctness).

Output of Phase 2: Aggregated GUTA proof(s) from each active Realm (or a NoChange proof), valid relative to `[R_Realm.1]` and `[R_Realm.3]`.

Phase 3: Coordinator Level Aggregation - Network Execution

(Coordinator combines proofs from Realms and global tree updates)

Step 6: Process User Registrations

- **Circuit:** `BatchAppendUserRegistrationTreeCircuit`
- **Proves:** Correct batch append to `URT` (output root valid given input root `[A6.3]`). Respects `register_users_circuit_whitelist` (`[A6.2]`).
- **Assumes:** `[A6.x]` (Witness, whitelist, input `URT` root).
- **Discharges:** Internal append proof consistency.
- **Remaining:** `[R6.2]` (Whitelist), `[R6.3]` (Input `URT` root correctness).

Step 7: Process Contract Deployments

- **Circuit:** `BatchDeployContractsCircuit`
- **Proves:** Correct batch append to `GCON` (output root valid given input root `[A7.3]`). Witness leaves match hashes. Respects `deploy_contract_circuit_whitelist` (`[A7.2]`).
- **How:** `BatchDeployContractsGadget`.
- **Assumes:** `[A7.x]` (Witness, whitelist, input `GCON` root).
- **Discharges:** Internal append/leaf consistency.
- **Remaining:** `[R7.2]` (Whitelist), `[R7.3]` (Input `GCON` root correctness).

Step 8: Aggregate Part 1 (Combine UserReg + Deploy + GUTA)

- **Circuit:** `VerifyAggUserRegistrationDeployContractsGUTACircuit`
- **Proves:** Input proofs (Agg UserReg, Agg Deploy, Agg GUTA) valid & used respective whitelisted circuits (`[R6.2]`, `[R7.2]`, `[R_GUTA.3]` discharged).

All inputs based on same `CHKP` root (`[R_GUTA.1]` verified across inputs).
Output header correctly combines state transitions.

- **How:** `VerifyAggUserRegistrationDeployContractsGUTAGadget` .
- **Assumes:**
 - `[A8.1]` Witness data correct initially.
 - `[R_GUTA.1]` (Implicit common `CHKP` root from inputs).
 - `[R6.3]` (Input `URT` root correctness).
 - `[R7.3]` (Input `GCON` root correctness).
- **Discharges:** Whitelists (`[R6.2]` , `[R7.2]` , `[R_GUTA.3]`). Input proof validity. Consistency of `CHKP` root `[R_GUTA.1]` .
- **Remaining:**
 - `[R8.1]` = `[R_GUTA.1]` (`CHKP` root correctness).
 - `[R8.3]` = `[R6.3]` (Input `URT` root correctness).
 - `[R8.4]` = `[R7.3]` (Input `GCON` root correctness).

Output of Phase 3: Single "Part 1" proof, valid relative to `[R8.1]` , `[R8.3]` , `[R8.4]` .

Phase 4: Final Block Proof - Network Execution

Step 9: Final Block Transition

- **Circuit:** `PsyCheckpointStateTransitionCircuit`
- **High-Level Purpose:** To generate the definitive proof for the block, verifying all aggregated work and cryptographically linking the block to its predecessor, thereby discharging all temporary assumptions made during parallel processing.
- **Proves:**
 - Part 1 Agg proof (Step 8) valid & used correct circuit.
 - New `CHKP` Leaf computed correctly from Part 1 outputs (new global roots for `URT` (`[R8.3]` discharged), `GCON` (`[R8.4]` discharged), `GUSR`), stats, time, randomness.
 - `CHKP` tree append operation correct, transitioning from `previous_checkpoint_proof.root` (`[R8.1]`) to `new_checkpoint_tree_root` .

- **Final Chain Link:** `previous_checkpoint_proof.root` matches the **Public Input** `previous_block_chkp_root`.
- **How:** `CheckpointStateTransitionChildProofsGadget`, `CheckpointStateTransitionCoreGadget`.
- **Assumes:**
 - `[A9.1]` Witness data correct initially.
 - `[A9.2]` **Public Input** `previous_block_chkp_root` == **previous block's finalized CHKP root**.
- **Discharges:** `[R8.1]` (`CHKP` root correctness discharged against public input `[A9.2]`). `[R8.3]` (`URT` root correctness), `[R8.4]` (`GCON` root correctness) implicitly discharged by relying on the verified Part 1 proof output.
- **Remaining Assumptions: None.**

Output: Final Block Proof. Its verification confirms the entire block's state transition is valid, contingent only on the validity of the previous block's state hash (`[A9.2]`) and ZKP soundness.

Psy: Horizontally Scalable Blockchain via PARTH and ZK Proofs

Introduction: The Serial Bottleneck

Traditional blockchains suffer from a serial execution bottleneck. They process transactions sequentially within a single state machine, meaning adding more nodes doesn't increase overall throughput (TPS). Parallel execution attempts often lead to race conditions and inconsistencies. Psy overcomes this fundamental limitation with its novel **PARTH** architecture and end-to-end Zero-Knowledge Proof (ZKP) system.

The PARTH Architecture: Foundation for Parallelism

PARTH (Parallelizable Account-based Recursive Transaction History) redefines blockchain state organization:

1. **Granular State:** Instead of one global state tree, Psy maintains a hierarchy of Merkle trees, most notably:
 - **Per-User, Per-Contract State (CSTATE):** Each user has a *separate* Merkle tree (CSTATE) representing their specific state within *each* smart contract they interact with.
 - **User Contract Tree (UCON):** Aggregates all CSTATE roots for a single user, representing their overall state across all contracts.
 - **Global User Tree (GUSR):** Aggregates all UCON roots, representing the state of all users.
 - **Global Contract Tree (GCON):** Represents the global state related to contract code/definitions.
 - **Checkpoint Tree (CHKP):** The top-level tree whose root hash represents a verifiable snapshot of the *entire blockchain state* at a given block.

2. Controlled Interaction Rules (Key to Scalability):

- **Write Locally:** A transaction initiated by a user can **only modify (write to)** the state within that specific user's own trees (primarily their `CSTATE` and `UCON` trees).
- **Read Globally (Previous State):** A transaction can **read** the state from *any* other user's trees (or global trees like `GCON`), but critically, it only sees the state **as it was finalized at the end of the *previous* block**.

3. **Enabling Parallelism:** Because write operations are isolated to the sender's state trees, and read operations access the immutable, finalized state of the *last* block, transactions from *different users* within the *same* block operate independently. They cannot conflict or cause race conditions. This independence is the architectural breakthrough that allows for massive parallel processing.

The End-to-End ZK Proof Process: Securing Parallelism

Psy uses a sophisticated system of ZK circuits and recursive proofs to verify all state transitions securely, even when processed in parallel.

Step 1: Local Transaction Execution & Proving (User Level)

- **Action:** A user interacts with a dApp, triggering one or more smart contract function calls.
- **Execution:** The logic defined in the specific **Contract Function Circuit (CFC)** associated with the smart contract is executed locally (or by a delegated prover).
- **State Update:** This execution modifies the user's `CSTATE` tree for that contract. The Merkle root of the user's `UCON` tree is also updated to reflect this change.
- **Proof Generation:** The user generates ZK proofs using **User Proving Session (UPS) circuits**. These proofs attest that:
 - The contract logic (CFC) was executed correctly.
 - The user's `CSTATE` and `UCON` trees transitioned correctly from a known previous state root to a new state root.

- **Assumptions:** These proofs rely on *public inputs* that declare assumptions about the state of the blockchain *before* the transaction (e.g., the user's previous `uCON` root, the relevant `GCON` root, and crucially, the `CHKP` root of the last finalized block which guarantees the validity of any global state read).
- **Recursion:** If the user performs multiple actions, recursive proofs compress them into a single proof representing the net state change for that user within the block.

Step 2: Parallel Proof Aggregation (Decentralized Proving Network + Realms)

- **Submission:** Users submit their final proofs and delta merkle proofs (representing their state transitions for the block) to their corresponding realm node on the Psy network. (see `handle_rcv_end_cap_from_user`)
- **Parallel Processing:** Thanks to the PARTH architecture ensuring user-transaction independence, the **Decentralized Proving Network** can take proofs from *thousands or millions* of users and begin verifying and aggregating them **in parallel**. Realms are in charge of sending the proofs to the queue to be aggregated by the proof workers
- Once all the update proofs are aggregated in a realm into a single proof, the realm sends that proof off to the block coordinator to be aggregated into a single proof of all realm updates at the end of the block.
- **Hierarchical Aggregation:** Specialized **Aggregation Circuits** (e.g., for aggregating user contract trees, or global user trees) recursively verify batches of proofs from the layer below within the PARTH structure.
 - Each aggregation circuit checks the validity of the input proofs it receives.
 - It **enforces the assumptions** declared in the public inputs of the lower-level proofs. For example, a circuit aggregating user states ensures all input proofs correctly referenced the same previous global user state root.
 - It outputs a single, smaller proof representing the validity of the entire batch it processed.

Step 3: Final Block Proof Generation (Consensus / Block Leader)

- **Final Aggregation:** The parallel aggregation process continues up the hierarchy, culminating in the **Checkpoint Tree "Block" Circuit**. This final circuit aggregates proofs from the top-level trees (like `GUSR` , `GCON`).
- **Inductive Verification:** Assumptions made by lower circuits are checked and discharged by higher circuits during the recursive process. By the time execution reaches the final "Block" circuit, the *only* remaining external assumption is the **root hash of the previous block's Checkpoint Tree (`CHKP` root)**.
- **Trustless Link:** This final circuit verifies this previous `CHKP` root against the public input provided (which is the known, finalized root from the preceding block). This check creates a cryptographic, trustless link between consecutive blocks.
- **Proof Singularity:** The output is a **single, succinct ZK proof** for the *entire block*, proving the validity of *all* transactions and the overall state transition from the previous `CHKP` root to the new `CHKP` root, without needing to reveal individual transaction details or re-execute anything.

Step 4: Verification and Finalization

- The final block proof is broadcast and verified by consensus nodes (or potentially bridge contracts on other L1s).
- Verification is extremely fast as it only involves checking the single ZK proof and the link to the previous block's state.
- Once verified, the new `CHKP` root becomes the finalized, canonical state snapshot for that block.

The Role of the Checkpoint Tree (`CHKP`) and Circuits

- **Global State Snapshot (`CHKP`):** The `CHKP` root acts as a universal anchor. It captures a full snapshot of the entire chain and allows anyone to efficiently verify *any* piece of state information (user balance, contract variable, etc.) from *any* block by providing a Merkle proof linking that data back to the validated `CHKP` root of that block.
- **Circuit Enforcement:** Each tree update within the PARTH structure is strictly controlled. Updates are only permitted if accompanied by a valid ZK proof generated by a pre-approved, **whitelisted circuit** designed for

that specific tree transition. This constraint, enforced by the aggregation circuits above, ensures state changes adhere precisely to the defined rules (contract logic, protocol rules).

Conclusion: Achieving Horizontal Scalability

Psy achieves true horizontal scalability through the synergy of:

1. **PARTH Architecture:** Its granular state and specific read/write rules eliminate conflicts between transactions from different users within a block.
2. **Parallel Proving:** This independence allows the computationally intensive task of ZK proof generation and aggregation to be massively parallelized across the Decentralized Proving Network.
3. **Recursive ZK Proofs:** Efficiently compress and verify vast amounts of computation and state changes into a single, easily verifiable proof for each block, secured by the inductive verification up to the final Block Circuit linking to the previous state.

By processing user transactions independently and in parallel, secured by end-to-end ZK verification, Psy breaks the serial bottleneck and offers a path to blockchain performance potentially orders of magnitude higher than traditional architectures, truly scaling horizontally as more proving resources are added.

Key terms: DPN - Dapen, the code name for the Psy rust smart contract language
UPS - User Proving Session, the process by which users prove transactions locally and generate the delta merkle proofs to be sent off to chain
End Cap - The last recursive proof which checks the signature and state deltas for a user
GUTA - Global User Tree Aggregation

The Psy User Proving Session (UPS) Proof Tree: Efficient Recursive Verification

1. Introduction: The Challenge of Recursive Verification Costs

The User Proving Session (UPS) relies on recursive ZK proofs, where each step cryptographically verifies the previous one. A naive approach might involve embedding the *entire* verification logic of the previous step's circuit *inside* the current step's circuit. However, this leads to several problems:

1. **Variable Circuit Complexity:** Different UPS steps might verify different underlying circuits (standard CFC, deferred CFC, start session), each with varying verification costs. This makes creating fixed-size, efficient recursive circuits difficult.
2. **Computational Bloat:** Repeatedly verifying complex proofs within recursion significantly increases the computational cost and proving time for each step.
3. **Limited Parallelism:** Tightly coupling verification logic hinders the potential to parallelize the *proving* work, even if the *witness generation* is serial.

The **UPS Proof Tree** architecture elegantly solves these issues by **deferring the direct verification cost** and replacing it with **cheap cryptographic commitments and existence checks**.

2. The UPS Proof Tree: Commit, Don't Verify (Yet)

2.1 Core Concept: A Merkle Tree of Proof Commitments

Instead of fully verifying the previous proof *within* the current step's circuit, the UPS system commits each generated proof to a Merkle tree specific to the session.

- **Leaf Content:** Each leaf i stores a hash committing to the proof's identity and context:
 - **Standard Proofs (like ZK Signature):** $\text{LeafValue}_i = \text{Hash}(\text{ProofFingerprint}_i, \text{PublicInputsHash}_i)$
 - **Tree-Aware Proofs (UPS Steps):** $\text{LeafValue}_i = \text{Hash}(\text{ProofFingerprint}_i, \text{TreeAwarePublicInputsHash}_i)$
 - Where $\text{TreeAwarePublicInputsHash}_i = \text{Hash}(\text{ProofTreeRoot}_{i-1}, \text{InnerPublicInputsHash}_i)$
- **Append-Only Construction:** As each proof (CFC execution proof, UPS step proof, signature proof) is generated *locally*, its corresponding LeafValue is computed and appended to the tree. The tree root updates with each addition.

2.2 Cheap In-Circuit Checks: Attesting Existence and Linkage

The core innovation lies in using specialized gadgets *instead* of full verifiers within the recursive steps:

- **Gadget 1: $\text{AttestProofInTreeGadget}$ (For non-tree-aware proofs like Signatures)**
 - **Function:** Proves that a commitment $\text{Hash}(\text{Fingerprint}, \text{PublicInputsHash})$ exists as a leaf within a tree identified by $\text{AttestedProofTreeRoot}$.

- **Mechanism:** Takes `Fingerprint`, `PublicInputsHash`, and a Merkle `inclusion_proof` witness. Computes the expected leaf hash and verifies the Merkle proof against it.
 - **Cost:** Primarily the cost of Merkle proof verification (logarithmic hashing) and a few hashes/comparisons – significantly cheaper than verifying the actual ZK proof.
 - **Assumption:** The ZK proof corresponding to the `Fingerprint` and `PublicInputsHash` *is itself valid*. This gadget only proves *commitment existence*.
- **Gadget 2: `AttestTreeAwareProofInTreeGadget` (For tree-aware UPS step proofs)**
 - **Function:** Proves that a commitment for a *tree-aware* proof exists in the tree, linking it to the *correct historical state* of the tree.
 - **Mechanism:** Takes `Fingerprint`, `InnerPublicInputsHash`, an `inclusion_proof` (in the *current* tree), and a `historical_root_proof` (pivot proof showing `Root_N-2 -> Root_N-1`). It computes the expected *tree-aware* leaf hash `Hash(Fingerprint, Hash(Root_N-1, InnerPublicInputsHash))` and verifies the `inclusion_proof` against it and the `current root`. It also verifies the `historical_root_proof`.
 - **Cost:** Cost of two Merkle proof verifications (inclusion and historical pivot) plus hashing/comparisons – still much cheaper than full ZK proof verification.
 - **Assumption:** The ZK proof corresponding to the `Fingerprint` and `InnerPublicInputsHash` (relative to `Root_N-1`) *is itself valid*.

2.3 How Recursion Works with the Tree: Deferral in Action

Consider UPS Step N verifying Step N-1:

1. **Step N-1 Runs:** Generates its ZK proof (`Proof_N-1`) and computes its tree-aware public inputs hash `TAPH_N-1 = Hash(Root_N-2, InnerPubHash_N-1)`. Its commitment `LeafValue_N-1 =`

`Hash(Fingerprint_N-1, TAPH_N-1)` is added to the tree, updating the root to `Root_N-1`.

2. Step N Circuit Runs:

- **Does NOT Verify** `Proof_N-1` directly.
- Instead, it uses `AttestTreeAwareProofInTreeGadget`.
- **Assumes:** The `Proof_N-1` (whose commitment is being checked) is *valid*.
- **Takes Witness:** `Fingerprint_N-1`, `InnerPubHash_N-1`, `inclusion_proof` (for Leaf N-1 in tree N-1), `historical_root_proof` (Root N-2 -> Root N-1).
- **Verifies:** That the commitment to `Proof_N-1` exists correctly linked to `Root_N-2` within the tree state `Root_N-1`. This cryptographically confirms the sequential link.
- **Computes:** Its own output header hash `InnerPubHash_N`.
- **Generates:** Its own ZK proof (`Proof_N`) whose public inputs commit to `Hash(Root_N-1, InnerPubHash_N)`.

Crucially, the circuit for Step N has a **fixed, relatively low complexity**, dominated by the Merkle proof gadgets, regardless of the complexity of the circuit used for Step N-1. It only deals with *commitments* to proofs, not the proofs themselves.

3. Discharging the Assumptions: The End Cap and Beyond

Throughout the UPS, the core assumption accumulates: **"All ZK proofs committed to the tree are valid."**

- **The End Cap's Role:** The `UPSStandardEndCapCircuit` is where this primary assumption begins to be addressed *for the UPS internal proofs*.
 - It verifies the *last* UPS step proof's commitment using `AttestTreeAwareProofInTreeGadget`.
 - It verifies the *ZK Signature proof's* commitment using `AttestProofInTreeGadget`.

- It ensures both verifications reference the *same final proof tree root*, linking the signature to the end state of the transaction sequence.
 - **Optional (but recommended):** It often includes a `VerifyAggProofGadget` (or `VerifyAggRootGadget`). This gadget takes a ZK proof (generated *outside* the UPS circuit) that recursively verifies *all the actual proofs committed to the UPS Proof Tree*. This aggregation proof essentially proves the core assumption: "All proofs committed to the tree with root `FinalRoot` are valid". By verifying *this single aggregation proof*, the End Cap circuit discharges the validity assumption for all internal UPS steps.
- **Deferred Verification:** The actual computationally expensive work of verifying all the individual UPS step proofs and the signature proof is bundled into generating the separate UPS Proof Tree aggregation proof. This aggregation can happen:
 - **Locally:** After witness generation but before End Cap proving.
 - **Remotely/Parallel:** In a future design, the witness data and tree commitments could be sent to parallel provers. They generate proofs for each step and the final aggregation proof. The End Cap circuit then only needs to verify the *single* aggregation proof.

4. Benefits Summary

1. **Constant Recursive Step Cost:** The cost of verifying the previous step inside the current step's circuit is fixed and low (Merkle checks), independent of the previous step's circuit complexity. This allows for efficient recursion with consistent circuit degrees.
2. **Deferral of Computation:** The heavy lifting of verifying the actual ZK proofs is pushed *out* of the main recursive path and consolidated into a single (optional but recommended) aggregation proof verified at the end.
3. **Enables Parallel Proving:** By using the tree commitments as interfaces, the *proving* of individual UPS steps (and the final tree aggregation) can be parallelized across multiple workers, even if witness generation is serial. Workers only need the relevant witnesses and the expected tree

roots/proofs to verify their step's link, without needing to run the *entire* previous circuit's verification logic.

4. **Architectural Flexibility:** The system clearly separates *committing* to a proof's existence/context from *verifying* the proof's internal validity, allowing different strategies for when and how the full verification occurs.

In essence, the UPS Proof Tree acts as a highly efficient cryptographic ledger *within* the proving session, allowing circuits to cheaply attest to the existence and sequential linkage of prior computational steps, while deferring the bulk verification cost to a final, potentially parallelizable, aggregation step.

Psy Architecture: Horizontally Scalable Blockchain via PARTH and ZK Proofs

1. Introduction: Beyond Sequential Limits

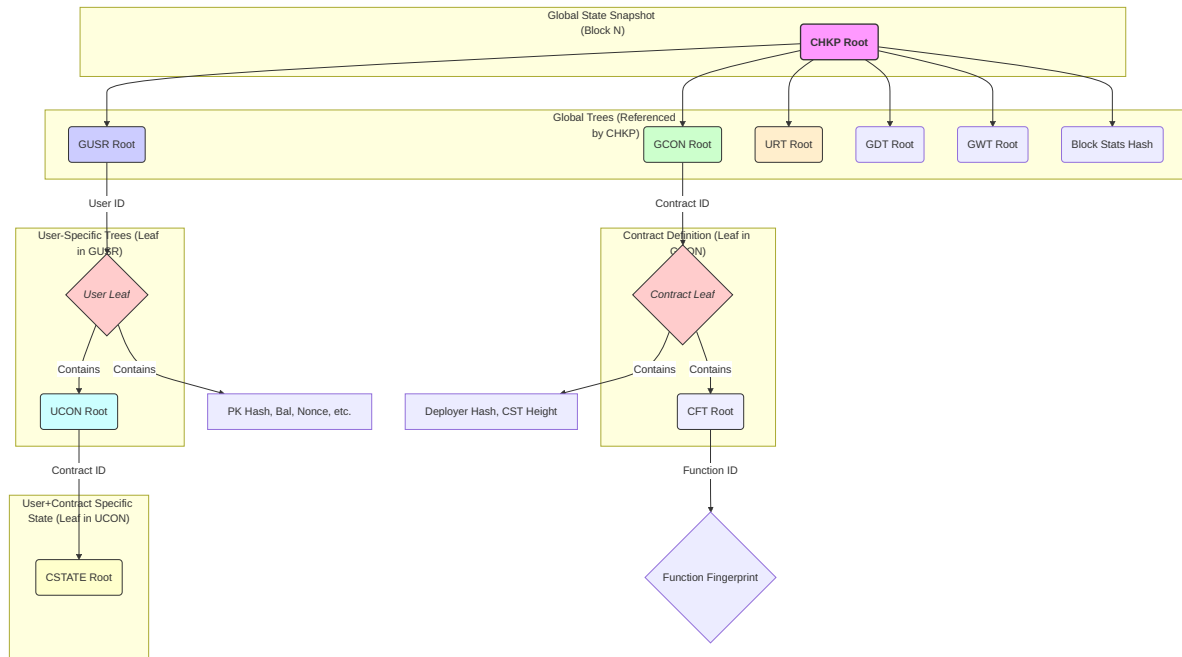
The evolution of blockchain technology has been marked by a persistent challenge: **scalability**. Traditional designs, processing transactions sequentially within a monolithic state machine, hit a throughput ceiling that cannot be overcome simply by adding more network nodes. Psy represents a fundamental leap forward, tackling this bottleneck through a revolutionary state architecture known as **PARTH** and a meticulously designed, end-to-end **Zero-Knowledge Proof (ZKP)** system. This architecture unlocks true horizontal scalability, enabling unprecedented transaction processing capacity while maintaining rigorous cryptographic security.

2. The PARTH Architecture: A Foundation for Parallelism

PARTH (Parallelizable Account-based Recursive Transaction History) dismantles the concept of a single, conflict-prone global state. Instead, it establishes a granular, hierarchical structure where state modifications are naturally isolated, paving the way for massive parallel processing.

2.1 The Hierarchical State Forest

Psy's state is not a single tree, but a "forest" of interconnected Merkle trees, each with a specific domain:



- **CHKP (Checkpoint Tree):** The ultimate source of truth for a given block. Its root immutably represents the entire state snapshot, committing to the roots of all major global trees and block statistics. Verifying the **CHKP** root transitively verifies the entire state.
- **GUSR (Global User Tree):** Aggregates all registered users. Each leaf (**ULEAF**) corresponds to a user ID and contains their public key commitment, balance, nonce, last synchronized checkpoint ID, and, crucially, the root of their personal **UCON** tree.
- **UCON (User Contract Tree):** A *per-user* tree mapping Contract IDs to the roots of the user's corresponding **CSTATE** trees. This tree represents the user's state footprint across all contracts they've interacted with.
- **CSTATE (Contract State Tree):** The most granular level. This tree is *specific to a single user AND a single contract*. It holds the actual state variables (storage slots) pertinent to that user within that contract. *This is where smart contract logic primarily operates.*
- **GCON (Global Contract Tree):** Stores global information about deployed contracts via **CLEAF** nodes (Contract Leaf).
- **CLEAF (Contract Leaf):** Contains the deployer's identifier hash, the root of the contract's Function Tree (**CFT**), and the required height (size) for its associated **CSTATE** trees.

- **CFT (Contract Function Tree):** *Per-contract* tree whitelisting executable functions. Maps Function IDs to the ZK circuit fingerprint of the corresponding `DapenContractFunctionCircuit`, ensuring only verified code can be invoked.
- **URT (User Registration Tree):** Commits to user public keys during the registration process, ensuring uniqueness and linking registrations to cryptographic identities.
- **Other Trees:** Dedicated global trees handle deposits (`GDT`), withdrawals (`GWT`), event data (`EDATA` - conceptually), etc.

2.2 PARTH Interaction Rules: Enabling Concurrency

The genius of PARTH lies in its strict state access rules:

1. **Localized Writes:** A transaction initiated by User A **can only modify state within User A's own trees**. This typically involves changes within one or more of User A's `CSTATE` trees, which then requires updating the corresponding leaves in User A's `UCON` tree, ultimately updating User A's `ULEAF` in the `GUSR`. **Crucially, User A cannot directly alter User B's `CSTATE`, `UCON`, or `ULEAF`.**
2. **Historical Global Reads:** A transaction can read *any* state from the blockchain (e.g., User B's balance stored in their `ULEAF`, a variable in User B's `CSTATE` via User B's `UCON` root, or global contract data from `GCON`). However, these reads **always access the state as it was finalized in the previous block's `CHKP` root**. The current block's ongoing, parallel state changes are invisible to concurrent transactions.

2.3 The Scalability Breakthrough: Conflict-Free Parallelism

These rules eliminate the core bottleneck of traditional blockchains:

- **No Write Conflicts:** Since users only write to their isolated state partitions, transactions from different users within the same block cannot conflict.

- **No Read-Write Conflicts:** Reading only the previous, immutable block state prevents race conditions where one transaction's read is invalidated by another's concurrent write.
- **Massively Parallel Execution & Proving:** The PARTH architecture guarantees that the execution of transactions (CFCs) and the generation of their initial proofs (UPS) for different users are independent processes that can run entirely in parallel without requiring locks or complex synchronization.

3. End-to-End ZK Proof System: Securing Parallelism

Psy employs a multi-layered, recursive ZK proof system to cryptographically guarantee the integrity of every state transition, even those occurring concurrently.

3.1 Contract Function Circuits (CFCs) & Dapen (DPN)

- **Role:** Encapsulate the verifiable logic of smart contracts. They define the allowed state transitions within a user's `CSTATE` for that contract.
- **Technology:** Developed using high-level languages (TypeScript/JavaScript) and compiled into ZK circuits (`DapenContractFunctionCircuit`) via the **Dapen (DPN)** toolchain.
- **Execution:** run **locally** during a User Proving Session (UPS). A ZK proof is generated for each CFC execution, attesting that the logic was followed correctly given the inputs and starting state provided in its context.

3.2 User Proving Session (UPS)

- **Role:** Enables users (or their delegates) to locally process a sequence of their transactions for a block, generating a single, compact "End Cap" proof that summarizes and validates their entire session activity.
- **Process:**

1. **Initialization (`UPSStartSessionCircuit`)**: Securely anchors the session's starting state to the last globally finalized `CHKP` root and the user's corresponding `ULEAF` .
 2. **Transaction Steps (`UPSCFCStandardTransactionCircuit` , etc.)**: For each transaction:
 - Verifies the ZK proof of the **locally executed CFC** (from step 3.1).
 - Verifies the ZK proof of the **previous UPS step** (ensuring recursive integrity).
 - Proves that the **UPS state delta** (changes to `UCON` root, debt trees, tx count/stack) correctly reflects the verified CFC's outcomes.
 3. **Finalization (`UPSStandardEndCapCircuit`)**: Verifies the last UPS step, verifies the user's ZK signature authorizing the session, checks final conditions (e.g., all debts cleared), and outputs the net state change (`start_user_leaf_hash` -> `end_user_leaf_hash`) and aggregated statistics (`GUTASats`).
- **Scalability Impact**: Drastically reduces the on-chain verification burden. Instead of verifying every transaction individually, the network only needs to verify one aggregate End Cap proof per active user per block.

3.3 Network Aggregation (Realms, Coordinators, GUTA)

The network takes potentially millions of End Cap proofs and efficiently aggregates them in parallel using a hierarchy of specialized ZK circuits.

- **Realms**:
 - **Role**: Distributed ingestion and initial aggregation points for user state changes (`GUSR`), sharded by user ID ranges.
 - **Function**:
 1. Receive End Cap proofs from users within their range.
 2. Verify these proofs using circuits like `GUTAVerifySingleEndCapCircuit` (for individual proofs) or `GUTAVerifyTwoEndCapCircuit` (for pairs). These circuits use the `VerifyEndCapProofGadget` internally to check the End Cap

- proof validity, fingerprint, and historical checkpoint link, outputting a standardized `GlobalUserTreeAggregatorHeader`.
3. Recursively aggregate the resulting GUTA headers using circuits like `GUTAVerifyTwoGUTACircuit`, `GUTAVerifyLeftGUTARightEndCapCircuit`, or `GUTAVerifyLeftEndCapRightGUTACircuit`. These employ `VerifyGUTAProofGadget` to check sub-proofs and `TwoNCASStateTransitionGadget` (or line proof logic) to combine state transitions.
 4. If necessary, use `GUTAVerifyGUTAToCapCircuit` (which uses `VerifyGUTAProofToLineGadget`) to bring a proof up to the Realm's root level.
 5. Handle periods of inactivity using `GUTANoChangeCircuit`.
 6. Submit the final aggregated GUTA proof for their user segment (representing the net change at the Realm's root node in `GUSR`) to the Coordinator layer.
- **Scalability Impact:** Distributes the initial proof verification and `GUSR` aggregation load.

- **Coordinators:**

- **Role:** Higher-level aggregators combining proofs *across* Realms and *across* different global state trees.
- **Function:**
 1. Verify aggregated GUTA proofs from multiple Realms (using `GUTAVerifyTwoGUTACircuit` or similar, employing `VerifyGUTAProofGadget`).
 2. Verify proofs for global operations:
 - User Registrations (`BatchAppendUserRegistrationTreeCircuit`).
 - Contract Deployments (`BatchDeployContractsCircuit`).
 3. Combine these different types of state transitions using aggregation circuits like `VerifyAggUserRegistartionDeployContractsGUTACircuit`, ensuring consistency relative to the same checkpoint.
 4. Prepare the final inputs for the block proof circuit (`PsyCheckpointStateTransitionCircuit`).

- **Scalability Impact:** Manages the convergence of parallel proof streams from different state components and realms.
- **Proving Workers:**
 - **Role:** The distributed computational workforce of the network. They execute the intensive ZK proof generation tasks requested by Realms and Coordinators.
 - **Function:** Stateless workers that fetch proving jobs (circuit type, input witness ID, dependency proof IDs) from a queue, retrieve necessary data from the Node State Store, generate the required ZK proof, and write the result back to the store.
 - **Scalability Impact:** The core engine of computational scalability. The network's proving capacity can be scaled horizontally simply by adding more (potentially permissionless) Proving Workers.

3.4 Final Block Proof Generation

- **Role:** Creates the single, authoritative ZK proof for the entire block.
- **Circuit:** `PsyCheckpointStateTransitionCircuit`.
- **Function:** Takes the final aggregated state transition proofs from the Coordinator layer (representing net changes to `GUSR`, `GCON`, `URT`, etc.). Verifies these proofs. Computes the new global state roots and combines them with aggregated block statistics (`PsyCheckpointLeafStats`) to form the new `PsyCheckpointLeaf`. Proves the correct update of the `CHKP` tree by appending this new leaf hash. **Critically, it verifies that the entire process correctly transitioned from the state defined by the *previous block's finalized* `CHKP` root (provided as a public input).**
- **Output:** A highly succinct ZK proof whose public inputs are the previous `CHKP` root and the new `CHKP` root.

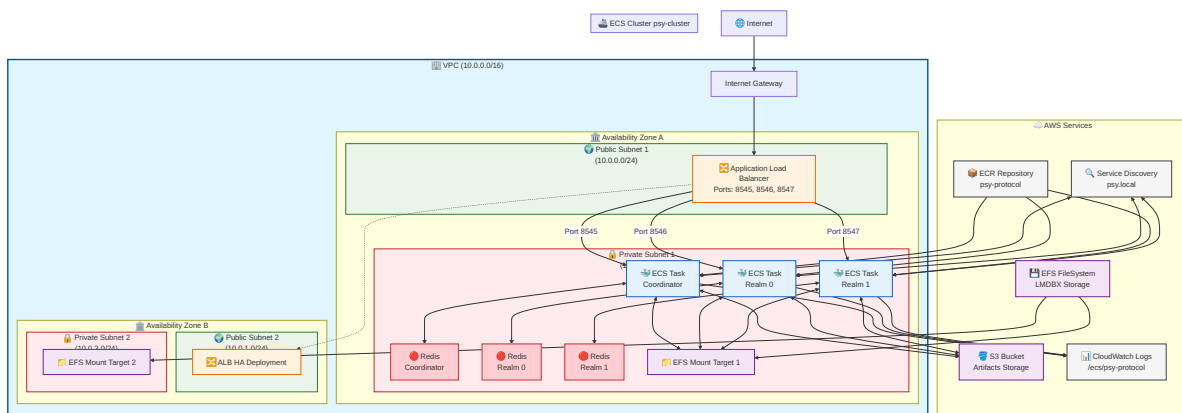
4. Node State Architecture: Redis & KVQ Backend

Supporting this massive parallelism requires a high-performance, shared backend infrastructure.

- **Core Technology:** Psy leverages **Redis**, a distributed in-memory key-value store known for its speed and scalability, as the primary backend. Redis Clusters allow horizontal scaling of storage and throughput.
- **Abstraction Layer (KVQ):** A custom Rust library providing traits and adapters (`KVQSerializable` , `KVQStandardAdapter` , model types like `KVQFixedConfigMerkleTreeModel`) for structured, type-safe interaction with Redis. It simplifies key generation, serialization, and potentially caching.
- **Logical Components:**
 - **Proof Store (`ProofStoreFred` , implements `QProofStore...` traits):** Stores ZK proofs and input witnesses, keyed by `QProvingJobDataID` . Uses Redis Hashes (`HSET` , `HGET`) and potentially atomic counters (`HINCRBY`) for managing job dependencies.
 - **State Store (Models implementing `PsyCoordinatorStore...` , `PsyRealmStore...` traits):** Stores the canonical blockchain state, primarily Merkle tree nodes (`KVQMerkleNodeKey`) and leaf data (`UserLeaf` , `ContractLeaf` , etc.). Uses standard Redis keys managed via KVQ models.
 - **Queues (`CheckpointDrainQueue` , `CheckpointHistoryQueue` , `WorkerEventQueue` traits):** Implement messaging between components. Uses Redis Lists (`LPUSH` , `LPOP` / `BLPOP` , `LRANGE`) for job queues and potentially Pub/Sub or simple keys/sorted sets for history tracking and notifications. `ProofStoreFred` often implements these queue interaction traits.
 - **Local Caching (`PsyCmdStoreWithCache` , used within `PsyLocalProvingSessionStore`):** Provides an in-memory cache layer for frequently accessed state data (e.g., contract definitions, user leaves from the previous block) during local UPS execution or within Realm/Coordinator nodes, reducing load on the central Redis cluster.

- **Scalability:**

- **Redis Performance:** Provides low-latency access required for coordinating many workers.
- **Horizontal Scaling:** Redis clusters can scale to handle increased load.
- **Concurrency:** Redis handles concurrent connections from numerous DPN nodes.
- **Decoupling:** Proving computation (Workers) is separated from state storage and coordination (Redis + Control Nodes), allowing independent scaling.



5. Security Guarantees

Psy's security rests on multiple pillars:

1. **ZK Proof Soundness:** Mathematical guarantee that invalid computations or state transitions cannot produce valid proofs.
2. **Circuit Whitelisting:** State trees (GUSR , GCON , CFT , etc.) can only be modified by proofs generated from circuits whose fingerprints are present in designated whitelist Merkle trees. This prevents unauthorized code execution. Aggregation circuits enforce these checks recursively.
3. **Recursive Verification:** Each layer of aggregation cryptographically verifies the proofs from the layer below.
4. **Checkpoint Anchoring:** The final block circuit explicitly links the new state to the previous block's verified **CHKP** root, creating an unbroken

chain of state validity.

6. Conclusion: A New Era of Blockchain Scalability

Psy's architecture is a fundamental departure from sequential blockchain designs. By leveraging the **PARTH state model** for conflict-free parallel execution and securing it with an **end-to-end recursive ZKP system**, Psy achieves true horizontal scalability. The intricate dance between local user proving (UPS/CFC), distributed network aggregation (Realms/Coordinators/GUTA), and a scalable backend (Redis/KVQ) allows the network's throughput to grow with the addition of computational resources (Proving Workers), paving the way for decentralized applications demanding high performance and robust security.

Realm & GUTA Gadgets

These gadgets are used within the circuits run by the network's Realm and Coordinator nodes for aggregating proofs.

GUTASTatsGadget

- **File:** `guta_stats_rs.txt`
 - **Purpose:** Represents and aggregates key statistics during GUTA processing (fees, operations counts, slots modified).
 - **Technical Function:** Data structure holding targets for stats. Provides `combine_with` method for additive aggregation and `to_hash` for commitment.
 - **Inputs/Witness:** Targets for `fees_collected`, `user_ops_processed`, `total_transactions`, `slots_modified`.
 - **Outputs/Computed:** Combined stats (via `combine_with`), hash of stats (`to_hash`).
 - **Constraints:** `combine_with` uses addition constraints. `to_hash` uses packing/hashing.
 - **Assumptions:** Assumes input target values are correct.
 - **Role:** Tracks operational metrics through the aggregation tree.
-

GlobalUserTreeAggregatorHeaderGadget

- **File:** `guta_header_rs.txt`
- **Purpose:** Defines the standard public input structure for all GUTA-related aggregation circuits. Encapsulates the result of an aggregation step.
- **Technical Function:** Data structure holding `guta_circuit_whitelist` root, `checkpoint_tree_root`, the `state_transition` (`SubTreeNodeStateTransitionGadget`) for the `GUSR` tree segment

covered, and aggregated `stats` (`GUTASStatsGadget`). Provides `to_hash` method.

- **Inputs/Witness:** Component targets/gadgets.
 - **Outputs/Computed:** Hash of the header (`to_hash`).
 - **Constraints:** `to_hash` combines hashes of components.
 - **Assumptions:** Assumes input components are correctly formed/verified.
 - **Role:** Standardizes the interface between recursive GUTA circuits, ensuring consistent information propagation and verification.
-

VerifyEndCapProofGadget

- **File:** `verify_end_cap_rs.txt`
- **Purpose:** Verifies a user's submitted End Cap proof (output of UPS Phase 1) at the entry point of the GUTA aggregation (typically within a Realm node circuit).
- **Technical Function:** Verifies the End Cap ZK proof, checks its fingerprint against the known constant, matches public inputs against witness data (`result/stats`), verifies the user's claimed checkpoint root against a historical checkpoint proof, and translates the result into a `GlobalUserTreeAggregatorHeaderGadget`.
- **Inputs/Witness:**
 - `end_cap_result_gadget`, `guta_stats`: Witness for claimed outputs.
 - `checkpoint_historical_merkle_proof`: Witness proving user's `checkpoint_tree_root_hash` was valid historically.
 - `verifier_data`, `proof_target`: The End Cap proof itself and its verifier data.
 - `known_end_cap_fingerprint_hash`: Constant parameter.
- **Outputs/Computed:** Implements `ToGUTAHeader` to output a `GlobalUserTreeAggregatorHeaderGadget`.
- **Constraints:**
 - Verifies `proof_target` using `verifier_data`.
 - Computes fingerprint from `verifier_data`, connects to `known_end_cap_fingerprint_hash`.

- Computes expected public inputs hash from `end_cap_result_gadget` and `guta_stats`, connects to `proof_target.public_inputs`.
 - Verifies `checkpoint_historical_merkle_proof` using `HistoricalRootMerkleProofGadget`.
 - Connects `historical_proof.historical_root` to `end_cap_result.checkpoint_tree_root_hash`.
 - Constructs output GUTA header using `historical_proof.current_root` as the `checkpoint_tree_root`, deriving the state transition from `end_cap_result` (leaf hashes and user ID), and using the verified `guta_stats`.
 - **Assumptions:** Assumes witness data is valid initially. Assumes `known_end_cap_fingerprint_hash` and input `default_guta_circuit_whitelist` are correct.
 - **Role:** Securely ingests a user's proven session result into the GUTA aggregation, validating it against global rules and historical state before converting it to the standard GUTA format.
-

VerifyGUTAProofGadget

- **File:** `verify_guta_proof_rs.txt`
- **Purpose:** Verifies a GUTA proof generated by a lower level in the aggregation hierarchy (e.g., verifying a Realm's proof at the Coordinator level, or verifying sub-realm proofs within a Realm).
- **Technical Function:** Verifies the input GUTA ZK proof, checks its fingerprint against the GUTA circuit whitelist, and ensures its public inputs match the claimed GUTA header witness.
- **Inputs/Witness:**
 - `guta_proof_header_gadget` : Witness for the claimed header of the proof being verified.
 - `guta_whitelist_merkle_proof` : Witness proving the sub-proof's circuit fingerprint is in the GUTA whitelist.
 - `verifier_data`, `proof_target` : The GUTA proof and its verifier data.

- **Outputs/Computed:** The verified `guta_proof_header_gadget`.
 - **Constraints:**
 - Verifies `proof_target` using `verifier_data`.
 - Computes fingerprint from `verifier_data`.
 - Verifies `guta_whitelist_merkle_proof`.
 - Connects `guta_proof_header.guta_circuit_whitelist` to `whitelist_proof.root`.
 - Computes expected public inputs hash from `guta_proof_header`, connects to `proof_target.public_inputs`.
 - Connects `whitelist_proof.value` to computed fingerprint.
 - **Assumptions:** Assumes witness data is valid initially.
 - **Role:** The core recursive verification step for GUTA aggregation circuits. Ensures that only valid proofs generated by allowed GUTA circuits are incorporated into higher levels of aggregation.
-

TwoNCAStateTransitionGadget

- **File:** `two_nca_state_transition_rs.txt`
- **Purpose:** Combines the `GUSR` state transitions from two independent GUTA proofs (A and B) that modify different branches of the tree, computing the resulting state transition at their Nearest Common Ancestor (NCA).
- **Technical Function:** Uses `UpdateNearestCommonAncestorProofOptGadget` to verify the NCA proof witness against the state transitions provided by the input GUTA headers (`a_header` , `b_header`). Combines statistics and outputs a new GUTA header for the NCA node.
- **Inputs/Witness:**
 - `a_header` , `b_header` : Verified GUTA headers from the two child proofs.
 - `UpdateNearestCommonAncestorProof` : Witness containing NCA proof data (partial or full).
- **Outputs/Computed:** `new_guta_header` representing the transition at the NCA.
- **Constraints:**

- Connects `a_header.checkpoint_tree_root == b_header.checkpoint_tree_root`.
 - Connects `a_header.guta_circuit_whitelist == b_header.guta_circuit_whitelist`.
 - Connects `a_header.state_transition` fields (old/new value, index, level) to `update_nca_proof_gadget.child_a` fields.
 - Connects `b_header.state_transition` fields to `update_nca_proof_gadget.child_b` fields.
 - Computes `new_stats = a_header.stats.combine_with(b_header.stats)`.
 - Constructs `new_guta_header` using whitelist/checkpoint from children, the NCA state transition details from `update_nca_proof_gadget`, and `new_stats`.
 - **Assumptions:** Assumes input headers `a_header` and `b_header` have already been verified. Assumes the NCA proof witness is valid initially.
 - **Role:** Enables efficient parallel aggregation by merging results from independent subtrees using cryptographic proofs of their combined effect at the parent node.
-

GUTAHeaderLineProofGadget

- **File:** `guta_line_rs.txt`
- **Purpose:** Propagates a GUTA state transition upwards along a direct path in the `GUSR` tree (when there's only one child updating that path segment).
- **Technical Function:** Uses `SubTreeNodeTopLineGadget` to recompute the Merkle root hash from the child's transition level up to a specified higher level (e.g., Realm root or global root), using sibling hashes provided as witness.
- **Inputs/Witness:**
 - `child_proof_header` : The verified GUTA header from the lower level.
 - `siblings` : Witness array of Merkle sibling hashes for the path.
 - Height parameters.

- **Outputs/Computed:** `new_guta_header` with the state transition updated to reflect the higher level.
 - **Constraints:** Relies on `SubTreeNodeTopLineGadget` 's internal Merkle hashing constraints.
 - **Assumptions:** Assumes `child_proof_header` is verified. Assumes `siblings` witness is correct.
 - **Role:** Efficiently moves a verified state transition up the tree hierarchy when merging (NCA) is not required.
-

VerifyGUTAProofToLineGadget

- **File:** `verify_guta_proof_to_line_rs.txt`
 - **Purpose:** Combines verifying a lower-level GUTA proof with immediately propagating its state transition upwards using a line proof.
 - **Technical Function:** Orchestrates `VerifyGUTAProofGadget` followed by `GUTAHeaderLineProofGadget`.
 - **Inputs/Witness:** Combines witnesses for both sub-gadgets (proof, header, whitelist proof, siblings).
 - **Outputs/Computed:** `new_guta_header` at the top of the line.
 - **Constraints:** Instantiates and connects the two sub-gadgets.
 - **Assumptions:** Relies on sub-gadget assumptions.
 - **Role:** A common pattern gadget simplifying the verification and upward propagation of a single GUTA proof branch.
-

(Coordinator/GUTA Gadgets related to User Registration:

`GUTARegisterUserCoreGadget`, `GUTARegisterUserFullGadget`, `GUTARegisterUsersGadget`, `GUTAOnlyRegisterUsersGadget`, `GUTARegisterUsersBatchGadget` - Descriptions remain largely the same as the previous detailed version, focusing on GUSR tree updates and User Registration Tree checks.)

GUTANoChangeGadget

- **File:** `guta_no_change_gadget_rs.txt`
- **Purpose:** Creates a GUTA header signifying that the `GUSR` tree state did *not* change for this block/subtree, while still potentially updating the referenced `checkpoint_tree_root`.
- **Technical Function:** Verifies a checkpoint proof to get the current `checkpoint_tree_root` and the corresponding `user_tree_root` from the checkpoint leaf. Constructs a GUTA header with a "no-op" state transition (old=new=user_tree_root at level 0) and zero stats.
- **Inputs/Witness:**
 - `guta_circuit_whitelist`: Input constant/parameter.
 - `checkpoint_tree_proof`: Witness proving `checkpoint_leaf` existence.
 - `checkpoint_leaf_gadget`: Witness for the `PsyCheckpointLeafCompactWithStateRoots`.
- **Outputs/Computed:** `new_guta_header` (indicating no GUSR change).
- **Constraints:** Verifies checkpoint proof (`MerkleProofGadget`). Verifies consistency between proof value and leaf hash. Constructs header with no-op transition and zero stats.
- **Assumptions:** Assumes witness proof/leaf data is valid initially. Assumes input `guta_circuit_whitelist` is correct.
- **Role:** Allows the GUTA aggregation structure to remain consistent and synchronized with the main Checkpoint Tree advancement even during periods where no user state relevant to GUTA was modified.

Psy Protocol Wiki: Achieving Scalability with PARTH and ZK Proofs

1. Introduction: The Scalability Challenge and Psy's Solution

Traditional blockchains often face a **serial execution bottleneck**. Transactions are typically processed one after another within a single state machine context. Adding more validator nodes doesn't necessarily increase the overall transaction processing capacity (TPS), as they all work on the same sequential task list. Attempts at parallel execution often introduce complexity, race conditions, and potential state inconsistencies.

Psy (Quantum Entangled Data) tackles this fundamental limitation through two core innovations:

1. **PARTH (Parallelizable Account-based Recursive Transaction History) Architecture:** A novel way of organizing blockchain state that inherently allows for parallel processing of transactions from different users within the same block.
2. **End-to-End Zero-Knowledge Proofs (ZKPs):** A sophisticated system of ZK circuits that rigorously verify every state transition, ensuring the security and consistency of the parallel execution enabled by PARTH.

This wiki page details the journey of a transaction from user initiation to final block inclusion, focusing on the ZK circuits involved, the assumptions they make, what they prove, and how these assumptions are systematically verified and discharged throughout the process.

2. The PARTH Architecture: Foundation for Parallelism

PARTH reorganizes blockchain state into a hierarchy of Merkle trees, enabling fine-grained state access and modification control:

- **Per-User, Per-Contract State (CSTATE):** Each user maintains a *separate* Merkle tree (CSTATE) for their specific state within *each* smart contract they interact with.
- **User Contract Tree (UCON):** Aggregates all CSTATE roots for a single user, representing their state across all contracts.
- **Global User Tree (GUSR):** Aggregates all UCON roots, representing the state of all users.
- **Global Contract Tree (GCON):** Represents the global state related to contract code definitions and metadata.
- **User Registration Tree:** Tracks registered users and their public keys (relevant for GUTARRegisterUserCircuit).
- **Checkpoint Tree (CHKP):** The top-level tree. Its root hash serves as a cryptographic snapshot (checkpoint) of the *entire verifiable blockchain state* at a specific block height.

Key PARTH Rules Enabling Scalability:

1. **Write Locally:** A transaction initiated by User A can **only modify (write to)** the state within User A's own trees (their various CSTATE s and subsequently their UCON root).
2. **Read Globally (Previous State):** A transaction can **read** state from *any* tree (User A's, User B's, GCON , etc.), but it reads the state **as it was finalized at the end of the previous block** (anchored by the previous block's CHKP root).

Because write operations are isolated and reads access immutable past state, transactions from *different users* within the *same block* operate independently and cannot conflict. This architectural design is the cornerstone of Psy's horizontal scalability.

3. The Big Picture: End-to-End ZK Proof Flow

The process of validating transactions and building a block involves three main phases, each relying on specific ZK circuits:

1. **Phase 1: User Proving Session (UPS) - Local Execution & Proving:** The user (or a delegated prover) executes their transactions locally and generates a chain of recursive ZK proofs culminating in a single "End Cap" proof for their activity within the block.
2. **Phase 2: Global User Tree Aggregation (GUTA) - Parallel Network Execution:** The Psy network's Decentralized Proving Network (DPN) takes End Cap proofs (and other GUTA-related proofs like registrations) from many users and aggregates them in parallel using specialized GUTA circuits.
3. **Phase 3: Final Block Proof Generation:** A final aggregation step combines the top-level GUTA proof (representing all user state changes) with proofs for other global state changes (like contract deployments) into a single block proof anchored to the previous block's state.

4. Phase 1: User Proving Session (UPS) Circuits

This phase occurs locally, building a user-specific proof chain.

4.1. `UPSSessionStartCircuit`

- **Purpose:** Initializes the proving session for a user, establishing a secure starting point based on the last globally finalized block state.
- **Core Gadget:** `UPSSessionStartStepGadget`
- **Input Data:** `UPSSessionStartStepInput` (contains the target starting `UserProvingSessionHeader`, the corresponding `PsyCheckpointLeaf` and `PsyCheckpointGlobalStateRoots` from the last block, and Merkle proofs (`checkpoint_tree_proof`, `user_tree_proof`) linking them together).
- **What it Proves:**

- **Consistency of Initial State:** The provided `UserProvingSessionHeader.session_start_context` is consistent with the provided `PsyCheckpointLeaf`, `PsyCheckpointGlobalStateRoots`, and the user's leaf (`start_session_user_leaf`) within the `user_tree_root` (part of `PsyCheckpointGlobalStateRoots`). It verifies that the user leaf, global roots, and checkpoint leaf all correctly correspond to the provided `checkpoint_tree_root` via the Merkle proofs.
- **Correct Initialization:** The `UserProvingSessionHeader.current_state` is correctly initialized based on the `session_start_context` (e.g., `user_leaf.last_checkpoint_id` is updated, `deferred_tx_debt_tree_root` and `inline_tx_debt_tree_root` are empty hashes, `tx_count` is zero, `tx_hash_stack` is an empty hash).
- **Header Integrity:** The hash of the entire `ups_header` is correctly computed.
- **Proof Tree Anchor:** The final public output hash combines the `ups_header` hash with the `empty_ups_proof_tree_root` (a known constant representing the start of this session's proof tree), using `compute_tree_aware_proof_public_inputs`.
- **Assumptions Made:**
 1. The witness data (`UPSStartStepInput`) provided by the user (fetched from querying the last finalized block state) is accurate *at the start of verification*.
 2. The globally known constant `empty_ups_proof_tree_root` (hash of an empty Merkle tree of height `UPS_SESSION_PROOF_TREE_HEIGHT`) is correct.
 3. The *previous block's* `checkpoint_tree_root` (implicitly contained within the input witness) is valid (this is the key assumption passed *into* the entire UPS).
- **How Assumptions are Discharged:**
 - Assumption 1 is checked by the circuit's constraints, ensuring internal consistency between the header, leaf data, global roots, and Merkle proofs. If the witness data is inconsistent, proof generation fails.
 - Assumption 2 is a system parameter.

- Assumption 3 is *not* discharged here; it's carried forward implicitly by using the state derived from that root as the starting point.
- **Contribution to Horizontal Scalability:** Provides a secure, independent starting point for each user's session based on the common, finalized global state, allowing many users to start their sessions in parallel.
- **High-Level Functionality:** Securely begins a user's batch of transactions for the current block context.

4.2. UPSCFCStandardTransactionCircuit

- **Purpose:** Processes a standard Contract Function Call (CFC) transaction, extending the user's recursive proof chain. Executed potentially multiple times per session.
- **Core Gadgets:** `VerifyPreviousUPSStepProofInProofTreeGadget` , `UPSVerifyCFCStandardStepGadget`
- **Input Data:** `UPSCFCStandardTransactionCircuitInput` (contains info about the previous UPS step's proof and header (`verify_previous_ups_step`), and details for the current CFC step (`standard_cfc_step`)).
- **What it Proves:**
 - **Previous Step Validity:** The ZK proof (`previous_step_proof`) provided for the previous UPS step (either the `UPSStartSessionCircuit` or another `UPSCFC...Circuit`) is valid.
 - **Previous Step Whitelist:** The previous step's proof was generated by a circuit whose fingerprint is present in the `ups_circuit_whitelist_root` specified in the `previous_step_header` .
 - **Previous Step Linkage:** The public inputs hash of the `previous_step_proof` correctly corresponds to the provided `previous_step_header` and the `previous_proof_tree_root` .
 - **CFC Proof Validity & Inclusion:** The ZK proof for the current CFC execution (`verify_cfc_proof_input`) is valid and is correctly included in the *current* `current_proof_tree_root` .
 - **Contract Function Validity:** The CFC proof corresponds to a function whose details (`cfc_inclusion_proof`) are correctly

included in the Global Contract Tree (`GCON`) root referenced within the session's checkpoint context (`checkpoint_state`).

- **State Transition Correctness:** Executing the CFC logically transitions the state represented by `previous_step_header.current_state` to the state represented by `new_header_gadget.current_state` . This includes updates to the user's `user_leaf` (specifically the `user_contract_tree_root`), potentially the `deferred_tx_debt_tree_root` or `inline_tx_debt_tree_root` , the `tx_hash_stack` , and incrementing the `tx_count` . The `session_start_context` and `ups_step_circuit_whitelist_root` remain unchanged.
- **Proof Tree Update:** The output public inputs correctly combine the hash of the `new_header_gadget` with the `current_proof_tree_root` .
- **Assumptions Made:**
 1. The witness data (`UPSCFCStandardTransactionCircuitInput`) for *this specific step* (including the previous step's proof/header, the CFC proof/details, state delta witnesses) is accurate initially.
 2. The `current_proof_tree_root` provided as input correctly represents the root of the session's proof tree *after* including the current CFC proof.
 3. The assumption about the original *starting* `checkpoint_tree_root` (from `UPSSstartSessionCircuit`) is still carried forward implicitly.
- **How Assumptions are Discharged:**
 - Assumption 1 is checked by verifying the previous step's proof, the CFC proof, the contract inclusion proof, and the state delta transition logic based on the witness.
 - Assumption 2 is *not* discharged here; it's passed implicitly to the next UPS step or the End Cap circuit, forming part of the recursive structure.
 - Assumption 3 remains implicitly carried forward.
- **Contribution to Horizontal Scalability:** Allows users to process their transactions sequentially *locally*, independent of other users' activities within the same block. The proof chain maintains self-consistency.
- **High-Level Functionality:** Securely executes and proves a single smart contract interaction within the user's session.

4.3. UPSCFCDeferredTransactionCircuit

- **Purpose:** Processes a transaction that first settles a deferred debt item and *then* executes the main CFC logic.
- **Core Gadgets:** `VerifyPreviousUPSSStepProofInProofTreeGadget` , `UPSVerifyPopDeferredTxStepGadget`
- **Input Data:** `UPSCFCDeferredTransactionCircuitInput`
- **What it Proves:** Everything proven by `UPSCFCStandardTransactionCircuit` , *plus*:
 - **Debt Removal:** A specific deferred transaction was correctly removed from the `deferred_tx_debt_tree` (verified via `ups_pop_deferred_tx_proof` , a `DeltaMerkleProofCore`). The state transition reflects this removal *before* the main CFC logic is applied.
 - **Debt-CFC Link:** The data associated with the removed debt item matches the parameters required by the subsequent CFC execution.
- **Assumptions Made:** Same as `UPSCFCStandardTransactionCircuit` , plus assumes the witness for the `DeltaMerkleProofCore` (debt removal) is accurate initially.
- **How Assumptions are Discharged:** Same as standard, plus verifies the debt removal proof and its consistency with the CFC witness data. The starting state assumption and proof tree root assumption are carried forward.
- **Contribution to Horizontal Scalability:** Local serial execution, same as standard.
- **High-Level Functionality:** Enables settlement of asynchronous/deferred transaction calls within the user's proving flow.

4.4. UPSStandardEndCapCircuit

- **Purpose:** Finalizes the user's entire proving session for the block, verifying the signature and packaging the net result.
- **Core Gadgets:** `UPSEndCapFromProofTreeGadget` (which uses `VerifyPreviousUPSSStepProofInProofTreeGadget` , `AttestProofInTreeGadget` , `UPSEndCapCoreGadget` , `PsyUserProvingSessionSignatureDataCompactGadget`)

- **Input Data:** `UPSEndCapFromProofTreeGadgetInput` (contains info about the last UPS step, the ZK signature proof (`verify_zk_signature_proof_input`), signature parameters (`user_public_key_param`, `nonce`), and `slots_modified`).
- **What it Proves:**
 - **Last Step Validity:** The proof for the *last* UPS transaction step is valid and used an allowed circuit (`verify_previous_ups_step_gadget`).
 - **Signature Proof Validity & Inclusion:** The ZK proof for the user's signature (`verify_zk_signature_proof_gadget`) is valid and is correctly included in the *final* UPS proof tree root (`current_proof_tree_root`).
 - **Signature Authentication:** The public key (`user_public_key_param`) used to verify the signature matches the public key stored in the `previous_step_header.current_state.user_leaf`.
 - **Signature Payload Integrity:** The ZK signature proof attests to a specific payload hash. This circuit proves that this payload hash correctly corresponds to a `PsyUserProvingSessionSignatureDataCompact` structure containing:
 - `start_user_leaf_hash`: Matches the `start_session_user_leaf_hash` from the `previous_step_header.session_start_context`.
 - `end_user_leaf_hash`: Matches the hash of the `previous_step_header.current_state.user_leaf`.
 - `checkpoint_leaf_hash`: Matches the `checkpoint_leaf_hash` from the `previous_step_header.session_start_context`.
 - `tx_stack_hash`: Matches the `tx_hash_stack` from the `previous_step_header.current_state`.
 - `tx_count`: Matches the `tx_count` from the `previous_step_header.current_state`.
 - **Nonce Validity:** The `nonce` used in the signature matches the `nonce` in the `previous_step_header.current_state.user_leaf`. (The state delta within the End Cap implicitly increments the nonce in the *output* `UPSEndCapResultCompact`).

- **Session Completion:** The `deferred_tx_debt_tree_root` and `inline_tx_debt_tree_root` in the `previous_step_header.current_state` are both empty hashes (verified by `end_cap_core_gadget.enforce_signature_constraints`).
 - **Checkpoint Consistency:** The `last_checkpoint_id` in the `previous_step_header.current_state.user_leaf` matches the `checkpoint_id` from the `session_start_context`.
 - **Final Output Structure:** The public inputs of the End Cap proof hash a `UPSEndCapResultCompact` structure containing the `start_user_leaf_hash`, `end_user_leaf_hash`, `checkpoint_tree_root_hash` (all from the final `previous_step_header`), and the `user_id`.
- **Assumptions Made:**
 1. The witness data (`UPSEndCapFromProofTreeGadgetInput`) provided is accurate initially.
 2. System constants like `network_magic`, `DEFERRED_TRANSACTION_TREE_HEIGHT`, `INLINE_TRANSACTION_TREE_HEIGHT` (for deriving empty roots) are correct.
 3. The assumption about the validity of the *starting* `checkpoint_tree_root` (from `UPSStartSessionCircuit`) is *still* carried forward implicitly into the `UPSEndCapResultCompact` output.
- **How Assumptions are Discharged:**
 - Assumption 1 is checked by verifying the last step proof, the signature proof, and ensuring consistency between the signature payload data and the final UPS header state.
 - Assumption 2 relies on correct system setup.
 - Assumption 3 is *not* discharged. The End Cap proof essentially certifies: "Assuming the starting checkpoint X was valid, User Y correctly transitioned their state to Z and authorized it". All *internal* UPS assumptions (like correct proof tree construction, valid intermediate steps, correct CFC execution) have been verified and compressed into this final proof.
- **Contribution to Horizontal Scalability:** Packages the user's entire block activity into a single, efficiently verifiable proof. This proof can be

submitted to the network and processed by the GUTA layer in parallel with End Cap proofs from countless other users.

- **High-Level Functionality:** Securely concludes and authorizes a user's transaction batch, preparing it for network-level aggregation.

5. Phase 2: Global User Tree Aggregation (GUTA) Circuits

The DPN receives End Cap proofs and potentially other state change proofs (like user registrations) and aggregates them in parallel using GUTA circuits. These circuits operate on `GlobalUserTreeAggregatorHeader` structures, which track state transitions within the Global User Tree (`GUSR`).

(Note: The exact circuit implementations are inferred from the gadgets. These circuits follow standard recursive aggregation patterns.)

5.1. `GUTAProcessEndCapCircuit` (Hypothetical)

- **Purpose:** Integrates a validated user End Cap proof into the GUTA hierarchy.
- **Core Gadget:** `VerifyEndCapProofGadget`
- **Input:** An `UPSSStandardEndCapCircuit` proof and its associated `UPSEndCapResultCompact` data. Also needs witness for `checkpoint_historical_merkle_proof` and `guta_stats`.
- **What it Proves:**
 - **End Cap Proof Validity:** The provided End Cap proof is valid and was generated by the correct circuit (`known_end_cap_fingerprint_hash`).
 - **End Cap Result Consistency:** The public inputs hash of the End Cap proof matches the hash of the provided `UPSEndCapResultCompact` data.
 - **Historical Checkpoint Validity:** The `checkpoint_tree_root_hash` claimed in the `UPSEndCapResultCompact` existed as a historical root

within the `checkpoint_historical_merkle_proof.current_root` (which represents the `CHKP` root being targeted by *this* GUTA aggregation step).

- **GUTA Header Output:** Correctly constructs a `GlobalUserTreeAggregatorHeader` where:
 - `guta_circuit_whitelist` is set (likely from input/config).
 - `checkpoint_tree_root` matches `checkpoint_historical_merkle_proof.current_root`.
 - `state_transition` reflects the change from `start_user_leaf_hash` to `end_user_leaf_hash` at the correct `user_id` index (level `GLOBAL_USER_TREE_HEIGHT`).
 - `stats` are populated based on input witness.
- **Assumptions Made:**
 1. Witness data (End Cap proof, result, stats, historical proof) is accurate initially.
 2. The `known_end_cap_fingerprint_hash` constant is correct.
 3. The `guta_circuit_whitelist` root provided (implicitly or explicitly) is correct for this GUTA context.
 4. The `checkpoint_historical_merkle_proof.current_root` provided accurately represents the target `CHKP` root for this aggregation level. (This implicitly carries forward the assumption about the *original* starting checkpoint root's validity).
- **How Assumptions are Discharged:**
 - Assumption 1 is checked by verifying the End Cap proof and the historical checkpoint proof.
 - Assumption 2 is a system parameter.
 - Assumptions 3 and 4 are *not* discharged; they are bundled into the output `GlobalUserTreeAggregatorHeader` and passed upwards to the next GUTA aggregation level.
- **Contribution to Horizontal Scalability:** Allows individual user session results (already proven locally) to be independently verified against historical state and prepared as standardized GUTA inputs for parallel aggregation.
- **High-Level Functionality:** Validates and incorporates user end-of-session proofs into the global state aggregation process.

5.2. GUTARegisterUserCircuit (Hypothetical)

- **Purpose:** Processes the registration of new users, updating the GUSR tree.
- **Core Gadgets:** GUTAOnlyRegisterUsersGadget , GUTARegisterUsersGadget , GUTARegisterUserFullGadget , GUTARegisterUserCoreGadget
- **Input:** Data for users being registered, including proofs linking their public keys to a user_registration_tree_root , and delta proofs for GUSR updates. Also needs the target guta_circuit_whitelist and checkpoint_tree_root .
- **What it Proves:**
 - **Valid Registration:** Each user's provided public_key is present in the user_registration_tree_root .
 - **Correct GUSR Insertion:** For each user, the GUSR tree correctly transitioned at the user_id index from an empty hash to the new PsyUserLeaf hash (initialized with the verified public_key and default_user_state_tree_root). This is verified using delta proofs.
 - **GUTA Header Output:** Correctly constructs a GlobalUserTreeAggregatorHeader representing the combined state transition for all registered users. stats are typically zero for pure registrations. The header uses the input guta_circuit_whitelist and checkpoint_tree_root .
- **Assumptions Made:**
 1. Witness data (registration proofs, user data, delta proofs) is accurate initially.
 2. The input guta_circuit_whitelist and checkpoint_tree_root are correct for this context.
 3. The default_user_state_tree_root constant is correct.
 4. The underlying assumption about the validity of the input checkpoint_tree_root is carried forward.
- **How Assumptions are Discharged:**
 - Assumption 1 is checked by verifying registration proofs and GUSR delta proofs.
 - Assumption 2 & 4 are passed upwards via the output header.
 - Assumption 3 is a system parameter.

- **Contribution to Horizontal Scalability:** Allows batches of user registrations to be processed, potentially in parallel branches of the GUTA aggregation tree.
- **High-Level Functionality:** Securely adds new users to the global system state.

5.3. GUTAAggregationCircuit (Hypothetical - Multiple Variants Possible)

- **Purpose:** The workhorse of parallel aggregation. Combines the results (headers) from two lower-level GUTA proofs (which could originate from End Caps, registrations, or previous aggregations).
- **Core Gadgets:** `VerifyGUTAProofGadget` , `TwoNCASStateTransitionGadget` (for combining different branches), `GUTAHeaderLineProofGadget` (for propagating up a single branch), `GUTASstatsGadget`
- **Input:** Two input GUTA proofs (`ProofWithPublicInputs`) and their corresponding `GlobalUserTreeAggregatorHeader` data. Witness for NCA proofs if needed.
- **What it Proves:**
 - **Input Proof Validity:** Both input GUTA proofs are valid.
 - **Input Proof Whitelist:** Both input proofs were generated by circuits listed in the *same* `guta_circuit_whitelist` (taken from their headers).
 - **Input Proof Checkpoint Consistency:** Both input proofs reference the *same* `checkpoint_tree_root` (taken from their headers).
 - **Input Header Consistency:** The public inputs hash of each input proof matches the hash of its corresponding provided `GlobalUserTreeAggregatorHeader` .
 - **State Transition Combination Logic:** The `state_transition` in the *output* GUTA header correctly combines the `state_transition s` from the two input headers. This involves:
 - Verifying NCA proofs if combining transitions on different branches (`TwoNCASStateTransitionGadget`).
 - Ensuring direct linkage (`old_root` matches `new_root`) if combining transitions on the same path.

- Correctly propagating the transition upwards if only one input represents a change (`GUTAHeaderLineProofGadget`).
 - **Stats Combination:** The `stats` in the output header are the correct sum of the stats from the input headers (`GUTASStatsGadget.combine_with`).
 - **GUTA Header Output:** Correctly constructs the output `GlobalUserTreeAggregatorHeader` using the combined state transition, combined stats, and the common `guta_circuit_whitelist` and `checkpoint_tree_root` from the inputs.
- **Assumptions Made:**
 1. Witness data (input proofs, headers, NCA/sibling proofs) is accurate initially.
 2. The assumptions about the validity of the common `guta_circuit_whitelist` and `checkpoint_tree_root` carried by the *input* proofs are implicitly carried forward.
- **How Assumptions are Discharged:**
 - Assumption 1 is checked by verifying the input proofs, their linkage to the input headers, and the logic for combining state transitions and stats.
 - Assumption 2 is *not* discharged; these common contextual assumptions are passed upwards in the output header.
- **Contribution to Horizontal Scalability:** This is the core mechanism enabling parallel processing. Thousands of these circuits can run concurrently on the DPN, merging branches of the GUTA proof tree structure, dramatically speeding up aggregation compared to sequential processing.
- **High-Level Functionality:** Securely and recursively combines verified state changes from multiple independent sources into larger, aggregated proofs.

5.4. `GUTANoChangeCircuit`

- **Purpose:** Handles intervals where no relevant user state changed (`GUSR` root remains the same) but the network needs to acknowledge the

advancement of the `checkpoint_tree_root`.

- **Core Gadget:** `GUTANoChangeGadget`
- **Input:** Proof (`checkpoint_tree_proof`) that a specific `checkpoint_leaf` exists in the target `checkpoint_tree_root`. The target `guta_circuit_whitelist` root.
- **What it Proves:**
 - **Checkpoint Validity:** The provided `checkpoint_leaf` hash matches the value proven to be in the `checkpoint_tree_proof`.
 - **GUSR Consistency:** The `global_state_roots.user_tree_root` within the `checkpoint_leaf` is used as both the `old_node_value` and `new_node_value` in the output header's `state_transition`.
 - **GUTA Header Output:** Correctly constructs a `GlobalUserTreeAggregatorHeader` with the input `guta_circuit_whitelist`, the input `checkpoint_tree_root`, a state transition showing no change in the `GUSR` root (at index 0, level 0), and zeroed `stats`.
- **Assumptions Made:**
 1. Witness data (checkpoint proof, leaf) is accurate initially.
 2. The input `guta_circuit_whitelist` root is correct for this context.
 3. The assumption about the validity of the input `checkpoint_tree_root` is carried forward.
- **How Assumptions are Discharged:**
 - Assumption 1 is checked by verifying the checkpoint proof.
 - Assumptions 2 and 3 are passed upwards via the output header.
- **Contribution to Horizontal Scalability:** Ensures the GUTA aggregation process can produce valid proofs even for periods of inactivity in user state changes, keeping the aggregation synchronized with the progressing checkpoint tree.
- **High-Level Functionality:** Allows the GUTA layer to represent a "no-op" state transition correctly anchored to an updated checkpoint.

6. Phase 3: Final Block Proof Generation

6.1. Checkpoint Tree "Block" Circuit (Top-Level Aggregation)

- **Purpose:** The final circuit in the chain. It aggregates the ultimate proofs from the top-level state trees (the final GUTA proof for `GUSR`, proofs for `GCON` updates, etc.) and verifies the transition against the previous block's finalized state.
- **Core Logic:** Likely uses variants of `VerifyStateTransitionProofGadget` or similar aggregation gadgets tailored for combining the top-level tree proofs. It takes the previous block's `CHKP` root as a crucial public input.
- **Input:** The final aggregated proof from the GUTA layer (representing all `GUSR` changes), potentially proofs for `GCON` changes (e.g., new contract deployments via `VerifyAggUserRegistrationDeployGuta` logic), and the **previous block's finalized `CHKP` root hash**.
- **What it Proves:**
 - **Top-Level Proof Validity:** The input proofs (e.g., final GUTA proof) are valid and adhere to their respective whitelists and checkpoint contexts.
 - **State Root Consistency:** The final roots of `GUSR`, `GCON`, etc., derived from the input proofs are correctly assembled into the `PsyCheckpointGlobalStateRoots` for the *new* block.
 - **New Checkpoint Leaf Construction:** The new `PsyCheckpointLeaf` is correctly constructed using the new global state roots and other block metadata (e.g., block number, timestamp).
 - **New Checkpoint Root Computation:** The new `CHKP` root hash is correctly computed based on the new leaf and its position in the Checkpoint Tree.
 - **Final State Link:** The **critical verification**: it proves that the state transitions represented by the input proofs (originating from potentially millions of user transactions) correctly and validly transform the state anchored by the **input `previous_block_chkp_root`** into the state represented by the **computed `new_chkp_root`**.
- **Assumptions Made:**

1. The **only** remaining significant external assumption is that the **public input `previous_block_chkp_root` hash is indeed the valid, finalized root hash of the immediately preceding block.**
- **How Assumptions are Discharged:**
 - All assumptions related to internal consistency, circuit whitelists, correct state transitions, user signatures, etc., have been recursively verified and discharged by the preceding UPS and GUTA circuit layers.
 - Assumption 1 is discharged by the consensus mechanism or the verifier. They *know* the hash of the last finalized block and check if the proof's public input matches it. If it matches, the proof demonstrates a valid state transition between the two blocks.
- **Contribution to Horizontal Scalability:** This final step takes the output of the massively parallel GUTA aggregation and produces a single, constant-size ZK proof for the entire block's validity. Verifying this proof is extremely fast, regardless of how many transactions were processed in parallel.
- **High-Level Functionality:** Creates the final, succinct, and efficiently verifiable proof for the entire block, cryptographically linking it to the previous block and enabling trustless verification of the entire chain's state transition.

7. Assumption Reduction Summary: The Journey to Trustlessness

The Psy circuit flow demonstrates a progressive reduction and discharge of assumptions:

1. **Start UPS:** Assumes initial state fetched from the last block is correct *locally* and that the *last block's CHKP root* was valid. Verifies local consistency.
2. **UPS Transactions:** Assumes previous step was valid, assumes current step's witness is correct. Verifies previous proof, verifies current CFC/debt logic, verifies state delta. Passes on proof tree root assumption and the *original* starting CHKP root assumption.

3. **UPS End Cap:** Assumes last step was valid, assumes signature proof/witness correct. Verifies last step, verifies signature proof, verifies consistency between final UPS state and signature payload. Discharges all *internal* UPS assumptions (proof tree validity, intermediate step validity). Outputs a proof carrying only the assumption about the *original* starting CHKP root.
4. **GUTA (Process End Cap/Register User):** Assumes End Cap/Registration proof is valid, assumes historical checkpoint/whitelist context. Verifies input proof against context. Outputs a GUTA header carrying the context assumptions upwards.
5. **GUTA (Aggregation):** Assumes input GUTA proofs are valid and share context. Verifies input proofs, verifies combination logic. Outputs an aggregated GUTA header carrying the common context assumptions upwards.
6. **Final Block Circuit:** Assumes top-level proofs (like final GUTA) are valid. Takes the *previous block's CHKP root* as the *only* major external assumption. Verifies input proofs, verifies construction of the new CHKP root based on inputs. Discharges all remaining contextual assumptions (whitelists, checkpoints derived from the input proofs). The final proof stands as: "IF the previous CHKP root was X, THEN the new CHKP root is validly Y".

This journey transforms broad initial assumptions about state correctness into a single, verifiable dependency on the previously accepted state, underpinned by the mathematical certainty of ZK proofs.

8. Conclusion: Scalability Through Parallelism and Proofs

Psy achieves true horizontal scalability by combining:

1. **PARTH Architecture:** Isolates user state modifications, enabling conflict-free parallel transaction execution *within* a block.
2. **User Proving Sessions (UPS):** Allows users to locally prove their own transaction sequences, offloading initial proving work.

3. **Parallel ZK Aggregation (GUTA & DPN):** Enables the network to verify and combine proofs from millions of users concurrently, overcoming the limitations of sequential processing.
4. **Recursive Proofs:** Compresses vast amounts of computation into succinct, fixed-size proofs, making final block verification extremely efficient.

This document describes the end-to-end flow of circuits involved in processing user transactions and aggregating them into a final block proof within the Psy system. It highlights the assumptions made at each stage and how they are progressively verified, ultimately enabling horizontal scalability.

Phase 1: User Proving Session (UPS) - Local Execution

This phase happens locally on the user's device (or via a delegated prover). The user builds a recursive chain of proofs for their transactions within a single block context.

1. `UPSStartSessionCircuit`

- **Purpose:** Initializes the proving session for a user based on the last finalized blockchain state.
- **What it Proves:**
 - The starting `UserProvingSessionHeader` is valid.
 - This header is correctly anchored to a specific `checkpoint_leaf_hash` which exists at `checkpoint_id` within the `checkpoint_tree_root` from the *last finalized block*.
 - The `session_start_context` within the header accurately reflects the user's state (`start_session_user_leaf_hash` , `user_id` , etc.) as found in the `user_tree_root` associated with the starting checkpoint.
 - The `current_state` within the starting header is correctly initialized (user leaf `last_checkpoint_id` updated, debt trees empty, tx count/stack zero).
- **Assumptions:**
 - The witness data (`UPSStartStepInput`) provided by the user (fetching state from the last block) is correct *initially*. Constraints verify its consistency.
 - The constant `empty_ups_proof_tree_root` (representing the start of the recursive proof tree for this session) is correct.
- **How Assumptions are Discharged:** Internal consistency checks verify the relationships between the provided header, checkpoint leaf, state roots,

user leaf, and the Merkle proofs linking them. The assumption about the *previous block's* `checkpoint_tree_root` being correct is implicitly carried forward, as this circuit uses it as the basis for initialization.

- **Contribution to Horizontal Scalability:** Establishes a user-specific, isolated starting point based on globally finalized state, allowing this session to proceed independently of other users' sessions within the *same* new block.
- **High-Level Functionality:** Securely starts a user's transaction batch processing.

2. `UPSCFCStandardTransactionCircuit` (Executed potentially multiple times)

- **Purpose:** Processes a single standard transaction (contract call) within the user's ongoing session, extending the recursive proof chain.
- **What it Proves:**
 - The ZK proof for the *previous UPS step* is valid.
 - The previous step's proof was generated by a circuit listed in the `ups_circuit_whitelist_root` specified in the previous step's header.
 - The public inputs (header hash) of the previous step's proof match the provided `previous_step_header`.
 - The ZK proof for the *current Contract Function Call (CFC)* exists within the current `current_proof_tree_root`.
 - This CFC proof corresponds to a function registered in the `GCON` tree (via checkpoint context).
 - Executing this CFC correctly transitions the state from the `previous_step_header` to the `new_header_gadget` state (updating `CSTATE -> UCON` root, debt trees, tx count/stack).
- **Assumptions:**
 - The witness data (`UPSCFCStandardTransactionCircuitInput`) for this specific step (CFC proof, state delta witnesses, previous step proof info) is correct initially.
 - The `current_proof_tree_root` provided matches the actual root of the recursive proof tree being built.
- **How Assumptions are Discharged:**

- Verifies the previous step's proof using `VerifyPreviousUPSSStepProofInProofTreeGadget` . This discharges the assumption about the previous step's validity and its public inputs.
- Verifies the CFC proof and its link to the contract state using `UPSVerifyCFCStandardStepGadget` .
- Verifies the state delta logic, ensuring the transition is correct based on witness data.
- The assumption about the `current_proof_tree_root` is passed implicitly to the *next* step or the End Cap circuit.
- **Contribution to Horizontal Scalability:** User processes transactions locally and serially *for themselves*, maintaining self-consistency without interacting with other users' *current* block activity.
- **High-Level Functionality:** Securely executes and proves individual smart contract interactions locally.

3. `UPSCFCDeferredTransactionCircuit` (Executed if applicable)

- **Purpose:** Processes a transaction that settles a deferred debt, then executes the main CFC logic.
- **What it Proves:** Similar to the standard circuit, but *additionally* proves:
 - A specific deferred transaction item was removed from the `deferred_tx_debt_tree` .
 - The item removed corresponds exactly to the call data of the CFC being executed.
 - The subsequent CFC state transition starts from the state *after* the debt was removed.
- **Assumptions:** Same as the standard circuit, plus assumes the witness for the deferred transaction removal proof (`DeltaMerkleProofGadget`) is correct initially.
- **How Assumptions are Discharged:** Verifies previous step proof. Verifies the debt removal proof and its consistency with the CFC call data. Verifies the subsequent state delta.
- **Contribution to Horizontal Scalability:** Same as standard transaction circuit (local serial execution).
- **High-Level Functionality:** Enables settlement of asynchronous transaction debts within the local proving flow.

4. `UPSSStandardEndCapCircuit`

- **Purpose:** Finalizes the user's entire proving session for the block.
- **What it Proves:**
 - The proof for the *last UPS transaction step* is valid and used an allowed circuit.
 - The ZK proof for the user's signature (authorizing the session) is valid and exists in the same UPS proof tree.
 - The signature corresponds to the user's registered public key (derived from signature proof parameters).
 - The signature payload (`PsyUserProvingSessionSignatureDataCompact`) correctly reflects the session's start/end user leaves, checkpoint, final tx stack, and tx count.
 - The nonce used in the signature is valid (incremented).
 - The final UPS state shows both `deferred_tx_debt_tree_root` and `inline_tx_debt_tree_root` are empty (all debts settled).
 - The `last_checkpoint_id` in the final user leaf matches the session's `checkpoint_id` and has progressed correctly.
 - (If aggregation proof verification included): The UPS proof tree itself was constructed using circuits from a known `proof_tree_circuit_whitelist_root`.
- **Assumptions:**
 - Witness data (`UPSEndCapFromProofTreeGadgetInput`, potentially aggregation proof witness) is correct initially.
 - Known constants (`network_magic`, empty debt roots, `known_ups_circuit_whitelist_root`, `known_proof_tree_circuit_whitelist_root`) are correct.
- **How Assumptions are Discharged:**
 - Verifies the last UPS step proof.
 - Verifies the ZK signature proof.
 - Connects signature data to the final UPS header state.
 - Checks nonce, checkpoint ID, empty debt trees.
 - Verifies proofs against whitelists using provided roots.
 - The *output* of this circuit (the End Cap proof) now implicitly carries the assumption that the *starting* `checkpoint_tree_root` (used in the

`UPSSessionCircuit`) was correct. All *internal* UPS assumptions have been discharged.

- **Contribution to Horizontal Scalability:** Creates a single, verifiable proof representing *all* of a user's activity for the block. This proof can now be processed in parallel with proofs from other users by the GUTA layer.
- **High-Level Functionality:** Securely concludes a user's transaction batch, authorizes it, and packages it for network aggregation.

Phase 2: Global User Tree Aggregation (GUTA) - Parallel Network Execution

The Decentralized Proving Network (DPN) takes End Cap proofs (and potentially other GUTA proofs like user registrations) from many users and aggregates them in parallel. This involves specialized GUTA circuits.

(Note: The provided files focus heavily on UPS and GUTA gadgets. The exact structure of the GUTA circuits using these gadgets is inferred but follows standard recursive proof aggregation patterns.)

Example GUTA Circuits (Inferred):

5. `GUTAProcessEndCapCircuit` (Hypothetical)

- **Purpose:** To take a user's validated `UPSStandardEndCapCircuit` proof and integrate its state change into the GUTA proof hierarchy.
- **Core Logic:** Uses `VerifyEndCapProofGadget`.
- **What it Proves:**
 - The End Cap proof is valid and used the correct circuit (`known_end_cap_fingerprint_hash`).
 - The `checkpoint_tree_root` claimed by the user in the End Cap result existed historically.
 - Outputs a standard `GlobalUserTreeAggregatorHeader` representing the user's `GUSR` tree state transition (start leaf hash -> end leaf hash at the user's ID index) and stats.
- **Assumptions:**
 - Witness (End Cap proof, result, stats, historical proof) is correct initially.

- The `known_end_cap_fingerprint_hash` constant is correct.
 - A `default_guta_circuit_whitelist` root is provided or known.
- **How Assumptions are Discharged:** Verifies the End Cap proof and historical checkpoint proof. Packages the result into a standard GUTA header. The assumption about the `default_guta_circuit_whitelist` is passed upwards. The assumption about the *current* `checkpoint_tree_root` (from the historical proof) is passed upwards.
- **Contribution to Horizontal Scalability:** Allows individual user session results to be verified independently and prepared for parallel aggregation.
- **High-Level Functionality:** Validates and incorporates user end-of-session proofs into the global aggregation process.

6. `GUTARegisterUserCircuit` (Hypothetical)

- **Purpose:** To process the registration of one or more new users.
- **Core Logic:** Uses `GUTAOnlyRegisterUsersGadget` (which uses `GUTARegisterUsersGadget`, `GUTARegisterUserFullGadget`, `GUTARegisterUserCoreGadget`).
- **What it Proves:**
 - For each registered user, their `public_key` was correctly inserted at their `user_id` index in the `GUSR` tree (transitioning from zero hash to the new user leaf hash).
 - The `public_key` used matches an entry in the `user_registration_tree_root`.
 - Outputs a `GlobalUserTreeAggregatorHeader` representing the aggregate `GUSR` state transition for all registered users, with zero stats.
- **Assumptions:**
 - Witness (registration proofs, user count) is correct initially.
 - `guta_circuit_whitelist` and `checkpoint_tree_root` inputs are correct for this context.
 - `default_user_state_tree_root` constant is correct.
- **How Assumptions are Discharged:** Verifies delta proofs for `GUSR` insertion and Merkle proofs against the registration tree. Outputs a standard GUTA header, passing assumptions about whitelist/checkpoint upwards.

- **Contribution to Horizontal Scalability:** User registration can be batched and potentially processed in parallel branches of the GUTA tree.
- **High-Level Functionality:** Securely adds new users to the system state.

7. `GUTAAggregationCircuit` (Hypothetical - Multiple Variants)

- **Purpose:** To combine the results (headers) from two or more lower-level GUTA proofs (which could be End Cap results, registrations, or previous aggregations).
- **Core Logic:**
 - Verifies each input GUTA proof using `VerifyGUTAProofGadget`.
 - Ensures all input proofs used circuits from the same `guta_circuit_whitelist` and reference the same `checkpoint_tree_root`.
 - Combines the `state_transition`s from the input proofs:
 - If transitions are on different branches, uses `TwoNCASStateTransitionGadget` with an NCA proof.
 - If transitions are on the same branch (e.g., one input is a line proof output), connects them directly (`old_root` of current matches `new_root` of previous).
 - If only one input, uses `GUTAHeaderLineProofGadget` to propagate upwards.
 - Combines the `stats` from input proofs using `GUTASStatsGadget.combine_with`.
 - Outputs a single `GlobalUserTreeAggregatorHeader` representing the combined state transition and stats.
- **What it Proves:** That given valid input GUTA proofs operating under the same whitelist and checkpoint context, the combined state transition and stats represented by the output header are correct.
- **Assumptions:**
 - Witness (input proofs, headers, NCA/sibling proofs) is correct initially.
- **How Assumptions are Discharged:** Verifies input proofs and their headers. Verifies the logic of combining state transitions (NCA/Line/Direct). Passes the common whitelist/checkpoint root assumptions upwards.
- **Contribution to Horizontal Scalability:** This is the core of parallel aggregation. Multiple instances of this circuit run concurrently across the

DPN, merging proof branches in a tree structure (like MapReduce).

- **High-Level Functionality:** Securely and recursively combines verified state changes from multiple sources into larger, aggregated proofs.

8. `GUTANoChangeCircuit` (Hypothetical)

- **Purpose:** To handle cases where no user state changed but the checkpoint advanced.
- **Core Logic:** Uses `GUTANoChangeGadget`.
- **What it Proves:** That given a new `checkpoint_leaf` verified to be in the `checkpoint_tree_proof`, the `GUSR` tree root remains unchanged, and stats are zero. Outputs a GUTA header reflecting this.
- **Assumptions:** Witness (checkpoint proof, leaf) is correct initially. Input `guta_circuit_whitelist` is correct.
- **How Assumptions are Discharged:** Verifies checkpoint proof. Outputs a standard GUTA header passing assumptions upward.
- **Contribution to Horizontal Scalability:** Allows the aggregation process to stay synchronized with the checkpoint tree even during periods of inactivity for certain state trees.
- **High-Level Functionality:** Advances the aggregated checkpoint state reference.

Phase 3: Final Block Proof

9. Checkpoint Tree "Block" Circuit (Top-Level Aggregation)

- **Purpose:** The final aggregation circuit that combines proofs from the roots of all major state trees (like `GUSR` via the top-level GUTA proof, `GCON`, etc.) for the block.
- **Core Logic:**
 - Verifies the top-level GUTA proof (and proofs for other top-level trees if applicable).
 - Takes the *previous block's* finalized `CHKP` root as a public input.
 - Constructs the new `CHKP` leaf based on the newly computed roots of `GUSR`, `GCON`, etc., and other block metadata.
 - Computes the new `CHKP` root.

- The *only* external assumption verified here is that the input `previous_block_chkp_root` matches the actual finalized root of the last block.
- **What it Proves:** That the entire state transition for the block, represented by the change from the `previous_block_chkp_root` to the `new_chkp_root`, is valid, having recursively verified all constituent user transactions and aggregations according to protocol rules and circuit whitelists.
- **Assumptions:** The *only* remaining input assumption is the hash of the previous block's `CHKP` root.
- **How Assumptions are Discharged:** All assumptions from lower levels (circuit whitelists, internal state consistencies) have been verified recursively. The final link to the previous block state is checked against the public input.
- **Contribution to Horizontal Scalability:** Represents the culmination of the massively parallel aggregation process, producing a single, succinct proof for the entire block's validity.
- **High-Level Functionality:** Creates the final, verifiable proof of state transition for the entire block, linking it cryptographically to the previous block. This proof can be efficiently verified by any node or light client.

Coordinator Gadgets

These gadgets are components used within circuits run by the Coordinator nodes.

BatchAppendUserRegistrationTreeGadget

- **File:** `append_user_registration_tree.rs` (Gadget definition)
- **Purpose:** Aggregates multiple "Spiderman" append proofs sequentially for the User Registration Tree (URT). Handles padding for a fixed maximum number of sub-tree appends.
- **Key Inputs/Witness:**
 - `user_registration_tree_height`, `batch_sub_tree_height`, `max_sub_trees` : Parameters.
 - `SpidermanUpdateProof[]` : Array witness containing the append proofs for each sub-tree batch being added.
- **Key Outputs/Computed Values:**
 - `old_root` : The root of the URT *before* all appends in this gadget instance.
 - `new_root` : The root of the URT *after* all appends in this gadget instance.
- **Core Logic/Constraints:**
 - Instantiates `max_sub_trees` instances of `SpidermanAppendProofGadget`.
 - Connects the `new_root` of one gadget to the `old_root` of the next in sequence.
 - Handles witness padding by setting dummy proofs for unused slots.
- **Assumptions:** Assumes witness `SpidermanUpdateProof` array is valid initially (constraints verify internal consistency). Assumes `old_root` of the first gadget matches the tree state before this operation.
- **Role:** Allows efficient batching of user registration appends into a single ZK proof step for the Coordinator.

BatchDeployContractsGadget

- **File:** `deploy_contract.rs` (Gadget definition)
- **Purpose:** Handles the proof logic for appending a batch of new contracts to the Global Contract Tree (`GCON`). Verifies one Spiderman append proof and ensures the provided contract leaf data matches the appended hashes.
- **Key Inputs/Witness:**
 - `contract_tree_height` , `batch_sub_tree_height` : Parameters.
 - `SpidermanUpdateProof` : Witness for the batch append operation on `GCON` .
 - `PsyContractLeaf[]` : Array witness containing the data for each deployed contract leaf in the batch.
- **Key Outputs/Computed Values:**
 - `old_root` , `new_root` : Start and end roots of the `GCON` tree for this batch append (from `spiderman_gadget`).
- **Core Logic/Constraints:**
 - Instantiates `SpidermanAppendProofGadget` .
 - Instantiates `PsyContractLeafGadget` for each potential leaf slot in the batch.
 - For each leaf slot marked as added (`is_added` from Spiderman proof):
 - Computes the hash of the corresponding `PsyContractLeafGadget` witness.
 - Asserts this computed hash matches the `new_leaves[i]` value from the Spiderman proof.
 - Handles witness padding for unused leaf slots.
- **Assumptions:** Assumes witness `SpidermanUpdateProof` and `PsyContractLeaf` array are valid initially. Assumes `old_root` of the Spiderman gadget matches the `GCON` state before this operation.
- **Role:** Securely proves the batch addition of new contracts to the global contract tree, verifying consistency between the state update and the provided contract metadata.

VerifyAggUserRegistrationDeployContractsGUTAHeaderGadget

- **File:** `verify_agg_user_registration_deploy_guta.rs`
- **Purpose:** Represents the combined state transitions resulting from aggregating User Registrations, Contract Deployments, and GUTA proofs. Acts as the core data structure within the Part 1 Aggregation circuit.
- **Key Inputs/Witness:** (Typically derived from verified sub-proofs)
 - `user_registration_tree_delta`: `AggStateTransitionGadget` for URT.
 - `global_contract_tree_delta`: `AggStateTransitionGadget` for GCON.
 - `global_user_tree_delta`: `GlobalUserTreeAggregatorHeaderGadget` for GUSR.
- **Key Outputs/Computed Values:**
 - `combined_hash`: A single hash representing the start/end states of all three trees and the GUTA header.
- **Core Logic/Constraints:** Primarily a data structure. `get_combined_hash` defines the specific hashing scheme to commit to all input state transitions.
- **Assumptions:** Assumes the input transition/header gadgets are correctly derived from verified proofs.
- **Role:** Standardizes the output structure of the Part 1 aggregation step, providing a single hash commitment for verification by the final block circuit.

VerifyAggUserRegistrationDeployContractsGUTAGadget

- **File:** `verify_agg_user_registration_deploy_guta.rs`
- **Purpose:** The core gadget within the Part 1 Aggregation circuit. Verifies the aggregated proofs for User Registrations, Contract Deployments, and GUTA, ensuring they are valid, used whitelisted circuits, and reference the same checkpoint state.
- **Key Inputs/Witness:**

- Parameters and configuration for verifying each of the three input proofs (common data, whitelist/fingerprint configs, GUTA params).
 - Proof objects and verifier data for each of the three input proofs.
 - Witnesses for state transitions/headers corresponding to each proof.
 - GUTA whitelist Merkle proof.
- **Key Outputs/Computed Values:**
 - header : A
VerifyAggUserRegistrationDeployContractsGUTAHeaderGadget
containing the verified state transitions.
- **Core Logic/Constraints:**
 - Instantiates VerifyStateTransitionProofGadget for User Registrations, verifying the proof against its config/fingerprint.
 - Instantiates VerifyStateTransitionProofGadget for Contract Deployments similarly.
 - Instantiates VerifyGUTAProofGadget for the GUTA proof, verifying it against its config/fingerprint and the GUTA whitelist root.
 - Connects the checkpoint_tree_root from the GUTA header to ensure consistency (implicitly assumes UserReg/Deploy proofs are for the same checkpoint, which should be enforced by job planning).
Correction: This gadget doesn't directly connect checkpoint roots; that consistency is usually handled by the job system ensuring proofs for the same checkpoint are aggregated.
 - Constructs the output header from the verified transition gadgets.
- **Assumptions:** Assumes witness proofs, headers, and verifier data are valid initially. Assumes input configuration (common data, fingerprint configs, whitelist root) is correct.
- **Role:** Securely combines the results of the three major parallel state update processes (User Reg, Deploy Contract, GUTA) into a single verifiable unit, discharging assumptions about their individual validity and circuit usage.

PsyPart1StateDeltaResultGadget

- **File:** checkpoint_state_transition_proofs.rs

- **Purpose:** Takes the verified combined header from the Part 1 aggregation (`VerifyAggUserRegistrationDeployContractsGUTAHeaderGadget`) and combines it with previous block stats and new block info (time, randomness) to calculate the *new* Checkpoint Leaf state.
- **Key Inputs/Witness:**
 - `part_1_header` : Output from the Part 1 aggregation gadget.
 - `old_stats` : `PsyCheckpointLeafStatsGadget` witness for the previous block's stats.
 - `block_time` : Target witness for the current block's timestamp.
 - `final_random_seed_contribution` : Hash witness for randomness.
- **Key Outputs/Computed Values:**
 - `old_state_roots` , `new_state_roots` : Derived directly from `part_1_header` .
 - `new_stats` : Computed by combining GUTA stats with time, randomness, etc.
 - `old_checkpoint_leaf` : Constructed from `old_state_roots` and `old_stats` .
 - `new_checkpoint_leaf` : Constructed from `new_state_roots` and `new_stats` .
- **Core Logic/Constraints:**
 - Constructs `old_state_roots` and `new_state_roots` gadgets.
 - Computes `new_stats` based on inputs (copying GUTA stats, hashing random seed, setting time, zeroing PM/DA stats for now).
 - Constructs `old_checkpoint_leaf` and `new_checkpoint_leaf` gadgets.
 - Asserts `block_time > old_stats.block_time` .
- **Assumptions:** Assumes `part_1_header` is correctly verified. Assumes `old_stats` , `block_time` , `final_random_seed_contribution` witnesses are correct.
- **Role:** Calculates the state transition specifically for the Checkpoint Leaf data based on the aggregated results from the rest of the block's activities.

CheckpointStateTransitionChildProofsGadget

- **File:** `checkpoint_state_transition_proofs.rs`
- **Purpose:** Verifies the "Part 1" aggregation proof within the final block circuit and instantiates the gadget (`PsyPart1StateDeltaResultGadget`) that calculates the Checkpoint Leaf transition.
- **Key Inputs/Witness:**
 - Parameters for verifying the Part 1 proof (common data, cap height, known fingerprint).
 - Part 1 proof object and verifier data.
 - Witnesses needed by `PsyPart1StateDeltaResultGadget` (`old_stats` , `block_time` , `random_seed`).
- **Key Outputs/Computed Values:**
 - `state_delta_gadget` : The instantiated `PsyPart1StateDeltaResultGadget` .
- **Core Logic/Constraints:**
 - Verifies the `part_1_proof_target` against `part_1_verifier_data` .
 - Checks the fingerprint of `part_1_verifier_data` against `known_part_1_fingerprint` .
 - Instantiates `PsyPart1StateDeltaResultGadget` .
 - Computes the expected public inputs hash for the Part 1 proof using the `state_delta_gadget.part_1_header` .
 - Asserts this computed hash matches the actual public inputs from `part_1_proof_target` .
- **Assumptions:** Assumes witness proofs, verifier data, and state delta inputs are correct initially. Assumes `known_part_1_fingerprint` constant is correct.
- **Role:** Securely incorporates the aggregated result of UserReg/Deploy/GUTA processing (the Part 1 proof) into the final block transition calculation.

CheckpointStateTransitionCoreGadget

- **File:** `checkpoint_state_transition.rs`

- **Purpose:** Handles the core Merkle proof logic for updating the Checkpoint Tree (`CHKP`) itself. Verifies the append operation for the new checkpoint leaf and its consistency with the previous checkpoint leaf.
- **Key Inputs/Witness:**
 - `checkpoint_tree_height` : Parameter.
 - `append_checkpoint_tree_proof` : `DeltaMerkleProofCore` witness for appending the new leaf.
 - `previous_checkpoint_proof` : `MerkleProofCore` witness proving the existence of the *previous* checkpoint leaf.
- **Key Outputs/Computed Values:**
 - `old_checkpoint_tree_root` , `new_checkpoint_tree_root` : Roots before/after append.
 - `old_checkpoint_leaf_hash` , `new_checkpoint_leaf_hash` : Leaf hashes involved.
- **Core Logic/Constraints:**
 - Instantiates `DeltaMerkleProofGadget` for the append proof and `MerkleProofGadget` for the previous proof.
 - Asserts `append_checkpoint_tree_proof.old_value` is zero hash (ensures append).
 - Asserts `append_checkpoint_tree_proof.old_root` matches `previous_checkpoint_proof.root` (ensures continuity).
 - Asserts `append_checkpoint_tree_proof.index == previous_checkpoint_proof.index + 1`.
- **Assumptions:** Assumes witness Merkle proofs are valid initially.
- **Role:** Enforces the append-only nature and sequential integrity of the main Checkpoint Tree, linking the current block's update directly to the previous block's verified state.

User Proving Session (UPS) Gadgets

These gadgets are primarily used within the circuits executed locally by users (or their delegates) to prove their transaction sequences.

CorrectUPSHeaderHashesGadget

- **File:** `correct_header_hashes_rs.txt`
 - **Purpose:** A data structure gadget to hold potentially *modified* starting debt tree roots for a UPS step. Used when a transaction (like debt repayment) needs to alter the context *before* the main state delta logic of that same step runs.
 - **Technical Function:** Stores `previous_step_deferred_tx_debt_tree_root` and `previous_step_inline_tx_debt_tree_root`. These values can override the corresponding roots from the actual previous step's header when passed into gadgets like `UPSCFCStandardStateDeltaGadget`.
 - **Inputs/Witness:** Takes a reference to the previous step's `UserProvingSessionHeaderGadget`.
 - **Outputs/Computed:** The overridden hash targets.
 - **Constraints:** None internally; constraints are applied where its output values are consumed.
 - **Assumptions:** Assumes the input `previous_step` header is correctly formed (though its values might be overridden).
 - **Role:** Enables flexible ordering of operations within a single UPS step, specifically allowing debt tree modifications (like popping an item) to be accounted for before the main transaction logic uses those trees.
-

UPSVerifyCFCProofExistsAndValidGadget

- **File:** `ups_cfc_verify_inclusion_rs.txt`

- **Purpose:** Verifies two critical aspects of a Contract Function Call (CFC) within a UPS: (1) That the ZK proof for the CFC execution exists and is valid within the user's current UPS proof tree, and (2) That the specific function being called is officially registered within the contract's definition on the blockchain (via checkpoint context).
- **Technical Function:** Combines proof attestation within the UPS tree with function inclusion verification against global contract state.
- **Inputs/Witness:**
 - `checkpoint_state_gadget` : Witness for `PsyCheckpointLeafCompactWithStateRoots` providing context (global roots).
 - `verify_cfc_proof_gadget` : Witness (`AttestTreeAwareProofInTreeInput`) for the CFC proof itself, including its Merkle proof within the `session_proof_tree`.
 - `cfc_inclusion_proof_gadget` : Witness (`PsyContractFunctionInclusionProof`) proving the function's fingerprint exists in the contract's function tree (`CFT`).
 - `ups_session_proof_tree_height` : Parameter.
- **Outputs/Computed:**
 - `attested_proof_tree_root` : Root of the UPS session proof tree containing the CFC proof.
 - `checkpoint_leaf_hash` : Hash of the contextual checkpoint leaf.
 - `cfc_fingerprint` : The verified fingerprint (verifier data hash) of the CFC circuit.
 - `cfc_inner_public_inputs_hash` : Hash of the CFC proof's *original* public inputs (before tree awareness wrapping).
 - `cfc_contract_id`, `cfc_method_id`, `cfc_num_inputs`, `cfc_num_outputs` : Metadata extracted from the function inclusion proof.
- **Constraints:**
 - Verifies CFC proof validity and inclusion in the session tree via `AttestTreeAwareProofInTreeGadget`.
 - Verifies function inclusion in the contract's `CFT` via `PsyContractFunctionInclusionProofGadget`.
 - Connects `cfc_inclusion_proof.contract_inclusion_proof.contract_tree_m`

- erkle_proof.root to
 - checkpoint_state_gadget.global_state_roots.contract_tree_root (ensuring CFC inclusion is checked against the correct global contract state).
 - o Connects cfc_inclusion_proof.function_verifier_fingerprint to verify_cfc_proof_gadget.fingerprint (ensuring the proven function matches the verified CFC circuit).
 - **Assumptions:** Assumes witness data (proofs, state, inclusion proofs) is valid initially. Assumes ups_session_proof_tree_height is correct.
 - **Role:** Acts as a crucial security gate within UPS, ensuring that the user is proving the execution of a legitimate, registered smart contract function and that this execution proof is part of their current, consistent session proof sequence.
-

UPSCFCStandardStateDeltaGadget

- **File:** ups_standard_cfc_state_delta_rs.txt
- **Purpose:** Calculates the precise changes to the user's state (UserProvingSessionHeader) resulting from a single, standard CFC transaction. It enforces the consistency between the transaction's claimed effects (witnessed context) and the cryptographic updates to the relevant Merkle trees.
- **Technical Function:** Verifies delta/pivot proofs for state updates and computes the next header state.
- **Inputs/Witness:**
 - o previous_step_header_gadget : Header state *before* this transaction.
 - o corrections : Optional CorrectUPSHeaderHashesGadget to override starting debt tree roots.
 - o contract_state_tree_height : Target specifying the height of the specific CSTATE tree being modified.
 - o UPSCFCStandardStateDeltaInput : Witness containing:
 - cfc_transaction_input_context : Start (transaction_call_start_ctx) and end (transaction_end_ctx) contexts claimed by the CFC execution.

- `user_contract_tree_update_proof`: `DeltaMerkleProofCore` showing the change to the user's `UCON` tree (updating the root hash for the specific `contract_id`).
 - `deferred_tx_debt_pivot_proof`, `inline_tx_debt_pivot_proof`: `MerkleProofCore` showing the start and end roots of the debt trees remain consistent (pivot proofs).
- **Outputs/Computed:**
 - `(Self, UserProvingSessionHeaderGadget)`: Returns the gadget instance and the *new* header state after applying the delta.
 - `cfc_inner_public_inputs_hash`: Hash of the `cfc_transaction_input_context` (used for linking to verification).
 - `cfc_contract_id`, `cfc_method_id`, etc.: Metadata passed through.
- **Constraints:**
 - **CFC Context Hash:** Computes `cfc_inner_public_inputs_hash` from witness.
 - **UCON Update:** Verifies `user_contract_tree_update_proof`:
 - `old_root` matches `previous_step_header.current_state.user_leaf.user_state_tree_root`.
 - `index` matches `cfc_transaction_input_context...contract_id`.
 - `old_value` consistency check: If zero, ensures `start_ctx.start_contract_state_tree_root` matches the *default* zero hash for the given `contract_state_tree_height`. If non-zero, ensures it matches `start_ctx.start_contract_state_tree_root`.
 - `new_value` matches `end_ctx.end_contract_state_tree_root`.
 - **Debt Tree Pivots:** Verifies `deferred/inline_tx_debt_pivot_proof`:
 - `historical_root` matches the *corrected* previous step debt root (from `previous_step_header` or `corrections`).
 - `current_root` matches the `end_ctx.end_deferred/inline_tx_debt_tree_root`.
 - **Start State Consistency:** Connects `start_ctx` fields (balance, event index, debt roots) to the corresponding fields in the (potentially corrected) `previous_step_header`.

- **Counters & Stack:** Increments `tx_count` . Pushes hash of `TransactionLogStackItemGadget` onto `tx_hash_stack` using `SimpleHashStackGadget` .
 - **Construct New Header:** Creates the output `UserProvingSessionHeaderGadget` with updated roots, counts, stack, and user leaf values (balance/event index updated based on `end_ctx`). *Note: Currently balance/event updates are disabled via constraints.*
 - **Assumptions:** Assumes witness proofs and contexts are valid initially. Assumes `previous_step_header` and `corrections` are correctly provided.
 - **Role:** The core state transition engine for UPS. It cryptographically enforces that the claimed effects of a CFC execution (start/end states) are correctly reflected in the updates to the user's persistent state trees (`UCON` , debt trees) and session metadata (tx count/stack).
-

UPSVerifyCFCStandardStepGadget

- **File:** `ups_cfc_standard_rs.txt`
- **Purpose:** Encapsulates a complete, standard transaction processing step within UPS. It combines the verification of the CFC proof's existence and validity with the calculation and verification of the resulting state delta.
- **Technical Function:** Orchestrates `UPSVerifyCFCProofExistsAndValidGadget` and `UPSCFCStandardStateDeltaGadget` , connecting their inputs and outputs to ensure consistency.
- **Inputs/Witness:**
 - `previous_step_header_gadget` : Header state before this step.
 - `current_proof_tree_root` : Root of the UPS proof tree this step belongs to.
 - `ups_session_proof_tree_height` : Parameter.
 - `UPSVerifyCFCStandardStepInput` : Witness containing inputs for both sub-gadgets.
- **Outputs/Computed:**

- `new_header_gadget` : The header state *after* this transaction step.
 - **Constraints:**
 - Instantiates the verification and state delta sub-gadgets.
 - `verify_cfc_exists_and_valid_gadget.attested_proof_tree_root == current_proof_tree_root`.
 - `verify_cfc_exists_and_valid_gadget.checkpoint_leaf_hash == previous_step_header_gadget.session_start_context.checkpoint_leaf_hash`.
 - Connects all key metadata (`contract_id` , `method_id` , `num_inputs/outputs` , `inner_public_inputs_hash`) between the verification and state delta gadgets, ensuring they operate on the *exact same verified transaction*.
 - **Assumptions:** Relies on sub-gadget assumptions. Assumes `current_proof_tree_root` is correct.
 - **Role:** Defines a standard, verifiable "block" within the user's local recursive proof chain, corresponding to processing one smart contract call.
-

UPSVerifyPopDeferredTxStepGadget

- **File:** `ups_cfc_standard_pop_deferred_tx_rs.txt`
- **Purpose:** Handles transactions specifically designed to settle a previously incurred deferred transaction debt. It verifies the debt removal and then processes the corresponding CFC execution.
- **Technical Function:** Verifies a delta proof for removing an item from the deferred debt tree, checks its consistency with the CFC being executed, and then uses `UPSVerifyCFCStandardStepGadget` with a corrected starting context.
- **Inputs/Witness:**
 - `previous_step_header_gadget` , `current_proof_tree_root` , `ups_session_proof_tree_height` : Same as standard step.
 - `UPSVerifyPopDeferredTxStepInput` : Witness containing standard step inputs *plus* `ups_pop_deferred_tx_proof` (a `DeltaMerkleProofCore` for the deferred debt tree).

- **Outputs/Computed:** Exposes outputs from the internal `standard_cfc_verify_gadget`, notably the `new_header_gadget`.
 - **Constraints:**
 - Verifies `ups_pop_deferred_tx_proof` using `DeltaMerkleProofGadget`.
 - `ups_pop_deferred_tx_proof.old_root == previous_step_header.current_state.deferred_tx_debt_tree_root`.
 - `ups_pop_deferred_tx_proof.new_value == ZERO_HASH` (proves removal).
 - Computes expected hash of the deferred item (`DeferredTransactionStackItemGadget`) based on the CFC's `call_data`.
 - `ups_pop_deferred_tx_proof.old_value == computed deferred item hash` (ensures correct debt removed).
 - Instantiates `CorrectUPSHeaderHashesGadget`, setting `previous_step_deferred_tx_debt_tree_root = ups_pop_deferred_tx_proof.new_root`.
 - Instantiates `UPSVerifyCFCStandardStepGadget` using the `corrections`.
 - **Assumptions:** Relies on sub-gadget assumptions. Assumes witness delta proof is valid initially.
 - **Role:** Enables verifiable settlement of asynchronous obligations (deferred calls) generated by previous transactions within the UPS flow.
-

PsyUserProvingSessionSignatureDataCompactGadget

- **File:** `ups_signature_data_rs.txt`
- **Purpose:** Defines the precise data structure that is cryptographically signed by the user to authorize the submission of their completed User Proving Session.
- **Technical Function:** Aggregates key state identifiers from the start and end of the UPS into a single, hashable structure, then combines it with context (network, user, nonce) for signing.

- **Inputs/Witness:** `start_user_leaf_hash`, `end_user_leaf_hash`, `checkpoint_leaf_hash`, `tx_stack_hash`, `tx_count`.
 - **Outputs/Computed:** `ups_end_cap_sighash` (via `get_sig_action_with_user_info`).
 - **Constraints:** Internal hashing logic (`to_hash` method) combines inputs. `get_sig_action_with_user_info` uses `compute_sig_action_hash_circuit` to combine the data hash with `network_magic`, `user_id`, `nonce`, and the `PSY_SIG_ACTION_SIGN_UPS_END_CAP` constant.
 - **Assumptions:** Assumes input hash/target values correctly represent the final UPS state.
 - **Role:** Standardizes the payload for UPS authorization, ensuring all critical state transition elements are committed to before the user signs off.
-

UPSEndCapResultCompactGadget

- **File:** `ups_end_cap_result_rs.txt`
 - **Purpose:** Defines the minimal, verifiable summary of a completed UPS, intended for submission to the GUTA layer.
 - **Technical Function:** A data structure containing the essential start/end state identifiers needed for aggregation.
 - **Inputs/Witness:** `start_user_leaf_hash`, `end_user_leaf_hash`, `checkpoint_tree_root_hash`, `user_id`.
 - **Outputs/Computed:** `end_cap_result_hash` (`to_hash` method).
 - **Constraints:** Hashing logic combines inputs with the `GLOBAL_USER_TREE_HEIGHT` constant.
 - **Assumptions:** Assumes input hash/target values correctly represent the final UPS state and context.
 - **Role:** Creates the standardized output data that represents the user's net state change for the block in a format suitable for GUTA circuits.
-

UPSEndCapCoreGadget

- **File:** `ups_end_cap_rs.txt`
- **Purpose:** Enforces the final set of critical constraints required to validly conclude a User Proving Session, linking the final state to the user's signature authorization.
- **Technical Function:** Verifies nonce progression, public key consistency, signature data correctness, checkpoint progression, empty debt trees, and computes final outputs (Result and Stats).
- **Inputs/Witness:**
 - `last_header_gadget` : Final header state of the UPS.
 - `sig_proof_public_inputs_hash` : Public inputs hash from the user's signature proof.
 - `sig_proof_fingerprint` , `sig_proof_param_hash` : Signature circuit identifier hashes.
 - `nonce` , `slots_modified` : Witness targets.
 - `network_magic` , empty debt root constants: Parameters.
- **Outputs/Computed:** `sig_data_compact_gadget` , `end_cap_result_gadget` , `guta_stats` .
- **Constraints:**
 - Nonce Check: `nonce > start_user_leaf.nonce` . Updates final leaf nonce.
 - PK Check: Derives expected PK from sig proof params, ensures start/end user leaves have this same PK.
 - User ID Check: Start/End user IDs match.
 - Sig Data Check: Instantiates `PsyUserProvingSessionSignatureDataCompactGadget` , computes expected `sig_proof_public_inputs_hash` , connects it to the input hash from the sig proof.
 - Checkpoint Check: Ensures `end_user_leaf.last_checkpoint_id == session_checkpoint_id > start_user_leaf.last_checkpoint_id` .
 - Debt Check: Connects `last_header.current_state` debt roots to empty root constants.
 - Output Generation: Instantiates `UPSEndCapResultCompactGadget` and `GUTASStatsGadget` .

- **Assumptions:** Assumes input `last_header_gadget` is correct (verified by previous step). Assumes signature proof is valid (verified elsewhere). Assumes witness nonce/slots/params are correct.
 - **Role:** The final gatekeeper for UPS validity, ensuring all protocol rules for session finalization are met and linking the state transition to cryptographic authorization before generating the outputs for network submission.
-

VerifyPreviousUPSStepProofInProofTreeGadget

- **File:** `verify_previous_ups_step_rs.txt`
- **Purpose:** Essential gadget for recursion within UPS. It verifies the ZK proof generated by the immediately preceding UPS step, ensuring the chain of proofs is unbroken and follows protocol rules.
- **Technical Function:** Verifies a proof (`AttestTreeAwareProofInTreeGadget`), checks its circuit fingerprint against a whitelist (`MerkleProofGadget`), and confirms its public inputs match the expected previous header state hash.
- **Inputs/Witness:**
 - `VerifyPreviousUPSStepProofInProofTreeInput` : Contains attestation witness, `previous_step_header` witness, and whitelist Merkle proof witness.
 - Tree height parameters.
- **Outputs/Computed:** `previous_step_header_gadget` (representing verified public inputs), `current_proof_tree_root`, `ups_step_circuit_whitelist_root`.
- **Constraints:**
 - Verifies proof attestation.
 - Verifies whitelist proof.
 - Connects attestation fingerprint to whitelist proof value.
 - Connects `previous_step_header.ups_step_circuit_whitelist_root` to whitelist proof root.
 - Computes hash of `previous_step_header` witness.

- Connects computed hash to `proof_attestation_gadget.inner_public_inputs_hash`.
 - **Assumptions:** Assumes witness data (proofs, header, whitelist proof) is valid initially.
 - **Role:** Enforces the integrity of the recursive proof chain generated locally by the user, step by step. Ensures only allowed UPS circuits are used.
-

VerifyPreviousUPSStepProofInProofTreePartialFromCurrentGadget

- **File:** `verify_previous_ups_step_partial_from_current_rs.txt`
 - **Purpose:** An optimized version of the previous gadget, used when the `session_start_context` is constant within the verifying circuit (like the End Cap circuit). Reduces witness size.
 - **Technical Function:** Similar to the full gadget, but reconstructs the `previous_step_header_gadget` internally using the known `session_start_context` (from `current_header` input) and only requiring the `previous_step_state` portion as witness.
 - **Inputs/Witness:**
 - `current_header` : *Input* gadget (not witness).
 - `VerifyPreviousUPSStepProofInProofTreePartialInput` : Contains attestation witness, `previous_step_state` witness, whitelist proof witness.
 - **Outputs/Computed:** Same as the full gadget.
 - **Constraints:** Similar to full gadget, plus connects `current_header.ups_step_circuit_whitelist_root` to the whitelist proof root. Reconstructs previous header internally before hashing and connecting to attestation inner public inputs.
 - **Assumptions:** Assumes input `current_header` is correct. Assumes witness data is valid initially.
 - **Role:** Witness optimization for recursive proof verification in specific circuit contexts.
-

UPSEndCapFromProofTreeGadget

- **File:** `ups_end_cap_tree_rs.txt`
 - **Purpose:** Top-level gadget within the `UPSStandardEndCapCircuit`. Orchestrates the verification of the final UPS step, verification of the ZK signature, and enforcement of final session constraints.
 - **Technical Function:** Instantiates and connects `VerifyPreviousUPSStepProofInProofTreeGadget` (often partial), `AttestProofInTreeGadget` (for signature), and `UPSEndCapCoreGadget`.
 - **Inputs/Witness:**
 - `UPSEndCapFromProofTreeGadgetInput` : Contains witnesses for previous step verification, signature verification, `user_public_key_param`, `nonce`, `slots_modified`.
 - Tree height and network parameters.
 - **Outputs/Computed:** `end_cap_core_gadget`, `current_proof_tree_root`.
 - **Constraints:**
 - Instantiates sub-gadgets.
 - Connects `verify_zk_signature_proof_gadget.attested_proof_tree_root` to `verify_previous_ups_step_gadget.current_proof_tree_root` (ensures signature and last step are in the same proof tree).
 - Passes verified data (previous header, signature hashes) and witnesses (nonce, slots, params) into `UPSEndCapCoreGadget` for final checks.
 - **Assumptions:** Relies on sub-gadget assumptions. Assumes witness data is valid initially.
 - **Role:** Coordinates the final verification checks within the End Cap circuit, ensuring the entire session is valid, linked, and authorized.
-

UPSStartStepGadget

- **File:** `ups_start_rs.txt`
- **Purpose:** Core logic for the `UPSStartSessionCircuit`. Verifies the user's provided initial state against the last finalized block's checkpoint data and initializes the session header.

- **Technical Function:** Verifies Merkle proofs linking the checkpoint leaf to the checkpoint root and the user leaf to the user tree root within that checkpoint. Ensures header consistency and correct initialization of session state (debt trees, counters).
- **Inputs/Witness:** `UPSStartStepInput` (header witness, checkpoint leaf/roots witness, checkpoint proof, user proof).
- **Outputs/Computed:** `header_gadget` (the validated starting header).
- **Constraints:**
 - Verifies consistency between `checkpoint_tree_proof` and `header_gadget.session_start_context` (root, leaf hash, ID).
 - Verifies consistency between `checkpoint_leaf_gadget`, `state_roots_gadget`, and header data.
 - Verifies `user_tree_proof.root` matches `state_roots_gadget.user_tree_root`.
 - Verifies `user_tree_proof.value` matches `header_gadget.session_start_context.start_session_user_leaf_hash`.
 - Verifies `user_tree_proof.index` matches `user_id`.
 - Verifies `current_state` initialization (updated `last_checkpoint_id`, empty debts, zero counts/stack).
- **Assumptions:** Assumes witness data is valid initially. Assumes empty tree root constants are correct.
- **Role:** Securely bootstraps the UPS, ensuring it starts from a globally valid and consistent state anchor.

This document details the various gadgets used within the Psy ZK circuits, primarily focusing on the User Proving Session (UPS) and Global User Tree Aggregation (GUTA) systems. Gadgets are reusable circuit components that enforce specific constraints and logic.

UPS Gadgets (User Proving Session)

These gadgets are components used within the circuits that users run locally to prove their sequence of transactions.

CorrectUPSHeaderHashesGadget

- **File:** `correct_header_hashes.rs`
- **Purpose:** To hold potentially *modified* hash roots from a previous UPS step header. This is specifically used when a transaction needs to alter the *starting* state assumptions for debt trees (e.g., paying back a deferred transaction *before* processing the current transaction's main logic).
- **Key Inputs/Witness:** Takes a `UserProvingSessionHeaderGadget` representing the *actual* previous step.
- **Key Outputs/Computed Values:**
 - `previous_step_deferred_tx_debt_tree_root` : The deferred debt root to *assume* as the starting point for the *next* step's logic.
 - `previous_step_inline_tx_debt_tree_root` : The inline debt root to *assume* as the starting point for the *next* step's logic.
- **Core Logic/Constraints:** Primarily a data structure. Its values are *used* by other gadgets (like `UPSCFCStandardStateDeltaGadget`) to override the default assumption that the starting debt roots are identical to the previous step's *ending* debt roots.
- **Assumptions:** Assumes the input `previous_step` header gadget is correctly formed (though its values might be overridden).
- **Role:** Enables flexible handling of transaction debt repayment within the UPS flow, allowing debts to be settled *before* the main state delta logic runs, without breaking the constraint flow.

UPSVerifyCFCProofExistsAndValidGadget

- **File:** `ups_cfc_verify_inclusion.rs`
- **Purpose:** To verify that a specific Contract Function Call (CFC) proof exists within the user's current UPS proof tree and that the function being called is validly registered within the global contract structure.
- **Key Inputs/Witness:**
 - `AttestTreeAwareProofInTreeInput` : Witness for the CFC proof's existence and validity within the UPS proof tree.
 - `PsyCheckpointLeafCompactWithStateRoots` : Witness for the relevant checkpoint state.
 - `PsyContractFunctionInclusionProof` : Witness proving the function's fingerprint exists in the contract's function tree.
 - `ups_session_proof_tree_height` : Parameter.
- **Key Outputs/Computed Values:**
 - `attested_proof_tree_root` : The root of the UPS proof tree the CFC proof is part of.
 - `checkpoint_leaf_hash` : Hash of the checkpoint leaf used for context.
 - `cfc_fingerprint` : The fingerprint (verifier data hash) of the CFC proof being verified.
 - `cfc_inner_public_inputs_hash` : The hash of the CFC proof's *internal* public inputs (before wrapping with tree awareness).
 - `cfc_contract_id`, `cfc_method_id`, `cfc_num_inputs`, `cfc_num_outputs` : Metadata about the contract function extracted from the inclusion proof.
- **Core Logic/Constraints:**
 - Verifies the CFC proof using `AttestTreeAwareProofInTreeGadget`.
 - Verifies the function's inclusion in the contract tree using `PsyContractFunctionInclusionProofGadget`.
 - Connects the contract tree root from the inclusion proof to the one in the checkpoint state.
 - Connects the CFC fingerprint from the proof attestation to the one in the inclusion proof.
- **Assumptions:** Assumes the witness data provided (proofs, checkpoint state) is valid for the constraints to pass.

- **Role:** Ensures that the CFC proof being processed corresponds to a valid, registered function and exists within the user's current session proof tree. Links the specific CFC execution to the global contract state via the checkpoint.

UPSCFCStandardStateDeltaGadget

- **File:** `ups_standard_cfc_state_delta.rs`
- **Purpose:** Calculates the state changes to a user's UPS header based on executing a standard CFC transaction. It updates the user's contract state tree root, transaction debt trees, transaction count, and transaction hash stack.
- **Key Inputs/Witness:**
 - `previous_step_header_gadget` : The header state *before* this transaction.
 - `corrections` : Optional `CorrectUPSHeaderHashesGadget` to override starting debt tree roots.
 - `contract_state_tree_height` : The height of the specific CSTATE tree being modified.
 - `UPSCFCStandardStateDeltaInput` : Witness containing the transaction input context (start/end states) and proofs for tree updates (user contract tree delta, debt tree pivots).
- **Key Outputs/Computed Values:**
 - `new_header_gadget` : The `UserProvingSessionHeaderGadget` representing the state *after* this transaction.
 - `cfc_inner_public_inputs_hash` : Hash of the CFC transaction input context (used for connecting to verification gadgets).
 - `cfc_contract_id`, `cfc_method_id`, `cfc_num_inputs`, `cfc_num_outputs` : Metadata passed through.
- **Core Logic/Constraints:**
 - Computes the expected `cfc_inner_public_inputs_hash` from the transaction context witness.
 - Verifies the `user_contract_tree_update_proof` (`DeltaMerkleProofGadget`):
 - Checks `old_root` matches the `user_state_tree_root` in the `previous_step_header_gadget`.

- Checks `index` matches the `tx_in_contract_id`.
 - Checks `old_value` consistency (handles initialization from zero hash to default root based on height).
 - Checks `new_value` matches the `tx_in_end_contract_state_tree_root`.
 - Verifies `deferred_tx_debt_pivot_proof` (HistoricalRootMerkleProofGadget):
 - Checks `historical_root` matches the *corrected* `previous_step_deferred_tx_debt_tree_root`.
 - Checks `current_root` matches `tx_in_end_deferred_tx_debt_tree_root`.
 - Verifies `inline_tx_debt_pivot_proof` similarly.
 - Checks consistency between previous header state (balance, event index) and transaction start context.
 - Updates transaction count (`new_tx_count = old_tx_count + 1`).
 - Pushes the transaction log item onto the `tx_hash_stack`.
 - Constructs the `new_header_gadget` with updated values.
- **Assumptions:** Assumes witness proofs and context are valid. Assumes the `previous_step_header_gadget` and `corrections` are correctly provided.
 - **Role:** The core engine for calculating user state updates resulting from a standard contract call within the UPS.

UPSVerifyCFCStandardStepGadget

- **File:** `ups_cfc_standard.rs`
- **Purpose:** Combines the verification of a CFC proof's existence/validity (`UPSVerifyCFCProofExistsAndValidGadget`) with the calculation of its resulting state changes (`UPSCFCStandardStateDeltaGadget`). It acts as the main gadget for processing a standard transaction step within a UPS circuit.
- **Key Inputs/Witness:**
 - `previous_step_header_gadget` : Header state before this step.
 - `current_proof_tree_root` : The root of the UPS proof tree this step belongs to.
 - `ups_session_proof_tree_height` : Parameter.

- `UPSVerifyCFCStandardStepInput` : Witness containing inputs for both sub-gadgets.
- **Key Outputs/Computed Values:**
 - `new_header_gadget` : The `UserProvingSessionHeaderGadget` representing the state *after* this transaction step.
 - (Internal gadgets expose their outputs too).
- **Core Logic/Constraints:**
 - Instantiates `UPSVerifyCFCProofExistsAndValidGadget` and `UPSCFCStandardStateDeltaGadget`.
 - Connects `current_proof_tree_root` to the `attested_proof_tree_root` of the verification gadget.
 - Connects the `checkpoint_leaf_hash` between the verification gadget and the `previous_step_header_gadget`.
 - Connects key metadata (`cfc_contract_id`, `cfc_method_id`, `cfc_num_inputs`, `cfc_num_outputs`, `cfc_inner_public_inputs_hash`) between the verification gadget and the state delta gadget to ensure they operate on the same transaction.
- **Assumptions:** Relies on the assumptions of its constituent gadgets. Assumes `current_proof_tree_root` is correctly provided.
- **Role:** Represents a complete, verifiable step for processing one standard CFC transaction within the user's local proving session.

UPSVerifyPopDeferredTxStepGadget

- **File:** `ups_cfc_standard_pop_deferred_tx.rs`
- **Purpose:** Processes a transaction that *pays back* a previously incurred deferred transaction debt. It verifies the removal of the debt item from the deferred debt tree and then processes the CFC transaction itself (using the standard step gadget but with a *corrected* starting deferred debt state).
- **Key Inputs/Witness:**
 - `previous_step_header_gadget` : Header state before this step.
 - `current_proof_tree_root` : The root of the UPS proof tree this step belongs to.
 - `ups_session_proof_tree_height` : Parameter.

- `UPSVerifyPopDeferredTxStepInput` : Witness containing inputs for the standard CFC step *and* the `DeltaMerkleProof` for removing the deferred transaction.
- **Key Outputs/Computed Values:**
 - (Exposes outputs from `UPSVerifyCFCStandardStepGadget` , notably the `new_header_gadget`).
- **Core Logic/Constraints:**
 - Instantiates `DeltaMerkleProofGadget` (`ups_pop_deferred_tx_proof`) for the deferred debt tree.
 - Connects `ups_pop_deferred_tx_proof.old_root` to the `deferred_tx_debt_tree_root` in `previous_step_header_gadget` .
 - Asserts `ups_pop_deferred_tx_proof.new_value` is the zero hash (proving removal).
 - Computes the hash of the expected deferred transaction item based on the call data from the CFC being processed.
 - Asserts `ups_pop_deferred_tx_proof.old_value` (the item removed) matches the computed hash.
 - Creates a `CorrectUPSHeaderHashesGadget` , setting `previous_step_deferred_tx_debt_tree_root` to `ups_pop_deferred_tx_proof.new_root` .
 - Instantiates `UPSVerifyCFCStandardStepGadget` using the *corrected* header hashes.
- **Assumptions:** Relies on assumptions of sub-gadgets. Assumes witness data (proofs, context) is valid.
- **Role:** Enables verifiable settlement of deferred transaction debts within the UPS, ensuring the debt removed matches the transaction being executed.

PsyUserProvingSessionSignatureDataCompactGadget

- **File:** `ups_signature_data.rs`
- **Purpose:** Defines the data structure that gets signed by the user's private key (or equivalent signature scheme) to authorize the end of a User Proving Session.
- **Key Inputs/Witness:**

- `start_user_leaf_hash` : Hash of the user leaf at the *start* of the session.
- `end_user_leaf_hash` : Hash of the user leaf at the *end* of the session.
- `checkpoint_leaf_hash` : Hash of the checkpoint leaf the session was based on.
- `tx_stack_hash` : Final hash of the transaction log stack.
- `tx_count` : Total number of transactions processed.
- **Key Outputs/Computed Values:**
 - `ups_end_cap_sighash` : The final hash (part of the signature pre-image) computed from the input data combined with network magic, user ID, and nonce.
- **Core Logic/Constraints:**
 - Computes an intermediate hash combining start/end user leaves.
 - Computes an intermediate hash combining tx stack and count.
 - Computes an intermediate hash combining checkpoint leaf and user leaf combo.
 - Computes the final data hash by combining the state context and tx combo hashes.
 - Uses `compute_sig_action_hash_circuit` to combine this data hash with network magic, user ID, nonce, and the specific signature action code (`PSY_SIG_ACTION_SIGN_UPS_END_CAP`) to produce the final `ups_end_cap_sighash`.
- **Assumptions:** Assumes input hashes and targets are correctly provided.
- **Role:** Standardizes the data payload for UPS end cap signatures, ensuring all necessary state transition information is committed to before signing.

UPSEndCapResultCompactGadget

- **File:** `ups_end_cap_result.rs`
- **Purpose:** Defines the compact data structure representing the *result* of a completed UPS, which is submitted to the GUTA layer for aggregation.
- **Key Inputs/Witness:**
 - `start_user_leaf_hash` : Hash of the user leaf at the *start* of the session.
 - `end_user_leaf_hash` : Hash of the user leaf at the *end* of the session.

- `checkpoint_tree_root_hash` : Root of the checkpoint tree the session was based on.
 - `user_id` : The ID of the user.
- **Key Outputs/Computed Values:**
 - `end_cap_result_hash` : The hash representing this result structure.
- **Core Logic/Constraints:**
 - Computes an intermediate hash combining user ID, start/end user leaves, and the global user tree height constant.
 - Computes the final `end_cap_result_hash` by hashing the intermediate hash with the `checkpoint_tree_root_hash`.
- **Assumptions:** Assumes input hashes and targets are correctly provided.
- **Role:** Creates the standardized, hashable output data for a completed UPS, suitable for inclusion as a public input in the end cap proof and for verification by GUTA circuits.

UPSEndCapCoreGadget

- **File:** `ups_end_cap.rs`
- **Purpose:** Enforces the core constraints for finalizing a User Proving Session (End Cap). It connects the final UPS state to the signature proof and prepares the final output data (result and stats).
- **Key Inputs/Witness:**
 - `last_header_gadget` : The header state at the *end* of the UPS (after the last transaction).
 - `sig_proof_public_inputs_hash` : Public inputs hash from the user's signature proof.
 - `sig_proof_fingerprint`, `sig_proof_param_hash` : Hashes related to the signature circuit's verifier data and parameters (used to derive the expected public key).
 - `nonce` : The nonce used in the signature, provided as witness.
 - `slots_modified` : Total storage slots modified, provided as witness.
 - `network_magic`, `empty_deferred_tx_debt_tree_root`, `empty_inline_tx_debt_tree_root` : Constants/parameters.
- **Key Outputs/Computed Values:**
 - `sig_data_compact_gadget` : The computed signature data payload.
 - `end_cap_result_gadget` : The computed end cap result structure.

- `guta_stats` : The computed GUTA statistics for this session.
- **Core Logic/Constraints:**
 - Checks nonce progression: Ensures the final `nonce` is greater than the starting nonce and updates the user leaf nonce.
 - Verifies public key consistency:
 - Derives the `expected_public_key` from the signature proof fingerprint/params.
 - Asserts the start and end user leaves have the same public key.
 - Asserts this public key matches the `expected_public_key`.
 - Verifies user ID consistency.
 - Instantiates `PsyUserProvingSessionSignatureDataCompactGadget` using data from the `last_header_gadget`.
 - Computes the expected `ups_end_cap_sighash` using the signature data gadget, network magic, user ID, and nonce.
 - Computes the expected `sig_proof_public_inputs_hash` by hashing the `ups_end_cap_sighash` with the `sig_proof_param_hash`.
 - Asserts the computed `sig_proof_public_inputs_hash` matches the one provided as input.
 - Instantiates `UPSEndCapResultCompactGadget` with final state data.
 - Checks checkpoint ID progression: Ensures the final user leaf's `last_checkpoint_id` matches the session's checkpoint ID and is greater than the starting leaf's `last_checkpoint_id`.
 - Asserts final debt trees are empty by comparing their roots in `last_header_gadget` to the provided empty tree root constants.
 - Instantiates `GUTASstatsGadget` using the final transaction count and provided `slots_modified`.
- **Assumptions:** Assumes input hashes, targets, and the `last_header_gadget` are correct. Assumes the signature proof itself is valid (verification happens elsewhere or is implied).
- **Role:** The central gadget for validating the conditions required to end a UPS, linking the final state to the signature authorization and generating the outputs needed for GUTA.

VerifyPreviousUPSStepProofInProofTreeGadget

- **File:** `verify_previous_ups_step.rs`

- **Purpose:** Verifies a ZK proof corresponding to the *previous* step in the User Proving Session's recursive chain. It ensures the proof is valid, exists in the expected UPS proof tree, used an allowed UPS circuit, and matches the expected previous header state.
- **Key Inputs/Witness:**
 - `ups_session_proof_tree_height`, `ups_circuit_whitelist_tree_height` : Parameters.
 - `VerifyPreviousUPSStepProofInProofTreeInput` : Witness containing:
 - `AttestTreeAwareProofInTreeInput` : Witness for the previous step's proof attestation.
 - `UserProvingSessionHeader` : Witness for the *header state* that should be the public input of the previous proof.
 - `MerkleProof` : Witness proving the previous proof's circuit fingerprint is in the UPS circuit whitelist.
- **Key Outputs/Computed Values:**
 - `proof_attestation_gadget` : The underlying attestation gadget.
 - `previous_step_header_gadget` : The header gadget representing the public inputs of the verified proof.
 - `ups_circuit_whitelist_merkle_proof` : The whitelist proof gadget.
 - `current_proof_tree_root` : The root of the UPS proof tree identified by the attestation proof.
 - `ups_step_circuit_whitelist_root` : The root of the UPS circuit whitelist tree.
- **Core Logic/Constraints:**
 - Instantiates `AttestTreeAwareProofInTreeGadget` to verify the previous proof's existence in the tree.
 - Instantiates `UserProvingSessionHeaderGadget` for the previous step's public inputs (header).
 - Instantiates `MerkleProofGadget` for the circuit whitelist check.
 - Connects the `fingerprint` from the proof attestation to the `value` in the whitelist proof.
 - Connects the `ups_step_circuit_whitelist_root` from the previous header gadget to the `root` of the whitelist proof gadget.
 - Computes the expected hash of the `previous_step_header_gadget`.

- Connects this expected hash to the `inner_public_inputs_hash` from the proof attestation gadget.
- **Assumptions:** Assumes witness data (proofs, headers) is valid for constraints to pass.
- **Role:** Crucial for the recursive nature of UPS. Each step verifies the *previous* step's proof, ensuring the integrity of the entire transaction chain generated locally by the user. Links the execution to the allowed set of UPS circuits.

VerifyPreviousUPSStepProofInProofTreePartialFromCurrentGadget

- **File:** `verify_previous_ups_step_partial_from_current.rs`
- **Purpose:** Similar to the full verification gadget, but optimized for cases where the *current* header's `session_start_context` is already known and fixed within the circuit. It only needs the *previous step's state* (`UserProvingSessionCurrentStateGadget`) as witness, reconstructing the full previous header internally.
- **Key Inputs/Witness:**
 - `current_header` : The *current* step's header gadget (provided as input, not witness).
 - `ups_session_proof_tree_height` ,
`ups_circuit_whitelist_tree_height` : Parameters.
 - `VerifyPreviousUPSStepProofInProofTreePartialInput` : Witness containing:
 - `AttestTreeAwareProofInTreeInput` : Witness for the previous proof attestation.
 - `UserProvingSessionCurrentState` : Witness for the *state portion* of the previous header.
 - `MerkleProof` : Witness for the UPS circuit whitelist proof.
- **Key Outputs/Computed Values:** Same as `VerifyPreviousUPSStepProofInProofTreeGadget`.
- **Core Logic/Constraints:**
 - Similar logic to the full version, but it *constructs* the `previous_step_header_gadget` by combining the known `session_start_context` from the `current_header` with the `previous_step_state` provided as witness.

- Enforces the same connections for fingerprint, whitelist root, and inner public inputs hash.
 - Adds a constraint connecting the `ups_step_circuit_whitelist_root` from the *current* header to the whitelist proof root.
- **Assumptions:** Assumes the provided `current_header` gadget is correct. Assumes witness data is valid.
- **Role:** An optimization for verifying previous steps within circuits where the session start context is constant (like the End Cap circuit). Reduces witness size.

UPSEndCapFromProofTreeGadget

- **File:** `ups_end_cap_tree.rs`
- **Purpose:** Orchestrates the entire End Cap process within a ZK circuit. It verifies the *last* UPS step proof, verifies the user's ZK signature proof, and enforces the final End Cap constraints.
- **Key Inputs/Witness:**
 - `ups_session_proof_tree_height` , `ups_circuit_whitelist_tree_height` : Parameters.
 - `network_magic` : Parameter.
 - `UPSEndCapFromProofTreeGadgetInput` : Witness containing:
 - `VerifyPreviousUPSStepProofInProofTreeInput` : Witness for verifying the last UPS step.
 - `AttestProofInTreeInput` : Witness for verifying the ZK signature proof.
 - `user_public_key_param` : Hash representing signature parameters.
 - `nonce` : The nonce used for signing.
 - `slots_modified` : Total storage slots modified.
- **Key Outputs/Computed Values:**
 - `end_cap_core_gadget` : The core gadget performing final checks.
 - `current_proof_tree_root` : The root of the UPS proof tree.
- **Core Logic/Constraints:**
 - Instantiates `VerifyPreviousUPSStepProofInProofTreeGadget` to verify the last UPS step.

- Instantiates `AttestProofInTreeGadget` to verify the ZK signature proof.
- Connects the `attested_proof_tree_root` from the signature proof verification to the `current_proof_tree_root` from the previous step verification (ensuring both proofs are in the same tree).
- Defines empty debt tree root constants.
- Instantiates `UPSEndCapCoreGadget`, passing outputs from the verification gadgets (`previous_step_header_gadget`, signature proof hashes) and witness values (`nonce`, `slots_modified`, `user_public_key_param`) along with constants.
- **Assumptions:** Relies on assumptions of sub-gadgets. Assumes witness data is valid.
- **Role:** The top-level gadget within the End Cap *circuit*, ensuring the final UPS state is valid, linked to the previous step, and properly authorized by a valid ZK signature, all within the same consistent UPS proof tree.

UPSStartStepGadget

- **File:** `ups_start.rs`
- **Purpose:** Initializes a User Proving Session. It takes the user's initial state (from the last finalized block's checkpoint) and sets up the starting header for the UPS circuit chain.
- **Key Inputs/Witness:**
 - `UPSStartStepInput` : Witness containing:
 - `UserProvingSessionHeader` : The *expected* starting header.
 - `PsyCheckpointLeaf` : Witness for the checkpoint leaf data.
 - `PsyCheckpointGlobalStateRoots` : Witness for the global state roots within the checkpoint.
 - `MerkleProof` : Proof linking the checkpoint leaf to the checkpoint tree root.
 - `MerkleProof` : Proof linking the user's starting leaf hash to the global user tree root.
- **Key Outputs/Computed Values:**
 - `header_gadget` : The validated starting header gadget.
- **Core Logic/Constraints:**
 - **Constraint Set 1 (Start Session Context):**

- Verifies `checkpoint_tree_proof` consistency with `header_gadget.session_start_context` (root, leaf hash, ID).
 - Verifies consistency between `checkpoint_leaf_gadget`, `state_roots_gadget`, and the `header_gadget` (checkpoint leaf hash, global chain root).
 - Verifies `user_tree_proof` root matches the `user_tree_root` in `state_roots_gadget`.
 - Verifies `user_tree_proof` value matches `header_gadget.session_start_context.start_session_user_leaf_hash`.
 - Verifies `user_tree_proof` index matches the `user_id` in the header's start leaf.
 - **Constraint Set 2 (Current State Initialization):**
 - Computes the hash of the expected *current* user leaf (start leaf with `last_checkpoint_id` updated to the session's checkpoint ID).
 - Asserts this computed hash matches the hash of the `header_gadget.current_state.user_leaf`.
 - Asserts the `deferred_tx_debt_tree_root` and `inline_tx_debt_tree_root` in `header_gadget.current_state` match the known empty tree root constants.
 - Asserts `tx_hash_stack` is zero hash and `tx_count` is zero.
 - **Assumptions:** Assumes the witness data (header, proofs, leaves, roots) is valid for constraints to pass. Assumes the empty tree root constants are correct.
 - **Role:** Securely bootstraps the User Proving Session, anchoring the initial state to a verified checkpoint and user leaf from the last finalized block and ensuring the session starts with clean debt trees and transaction counts.
-

GUTA Gadgets (Global User Tree Aggregation)

These gadgets are components used within the circuits run by the decentralized proving network to aggregate proofs from multiple users into a

single block proof.

GUTASTatsGadget

- **File:** `guta_stats.rs`
- **Purpose:** Represents and processes statistics aggregated during the GUTA process.
- **Key Inputs/Witness:** `fees_collected`, `user_ops_processed`, `total_transactions`, `slots_modified`.
- **Key Outputs/Computed Values:** Can compute a hash of the stats.
- **Core Logic/Constraints:** Primarily a data structure. Includes a `combine_with` method to add stats from two gadgets together (used during aggregation). `to_hash` packs the stats into a `HashOutTarget`.
- **Assumptions:** Assumes input target values are correct.
- **Role:** Tracks key metrics about the aggregated user operations within a GUTA proof branch.

GlobalUserTreeAggregatorHeaderGadget

- **File:** `guta_header.rs`
- **Purpose:** Represents the public inputs (header) of a GUTA proof. It encapsulates the essential information about the aggregation step.
- **Key Inputs/Witness:**
 - `guta_circuit_whitelist`: Root hash of the allowed GUTA circuits.
 - `checkpoint_tree_root`: Root hash of the checkpoint tree relevant to this aggregation step.
 - `state_transition`: A `SubTreeNodeStateTransitionGadget` representing the change in the Global User Tree (`GUSR`) this proof covers.
 - `stats`: A `GUTASTatsGadget` containing aggregated statistics.
- **Key Outputs/Computed Values:** Computes the hash of the entire header.
- **Core Logic/Constraints:** Data structure. `to_hash` computes the final header hash by combining hashes of `state_transition`, `stats`, `checkpoint_tree_root`, and `guta_circuit_whitelist`.
- **Assumptions:** Assumes input targets/gadgets are correctly formed.

- **Role:** Defines the standard public interface for all GUTA circuits, ensuring consistent information is passed and verified during recursive aggregation.

VerifyEndCapProofGadget

- **File:** `verify_end_cap.rs`
- **Purpose:** Verifies a user's End Cap proof within the GUTA aggregation process. It checks the proof's validity, ensures it used the correct End Cap circuit, verifies the user's claimed checkpoint root is historical, and extracts the state transition and stats.
- **Key Inputs/Witness:**
 - `proof_common_data`, `verifier_data_cap_height` : Parameters for proof verification.
 - `known_end_cap_fingerprint_hash` : Constant representing the hash of the official End Cap circuit's verifier data.
 - `UPSEndCapResultCompact` : Witness for the result claimed by the End Cap proof.
 - `GUTASTats` : Witness for the stats claimed by the End Cap proof.
 - `MerkleProofCore` : Witness for the historical checkpoint root proof.
 - `ProofWithPublicInputs` : The End Cap proof itself.
 - `VerifierOnlyCircuitData` : Verifier data matching the proof.
- **Key Outputs/Computed Values:**
 - (Implements `ToGUTAHeader`) Outputs a `GlobalUserTreeAggregatorHeaderGadget` representing the state transition performed by this user.
- **Core Logic/Constraints:**
 - Verifies the `proof_target` using the provided `verifier_data`.
 - Computes the `proof_fingerprint` from the `verifier_data`.
 - Asserts `proof_fingerprint` matches `known_end_cap_fingerprint_hash`.
 - Instantiates `UPSEndCapResultCompactGadget` and `GUTASTatsGadget` from witness.
 - Computes the expected public inputs hash by hashing the result and stats gadgets.

- Asserts the computed hash matches the hash derived from `proof_target.public_inputs`.
- Verifies the `checkpoint_historical_merkle_proof`, connecting its `historical_root` to the `checkpoint_tree_root_hash` from the end cap result gadget.
- Constructs the output `GlobalUserTreeAggregatorHeaderGadget` :
 - Uses a default/input `guta_circuit_whitelist`.
 - Uses the `current_root` from the historical proof as the `checkpoint_tree_root`.
 - Creates a `SubTreeNodeStateTransitionGadget` using start/end user leaf hashes and user ID from the result gadget, marking the level as `GLOBAL_USER_TREE_HEIGHT`.
 - Includes the verified `guta_stats`.
- **Assumptions:** Assumes witness data (proofs, results, stats) is valid. Assumes `known_end_cap_fingerprint_hash` is correct.
- **Role:** The entry point for incorporating a user's completed UPS into the GUTA aggregation tree. Verifies the user's session proof and translates its result into the standard GUTA header format for further aggregation.

VerifyGUTAProofGadget

- **File:** `verify_guta_proof.rs`
- **Purpose:** Verifies a GUTA proof generated by a lower level in the aggregation hierarchy. Checks the proof validity, ensures it used an allowed GUTA circuit, and extracts its header.
- **Key Inputs/Witness:**
 - `proof_common_data`, `verifier_data_cap_height` : Parameters.
 - `GlobalUserTreeAggregatorHeader` : Witness for the header claimed by the proof being verified.
 - `MerkleProof` : Witness proving the sub-proof's circuit fingerprint is in the GUTA circuit whitelist.
 - `ProofWithPublicInputs` : The GUTA proof itself.
 - `VerifierOnlyCircuitData` : Verifier data for the proof.
- **Key Outputs/Computed Values:**
 - `guta_proof_header_gadget` : The verified header gadget of the sub-proof.

- **Core Logic/Constraints:**
 - Verifies the `proof_target` using the provided `verifier_data`.
 - Computes the `proof_fingerprint` from the `verifier_data`.
 - Verifies the `guta_whitelist_merkle_proof`.
 - Connects the `guta_proof_header_gadget.guta_circuit_whitelist` to the `guta_whitelist_merkle_proof.root`.
 - Computes the expected public inputs hash from the `guta_proof_header_gadget`.
 - Asserts this matches the hash derived from `proof_target.public_inputs`.
 - Asserts the `guta_whitelist_merkle_proof.value` matches the computed `proof_fingerprint`.
- **Assumptions:** Assumes witness data is valid.
- **Role:** The core recursive verification step within GUTA. Allows aggregation circuits to securely incorporate results from lower-level GUTA proofs.

TwoNCAStateTransitionGadget

- **File:** `two_nca_state_transition.rs`
- **Purpose:** Combines the state transitions from two child GUTA proofs (`a_header`, `b_header`) that modify different parts of the Global User Tree. It uses a Nearest Common Ancestor (NCA) proof to compute the resulting state transition at their common parent node in the tree.
- **Key Inputs/Witness:**
 - `a_header`, `b_header` : The headers of the two child GUTA proofs.
 - `UpdateNearestCommonAncestorProof` : Witness containing the NCA proof data.
- **Key Outputs/Computed Values:**
 - `new_guta_header` : The combined GUTA header representing the transition at the NCA.
- **Core Logic/Constraints:**
 - Instantiates `UpdateNearestCommonAncestorProofOptGadget`.
 - Connects `a_header.checkpoint_tree_root` to `b_header.checkpoint_tree_root`.
 - Connects `a_header.guta_circuit_whitelist` to `b_header.guta_circuit_whitelist`.

- Connects `a_header.state_transition` fields (old/new value, index, level) to the `child_a` fields in the NCA proof gadget.
- Connects `b_header.state_transition` fields similarly to `child_b`.
- Combines stats: `new_stats = a_header.stats.combine_with(b_header.stats)`.
- Constructs `new_guta_header`:
 - Uses whitelist/checkpoint root from children (they must match).
 - Creates `state_transition` using the `old/new_nearest_common_ancestor_value`, `index`, and `level` from the NCA proof gadget.
 - Includes the `new_stats`.
- **Assumptions:** Assumes input headers are valid (verified previously). Assumes the NCA proof witness is valid. Assumes children operate on the same checkpoint and whitelist.
- **Role:** A fundamental building block for parallel aggregation. Allows merging results from independent branches of the GUTA proof tree efficiently using NCA proofs.

GUTAHeaderLineProofGadget

- **File:** `guta_line.rs`
- **Purpose:** Aggregates a GUTA proof state transition *upwards* along a direct line towards the root of the Global User Tree realm. Used when a node only has one child contributing to the update in that part of the tree.
- **Key Inputs/Witness:**
 - `global_user_tree_realm_height`, `global_user_tree_height`: Parameters.
 - `child_proof_header`: The header of the single child GUTA proof.
 - `siblings`: Witness containing the Merkle sibling hashes needed to recompute the root along the path.
- **Key Outputs/Computed Values:**
 - `new_guta_header`: The GUTA header representing the state transition at the top of the line (realm root).
- **Core Logic/Constraints:**

- Instantiates `SubTreeNodeTopLineGadget` , providing the child's state transition. This gadget internally performs the Merkle path hashing using the provided siblings.
- Constructs `new_guta_header` :
 - Copies whitelist/checkpoint root/stats from the child.
 - Uses the `new_state_transition` computed by the `SubTreeNodeTopLineGadget` .
- **Assumptions:** Assumes `child_proof_header` is valid. Assumes sibling hashes in the witness are correct.
- **Role:** Efficiently propagates a state change up the GUTA tree when no merging (NCA) is required. Used to bring proofs to a common level before NCA or to reach the final realm root.

VerifyGUTAProofToLineGadget

- **File:** `verify_guta_proof_to_line.rs`
- **Purpose:** Combines verifying a lower-level GUTA proof with aggregating its state transition upwards using a line proof.
- **Key Inputs/Witness:**
 - `proof_common_data` , `verifier_data_cap_height` : Parameters.
 - `global_user_tree_realm_height` , `global_user_tree_height` : Parameters.
 - `MerkleProofCore` : GUTA whitelist proof witness.
 - `GlobalUserTreeAggregatorHeader` : Child proof header witness.
 - `ProofWithPublicInputs` , `VerifierOnlyCircuitData` : Child GUTA proof and verifier data witness.
 - `top_line_siblings` : Sibling hashes for the line proof witness.
- **Key Outputs/Computed Values:**
 - `new_guta_header` : The final GUTA header at the top of the line.
- **Core Logic/Constraints:**
 - Instantiates `VerifyGUTAProofGadget` to verify the child proof.
 - Instantiates `GUTAHeaderLineProofGadget` , using the verified header from the first gadget as input.
- **Assumptions:** Relies on assumptions of sub-gadgets. Assumes witness data is valid.

- **Role:** A common pattern in GUTA aggregation: verify a child proof and immediately propagate its result upwards via a line proof.

GUTARegisterUserCoreGadget

- **File:** `guta_register_user_core.rs`
- **Purpose:** Handles the core logic for registering a *single* new user in the Global User Tree (GUSR). It verifies the update proof that inserts the new user leaf.
- **Key Inputs/Witness:**
 - `global_user_tree_realm_height`, `global_user_tree_height`: Parameters.
 - `default_user_state_tree_root`: Constant.
 - `input_height_target`: Optional target for variable height proof.
 - `public_key`: The public key hash for the new user (can be witness or input).
 - `DeltaMerkleProofCore`: Witness for the GUSR tree update.
- **Key Outputs/Computed Values:**
 - `user_id`: The ID (index) of the newly registered user.
 - `user_leaf_hash`: The hash of the newly created user leaf.
 - `state_transition`: Represents the GUTA state transition for this single registration.
- **Core Logic/Constraints:**
 - Instantiates `VariableHeightDeltaMerkleProofOptGadget` for the GUSR update.
 - Asserts the `old_value` in the proof is the zero hash (ensuring it's an insertion into an empty slot).
 - Creates the default `PsyUserLeafGadget` using the proof's `index` (user ID), the `public_key`, and `default_user_state_tree_root`.
 - Computes the `user_leaf_hash`.
 - Asserts the `new_value` in the proof matches the computed `user_leaf_hash`.
 - Calculates the `state_transition` based on the delta proof's old/new roots, height, and computed parent index.
- **Assumptions:** Assumes witness proof and public key (if witness) are valid. Assumes `default_user_state_tree_root` is correct.

- **Role:** The lowest-level gadget for handling user registration state changes in GUTA.

GUTARegisterUserFullGadget

- **File:** `guta_register_user_full.rs`
- **Purpose:** Extends the core registration by adding verification against a *user registration tree*. This tree (presumably managed off-chain or via a separate mechanism) maps user IDs to public keys. This gadget ensures the public key used for registration matches the one committed to in the registration tree.
- **Key Inputs/Witness:**
 - (Inherits inputs from Core gadget).
 - `MerkleProofCore` : Witness proving the `public_key` exists at the correct `index` (user ID) in the `user_registration_tree`.
- **Key Outputs/Computed Values:**
 - (Inherits outputs from Core gadget).
 - `user_registration_tree_root` : The root of the user registration tree.
- **Core Logic/Constraints:**
 - Instantiates `MerkleProofGadget` for the user registration tree.
 - Maps the proof's index bits to an expected user ID.
 - Asserts the `value` (public key) from the registration tree proof is non-zero.
 - Instantiates `GUTARegisterUserCoreGadget`, passing the `value` from the registration proof as the `public_key`.
 - Asserts the `user_id` from the Core gadget matches the `expected_user_id` derived from the registration proof index.
- **Assumptions:** Relies on Core gadget assumptions. Assumes the user registration tree proof witness is valid.
- **Role:** Adds a layer of validation, ensuring user registrations correspond to pre-committed public keys in a dedicated registration structure.

GUTARegisterUsersGadget

- **File:** `guta_register_users.rs`

- **Purpose:** Aggregates multiple user registration operations (using `GUTARRegisterUserFullGadget`) sequentially within a single circuit. Handles padding/disabling for a fixed maximum number of users.
- **Key Inputs/Witness:**
 - (Inherits inputs from Full gadget).
 - `max_users` : Parameter.
 - `GUTARRegisterUserFullInput[]` : Array witness for each potential user registration (proofs).
 - `register_user_count` : Witness target indicating the *actual* number of users being registered ($\leq$ `max_users`).
- **Key Outputs/Computed Values:**
 - `state_transition` : The aggregate state transition covering all registered users.
 - `user_registration_tree_root` : Root of the registration tree (taken from the first user, checked for consistency).
- **Core Logic/Constraints:**
 - Instantiates `max_users` instances of `GUTARRegisterUserFullGadget` .
 - Asserts `register_user_count` is non-zero.
 - Iterates from the second user onwards:
 - Compares loop index `i` with `register_user_count` to determine if the current user slot `is_disabled` .
 - If *not* disabled:
 - Connects the current user's `old_global_user_tree_root` to the previous user's `new_global_user_tree_root` .
 - Connects the current user's `user_registration_tree_root` to the root from the first user (ensuring consistency).
 - Connects proof heights.
 - Updates the aggregate `state_transition.new_node_value` to the current user's `new_global_user_tree_root` .
 - Selects the final `new_node_value` based on the last *enabled* user's output.
- **Assumptions:** Relies on Full gadget assumptions. Assumes witness array and count are valid. Assumes dummy inputs are used correctly for padding.

- **Role:** Allows batching multiple user registrations into a single GUTA proof step, improving aggregation efficiency.

GUTAOnlyRegisterUsersGadget

- **File:** `guta_only_register_users_gadget.rs`
- **Purpose:** A specialized GUTA gadget that *only* performs user registration (using `GUTAResisterUsersGadget`) and assumes *no other state changes* (zero stats).
- **Key Inputs/Witness:**
 - `guta_circuit_whitelist, checkpoint_tree_root`: Inputs (likely from a previous step or constant).
 - (Inherits inputs for `GUTAResisterUsersGadget`).
- **Key Outputs/Computed Values:**
 - `new_guta_header`: The GUTA header representing *only* the registration state change.
- **Core Logic/Constraints:**
 - Instantiates `GUTAResisterUsersGadget`.
 - Constructs `new_guta_header`:
 - Uses input `guta_circuit_whitelist` and `checkpoint_tree_root`.
 - Uses the `state_transition` from the `GUTAResisterUsersGadget`.
 - Creates a zeroed `GUTASStatsGadget`.
- **Assumptions:** Relies on `GUTAResisterUsersGadget` assumptions. Assumes the provided whitelist/checkpoint roots are correct for this context.
- **Role:** Provides a dedicated gadget for GUTA steps that solely involve registering new users.

GUTAResisterUsersBatchGadget

- **File:** `guta_register_users_batch.rs`
- **Purpose:** Combines verification of a previous GUTA proof (brought up to a certain tree level via a line proof) with a subsequent batch registration of new users.

- **Key Inputs/Witness:**
 - (Inherits inputs for `VerifyGUTAProofToLineGadget`).
 - (Inherits inputs for `GUTAResisterUsersGadget`).
- **Key Outputs/Computed Values:**
 - `new_guta_header` : The combined GUTA header.
- **Core Logic/Constraints:**
 - Instantiates `VerifyGUTAProofToLineGadget` (`verify_to_line_gadget`).
 - Instantiates `GUTAResisterUsersGadget` (`register_users_gadget`).
 - Connects the state transitions:
 - `line_state_transition.node_index == register_users_state_transition.node_index`.
 - `line_state_transition.node_level == register_users_state_transition.node_level`.
 - `line_state_transition.new_node_value == register_users_state_transition.old_node_value` (ensures the registration starts from the state reached by the verified line proof).
 - Constructs `new_guta_header` :
 - Uses whitelist/checkpoint root/stats from the line proof header.
 - Creates combined `state_transition: old_node_value` from line, `new_node_value` from registration, index/level from line (must match registration).
- **Assumptions:** Relies on assumptions of sub-gadgets. Assumes witness data is valid.
- **Role:** Handles a common GUTA pattern: verifying a previous aggregation step and then applying a batch of user registrations originating from the state achieved by that previous step.

GUTANoChangeGadget

- **File:** `guta_no_change_gadget.rs`
- **Purpose:** Represents a GUTA step where the Global User Tree (`GUSR`) does *not* change. It primarily serves to advance the `checkpoint_tree_root` based on a new checkpoint.
- **Key Inputs/Witness:**

- `guta_circuit_whitelist` : Input constant/parameter.
 - `checkpoint_tree_height` : Parameter.
 - `MerkleProofCore` : Witness proving a `checkpoint_leaf` exists in the `checkpoint_tree`.
 - `PsyCheckpointLeafCompactWithStateRoots` : Witness for the checkpoint leaf data.
 - **Key Outputs/Computed Values:**
 - `new_guta_header` : GUTA header indicating no user tree change but potentially a new checkpoint root.
 - **Core Logic/Constraints:**
 - Verifies the `checkpoint_tree_proof` (append-only).
 - Verifies the hash of `checkpoint_leaf_gadget` matches the `value` in the proof.
 - Constructs `new_guta_header` :
 - Uses input `guta_circuit_whitelist`.
 - Uses the `checkpoint_tree_proof.root` as the `checkpoint_tree_root`.
 - Creates a "no-op" `state_transition`: `old_node_value` and `new_node_value` are both set to the `user_tree_root` from the `checkpoint_leaf_gadget`, with index/level zero.
 - Creates a zeroed `GUTASTatsGadget`.
 - **Assumptions:** Assumes witness proof and leaf data are valid. Assumes `guta_circuit_whitelist` is correct.
 - **Role:** Allows the GUTA aggregation to incorporate new checkpoints even if no user state changed during that period, keeping the aggregated checkpoint root up-to-date.
-

Circuit Definition Files (Higher Level)

These files define complete ZK circuits, orchestrating various gadgets to perform end-to-end tasks like starting a session, processing transactions, or finalizing a session.

UPSSStartSessionCircuit

- **File:** `ups_start.rs` (Circuit definition wrapping `UPSSStartStepGadget`)
- **Purpose:** The circuit executed to begin a User Proving Session.
- **Core Logic:** Primarily uses the `UPSSStartStepGadget` to verify the initial state against a checkpoint and set up the starting header. Computes the final public inputs hash by wrapping the inner header hash (`start_step_gadget.header_gadget.to_hash()`) with tree awareness information (using `compute_tree_aware_proof_public_inputs`), assuming the proof tree starts empty (`empty_ups_proof_tree_root_target`).
- **What it Proves:** That a valid starting `UserProvingSessionHeader` has been constructed, correctly anchored to a specific, verified checkpoint and user leaf state from the last finalized block, and that the session starts with empty debt trees and zero transaction count.
- **Assumptions:** Assumes the witness (`UPSSStartStepInput`) containing the starting header, proofs, and state roots is valid *before* constraints are applied. Assumes the provided `empty_ups_proof_tree_root` constant is correct.
- **Role:** Securely initializes the recursive proof chain for a user's local transactions.

UPSCFCStandardTransactionCircuit

- **File:** `ups_cfc_standard.rs` (Circuit definition wrapping `VerifyPreviousUPSSStepProofInProofTreeGadget` and `UPSVerifyCFCStandardStepGadget`)
- **Purpose:** Processes a single, standard Contract Function Call (CFC) transaction within an ongoing User Proving Session.
- **Core Logic:**
 - Uses `VerifyPreviousUPSSStepProofInProofTreeGadget` to verify the proof of the *previous* UPS step.
 - Uses `UPSVerifyCFCStandardStepGadget` (which internally uses verification and state delta gadgets) to:
 - Verify the CFC proof exists in the current proof tree.
 - Verify the CFC function is valid.
 - Calculate the state changes based on the CFC execution.

- Connects the `current_proof_tree_root` from the previous step verification to the standard step gadget.
- Computes the final public inputs hash by wrapping the *new* header hash (`standard_cfc_step_gadget.new_header_gadget.to_hash()`) with tree awareness information (`current_proof_tree_root`).
- **What it Proves:** That given a valid previous UPS step proof and header, executing the specified CFC transaction (verified to exist and be valid) correctly transitions the UPS state to the new header state.
- **Assumptions:** Assumes the witness (`UPSCFCStandardTransactionCircuitInput`) containing the previous step proof details, the current transaction details (proofs, context), and circuit whitelist proofs is valid. Relies on the validity of the previous UPS step proof (which is verified internally).
- **Role:** The workhorse circuit for processing most user transactions locally within the recursive UPS chain.

UPSCFCDeferredTransactionCircuit

- **File:** `ups_cfc_deferred_tx.rs` (Circuit definition wrapping `VerifyPreviousUPSStepProofInProofTreeGadget` and `UPSVerifyPopDeferredTxStepGadget`)
- **Purpose:** Processes a transaction that pays back a deferred transaction debt within an ongoing User Proving Session.
- **Core Logic:**
 - Uses `VerifyPreviousUPSStepProofInProofTreeGadget` to verify the previous UPS step proof.
 - Uses `UPSVerifyPopDeferredTxStepGadget` to:
 - Verify the removal of the correct deferred transaction item from the debt tree.
 - Process the CFC itself using corrected starting state assumptions.
 - Connects the `current_proof_tree_root` appropriately.
 - Computes the final public inputs hash by wrapping the new header hash (`deferred_tx_cfc_step_gadget...new_header_gadget.to_hash()`) with tree awareness information.

- **What it Proves:** That given a valid previous UPS step, the specified deferred transaction debt was correctly removed, and executing the corresponding CFC transaction correctly transitions the UPS state to the new header state.
- **Assumptions:** Assumes the witness (`UPSCFCDeferredTransactionCircuitInput`) is valid. Relies on the validity of the previous UPS step proof.
- **Role:** Enables the settlement of deferred transaction debts within the user's local proof chain.

UPSStandardEndCapCircuit

- **File:** `end_cap.rs` (Circuit definition wrapping `UPSEndCapFromProofTreeGadget` and proof verification gadgets)
- **Purpose:** The final circuit in a User Proving Session. It verifies the last UPS step, verifies the user's ZK signature proof, enforces final state conditions (e.g., empty debt trees), and generates the final public outputs (End Cap Result hash and GUTA Stats hash).
- **Core Logic:**
 - Uses `UPSEndCapFromProofTreeGadget` which orchestrates:
 - Verification of the *last* UPS step proof (`VerifyPreviousUPSStepProofInProofTreeGadget`).
 - Verification of the ZK signature proof (`AttestProofInTreeGadget`).
 - Enforcement of final conditions via `UPSEndCapCoreGadget`.
 - (Original `end_cap.rs` also includes `VerifyAggProofGadget`): Verifies a proof about the UPS proof tree itself (e.g., ensuring the tree was built using allowed aggregation circuits). This connects the user's session to the global proof aggregation infrastructure rules.
 - Connects the proof tree root from the UPS gadget to the state transition end of the verified aggregation proof.
 - Connects the UPS circuit whitelist root from the UPS gadget to a known constant or input, ensuring the UPS steps used allowed circuits.
 - Connects the proof tree aggregation circuit whitelist root to a known constant (ensuring the tree proof used allowed circuits).

- Computes the final public inputs hash by hashing the `end_cap_result_gadget` hash and the `guta_stats` hash.
 - **What it Proves:** That the entire User Proving Session was valid (by verifying the last step), the final state is consistent (empty debts, correct nonce/checkpoint progression), the session is authorized by a valid ZK signature corresponding to the user's key, the session used allowed UPS circuits, and the session's proof tree was built correctly using allowed aggregation circuits. It outputs the final state transition hash and stats hash.
 - **Assumptions:** Assumes the witness (`UPSEndCapFromProofTreeGadgetInput`, aggregation proof witnesses) is valid. Assumes the known whitelist root constants are correct.
 - **Role:** Securely concludes the user's local proving session, producing a verifiable proof and result ready for submission to the GUTA layer. Links the user's activity to global rules via whitelist checks.
-

Circuits.md

This document describes the end-to-end flow of circuits involved in processing user transactions and aggregating them into a final block proof within the Psy system. It highlights the assumptions made at each stage and how they are progressively verified, ultimately enabling horizontal scalability.

Phase 1: User Proving Session (UPS) - Local Execution

This phase happens locally on the user's device (or via a delegated prover). The user builds a recursive chain of proofs for their transactions within a single block context.

1. `UPSStartSessionCircuit`

- **Purpose:** Initializes the proving session for a user based on the last finalized blockchain state.

- **What it Proves:**
 - The starting `UserProvingSessionHeader` is valid.
 - This header is correctly anchored to a specific `checkpoint_leaf_hash` which exists at `checkpoint_id` within the `checkpoint_tree_root` from the *last finalized block*.
 - The `session_start_context` within the header accurately reflects the user's state (`start_session_user_leaf_hash` , `user_id` , etc.) as found in the `user_tree_root` associated with the starting checkpoint.
 - The `current_state` within the starting header is correctly initialized (user leaf `last_checkpoint_id` updated, debt trees empty, tx count/stack zero).
- **Assumptions:**
 - The witness data (`UPSSstartStepInput`) provided by the user (fetching state from the last block) is correct *initially*. Constraints verify its consistency.
 - The constant `empty_ups_proof_tree_root` (representing the start of the recursive proof tree for this session) is correct.
- **How Assumptions are Discharged:** Internal consistency checks verify the relationships between the provided header, checkpoint leaf, state roots, user leaf, and the Merkle proofs linking them. The assumption about the *previous block's* `checkpoint_tree_root` being correct is implicitly carried forward, as this circuit uses it as the basis for initialization.
- **Contribution to Horizontal Scalability:** Establishes a user-specific, isolated starting point based on globally finalized state, allowing this session to proceed independently of other users' sessions within the *same* new block.
- **High-Level Functionality:** Securely starts a user's transaction batch processing.

2. `UPSCFCStandardTransactionCircuit` (Executed potentially multiple times)

- **Purpose:** Processes a single standard transaction (contract call) within the user's ongoing session, extending the recursive proof chain.
- **What it Proves:**
 - The ZK proof for the *previous UPS step* is valid.

- The previous step's proof was generated by a circuit listed in the `ups_circuit_whitelist_root` specified in the previous step's header.
- The public inputs (header hash) of the previous step's proof match the provided `previous_step_header`.
- The ZK proof for the *current Contract Function Call (CFC)* exists within the current `current_proof_tree_root`.
- This CFC proof corresponds to a function registered in the `GCON` tree (via checkpoint context).
- Executing this CFC correctly transitions the state from the `previous_step_header` to the `new_header_gadget` state (updating `CSTATE -> UCON` root, debt trees, tx count/stack).
- **Assumptions:**
 - The witness data (`UPSCFCStandardTransactionCircuitInput`) for this specific step (CFC proof, state delta witnesses, previous step proof info) is correct initially.
 - The `current_proof_tree_root` provided matches the actual root of the recursive proof tree being built.
- **How Assumptions are Discharged:**
 - Verifies the previous step's proof using `VerifyPreviousUPSSStepProofInProofTreeGadget`. This discharges the assumption about the previous step's validity and its public inputs.
 - Verifies the CFC proof and its link to the contract state using `UPSVerifyCFCStandardStepGadget`.
 - Verifies the state delta logic, ensuring the transition is correct based on witness data.
 - The assumption about the `current_proof_tree_root` is passed implicitly to the *next* step or the End Cap circuit.
- **Contribution to Horizontal Scalability:** User processes transactions locally and serially *for themselves*, maintaining self-consistency without interacting with other users' *current* block activity.
- **High-Level Functionality:** Securely executes and proves individual smart contract interactions locally.

3. `UPSCFCDeferredTransactionCircuit` (Executed if applicable)

- **Purpose:** Processes a transaction that settles a deferred debt, then executes the main CFC logic.
- **What it Proves:** Similar to the standard circuit, but *additionally* proves:
 - A specific deferred transaction item was removed from the `deferred_tx_debt_tree`.
 - The item removed corresponds exactly to the call data of the CFC being executed.
 - The subsequent CFC state transition starts from the state *after* the debt was removed.
- **Assumptions:** Same as the standard circuit, plus assumes the witness for the deferred transaction removal proof (`DeltaMerkleProofGadget`) is correct initially.
- **How Assumptions are Discharged:** Verifies previous step proof. Verifies the debt removal proof and its consistency with the CFC call data. Verifies the subsequent state delta.
- **Contribution to Horizontal Scalability:** Same as standard transaction circuit (local serial execution).
- **High-Level Functionality:** Enables settlement of asynchronous transaction debts within the local proving flow.

4. `UPSStandardEndCapCircuit`

- **Purpose:** Finalizes the user's entire proving session for the block.
- **What it Proves:**
 - The proof for the *last UPS transaction step* is valid and used an allowed circuit.
 - The ZK proof for the user's signature (authorizing the session) is valid and exists in the same UPS proof tree.
 - The signature corresponds to the user's registered public key (derived from signature proof parameters).
 - The signature payload (`PsyUserProvingSessionSignatureDataCompact`) correctly reflects the session's start/end user leaves, checkpoint, final tx stack, and tx count.
 - The nonce used in the signature is valid (incremented).
 - The final UPS state shows both `deferred_tx_debt_tree_root` and `inline_tx_debt_tree_root` are empty (all debts settled).

- The `last_checkpoint_id` in the final user leaf matches the session's `checkpoint_id` and has progressed correctly.
 - (*If aggregation proof verification included*): The UPS proof tree itself was constructed using circuits from a known `proof_tree_circuit_whitelist_root`.
- **Assumptions:**
 - Witness data (`UPSEndCapFromProofTreeGadgetInput` , potentially agg proof witness) is correct initially.
 - Known constants (`network_magic` , empty debt roots, `known_ups_circuit_whitelist_root` , `known_proof_tree_circuit_whitelist_root`) are correct.
- **How Assumptions are Discharged:**
 - Verifies the last UPS step proof.
 - Verifies the ZK signature proof.
 - Connects signature data to the final UPS header state.
 - Checks nonce, checkpoint ID, empty debt trees.
 - Verifies proofs against whitelists using provided roots.
 - The *output* of this circuit (the End Cap proof) now implicitly carries the assumption that the *starting* `checkpoint_tree_root` (used in the `UPSStartSessionCircuit`) was correct. All *internal* UPS assumptions have been discharged.
- **Contribution to Horizontal Scalability:** Creates a single, verifiable proof representing *all* of a user's activity for the block. This proof can now be processed in parallel with proofs from other users by the GUTA layer.
- **High-Level Functionality:** Securely concludes a user's transaction batch, authorizes it, and packages it for network aggregation.

Phase 2: Global User Tree Aggregation (GUTA) - Parallel Network Execution

The Decentralized Proving Network (DPN) takes End Cap proofs (and potentially other GUTA proofs like user registrations) from many users and aggregates them in parallel. This involves specialized GUTA circuits.

(Note: The provided files focus heavily on UPS and GUTA gadgets. The exact structure of the GUTA circuits using these gadgets is inferred but follows standard recursive

proof aggregation patterns.)

Example GUTA Circuits (Inferred):

5. GUTAProcessEndCapCircuit (Hypothetical)

- **Purpose:** To take a user's validated `UPSStandardEndCapCircuit` proof and integrate its state change into the GUTA proof hierarchy.
- **Core Logic:** Uses `VerifyEndCapProofGadget`.
- **What it Proves:**
 - The End Cap proof is valid and used the correct circuit (`known_end_cap_fingerprint_hash`).
 - The `checkpoint_tree_root` claimed by the user in the End Cap result existed historically.
 - Outputs a standard `GlobalUserTreeAggregatorHeader` representing the user's `GUSR` tree state transition (start leaf hash -> end leaf hash at the user's ID index) and stats.
- **Assumptions:**
 - Witness (End Cap proof, result, stats, historical proof) is correct initially.
 - The `known_end_cap_fingerprint_hash` constant is correct.
 - A `default_guta_circuit_whitelist` root is provided or known.
- **How Assumptions are Discharged:** Verifies the End Cap proof and historical checkpoint proof. Packages the result into a standard GUTA header. The assumption about the `default_guta_circuit_whitelist` is passed upwards. The assumption about the *current* `checkpoint_tree_root` (from the historical proof) is passed upwards.
- **Contribution to Horizontal Scalability:** Allows individual user session results to be verified independently and prepared for parallel aggregation.
- **High-Level Functionality:** Validates and incorporates user end-of-session proofs into the global aggregation process.

6. GUTARegisterUserCircuit (Hypothetical)

- **Purpose:** To process the registration of one or more new users.
- **Core Logic:** Uses `GUTAOnlyRegisterUsersGadget` (which uses `GUTARegisterUsersGadget`, `GUTARegisterUserFullGadget`, `GUTARegisterUserCoreGadget`).

- **What it Proves:**
 - For each registered user, their `public_key` was correctly inserted at their `user_id` index in the `GUSR` tree (transitioning from zero hash to the new user leaf hash).
 - The `public_key` used matches an entry in the `user_registration_tree_root`.
 - Outputs a `GlobalUserTreeAggregatorHeader` representing the aggregate `GUSR` state transition for all registered users, with zero stats.
- **Assumptions:**
 - Witness (registration proofs, user count) is correct initially.
 - `guta_circuit_whitelist` and `checkpoint_tree_root` inputs are correct for this context.
 - `default_user_state_tree_root` constant is correct.
- **How Assumptions are Discharged:** Verifies delta proofs for `GUSR` insertion and Merkle proofs against the registration tree. Outputs a standard GUTA header, passing assumptions about whitelist/checkpoint upwards.
- **Contribution to Horizontal Scalability:** User registration can be batched and potentially processed in parallel branches of the GUTA tree.
- **High-Level Functionality:** Securely adds new users to the system state.

7. `GUTAAggregationCircuit` (Hypothetical - Multiple Variants)

- **Purpose:** To combine the results (headers) from two or more lower-level GUTA proofs (which could be End Cap results, registrations, or previous aggregations).
- **Core Logic:**
 - Verifies each input GUTA proof using `VerifyGUTAProofGadget`.
 - Ensures all input proofs used circuits from the same `guta_circuit_whitelist` and reference the same `checkpoint_tree_root`.
 - Combines the `state_transitions` from the input proofs:
 - If transitions are on different branches, uses `TwoNCASStateTransitionGadget` with an NCA proof.
 - If transitions are on the same branch (e.g., one input is a line proof output), connects them directly (`old_root` of current

- matches `new_root` of previous).
 - If only one input, uses `GUTAHeaderLineProofGadget` to propagate upwards.
 - Combines the `stats` from input proofs using `GUTASTatsGadget.combine_with`.
 - Outputs a single `GlobalUserTreeAggregatorHeader` representing the combined state transition and stats.
- **What it Proves:** That given valid input GUTA proofs operating under the same whitelist and checkpoint context, the combined state transition and stats represented by the output header are correct.
- **Assumptions:**
 - Witness (input proofs, headers, NCA/sibling proofs) is correct initially.
- **How Assumptions are Discharged:** Verifies input proofs and their headers. Verifies the logic of combining state transitions (NCA/Line/Direct). Passes the common whitelist/checkpoint root assumptions upwards.
- **Contribution to Horizontal Scalability:** This is the core of parallel aggregation. Multiple instances of this circuit run concurrently across the DPN, merging proof branches in a tree structure (like MapReduce).
- **High-Level Functionality:** Securely and recursively combines verified state changes from multiple sources into larger, aggregated proofs.

8. `GUTANoChangeCircuit` (Hypothetical)

- **Purpose:** To handle cases where no user state changed but the checkpoint advanced.
- **Core Logic:** Uses `GUTANoChangeGadget`.
- **What it Proves:** That given a new `checkpoint_leaf` verified to be in the `checkpoint_tree_proof`, the `GUSR` tree root remains unchanged, and stats are zero. Outputs a GUTA header reflecting this.
- **Assumptions:** Witness (checkpoint proof, leaf) is correct initially. Input `guta_circuit_whitelist` is correct.
- **How Assumptions are Discharged:** Verifies checkpoint proof. Outputs a standard GUTA header passing assumptions upward.
- **Contribution to Horizontal Scalability:** Allows the aggregation process to stay synchronized with the checkpoint tree even during periods of inactivity for certain state trees.

- **High-Level Functionality:** Advances the aggregated checkpoint state reference.

Phase 3: Final Block Proof

9. Checkpoint Tree "Block" Circuit (Top-Level Aggregation)

- **Purpose:** The final aggregation circuit that combines proofs from the roots of all major state trees (like `GUSR` via the top-level GUTA proof, `GCON`, etc.) for the block.
- **Core Logic:**
 - Verifies the top-level GUTA proof (and proofs for other top-level trees if applicable).
 - Takes the *previous block's* finalized `CHKP` root as a public input.
 - Constructs the new `CHKP` leaf based on the newly computed roots of `GUSR`, `GCON`, etc., and other block metadata.
 - Computes the new `CHKP` root.
 - The *only* external assumption verified here is that the input `previous_block_chkp_root` matches the actual finalized root of the last block.
- **What it Proves:** That the entire state transition for the block, represented by the change from the `previous_block_chkp_root` to the `new_chkp_root`, is valid, having recursively verified all constituent user transactions and aggregations according to protocol rules and circuit whitelists.
- **Assumptions:** The *only* remaining input assumption is the hash of the previous block's `CHKP` root.
- **How Assumptions are Discharged:** All assumptions from lower levels (circuit whitelists, internal state consistencies) have been verified recursively. The final link to the previous block state is checked against the public input.
- **Contribution to Horizontal Scalability:** Represents the culmination of the massively parallel aggregation process, producing a single, succinct proof for the entire block's validity.
- **High-Level Functionality:** Creates the final, verifiable proof of state transition for the entire block, linking it cryptographically to the previous

block. This proof can be efficiently verified by any node or light client.

Proving Jobs Architecture

Overview

This document describes the proving jobs architecture for both Realm and Coordinator processors, including the tree structure of different proof types and their public inputs layout.

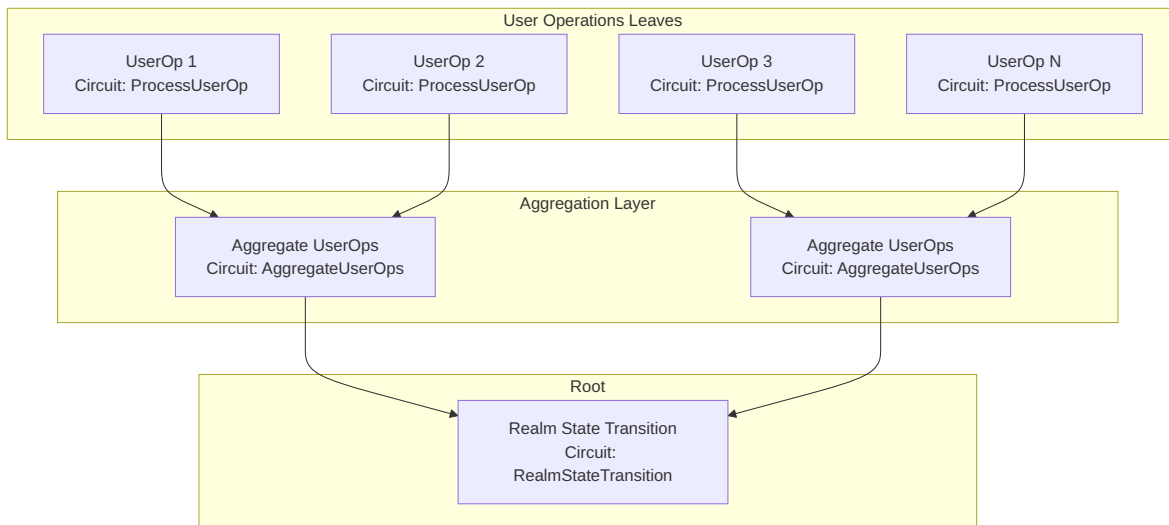
Public Inputs Layout Standard

All circuits follow a consistent public inputs layout:

- **[0..4]**: commitment
- **[4..8]**: worker_public_key
- **[8..11]**: pm_jobs_completed_stats (deploy_contracts_completed, register_users_completed, gutas_completed)
- **[11..15]**: circuit-specific hash (usually the hash of the main data structure)
- **[15..19]**: additional data (optional, circuit-specific)

Realm Proving Jobs

User Operations Tree



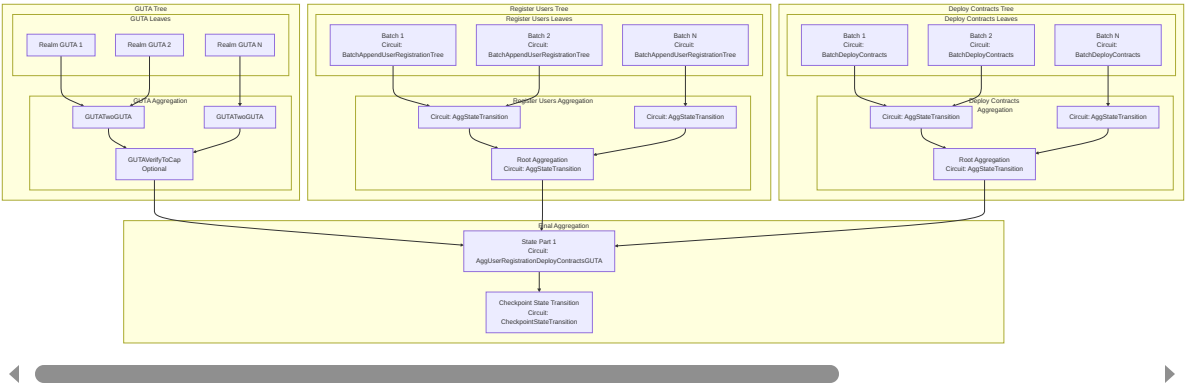
Realm Circuit Details

Circuit	Type	Public Inputs
ProcessUserOp	Leaf	[0..4]: commitment [4..8]: worker_public_key [8..11]: pm_jobs_completed_stats [11..15]: user_op_hash
AggregateUserOps	Intermediate	[0..4]: commitment [4..8]: worker_public_key [8..11]: pm_jobs_completed_stats [11..15]: agg_hash
RealmStateTransition	Root	[0..4]: commitment [4..8]: worker_public_key [8..11]: pm_jobs_completed_stats

Circuit	Type	Public Inputs
		[11..15]: state_transition_hash

Coordinator Proving Jobs

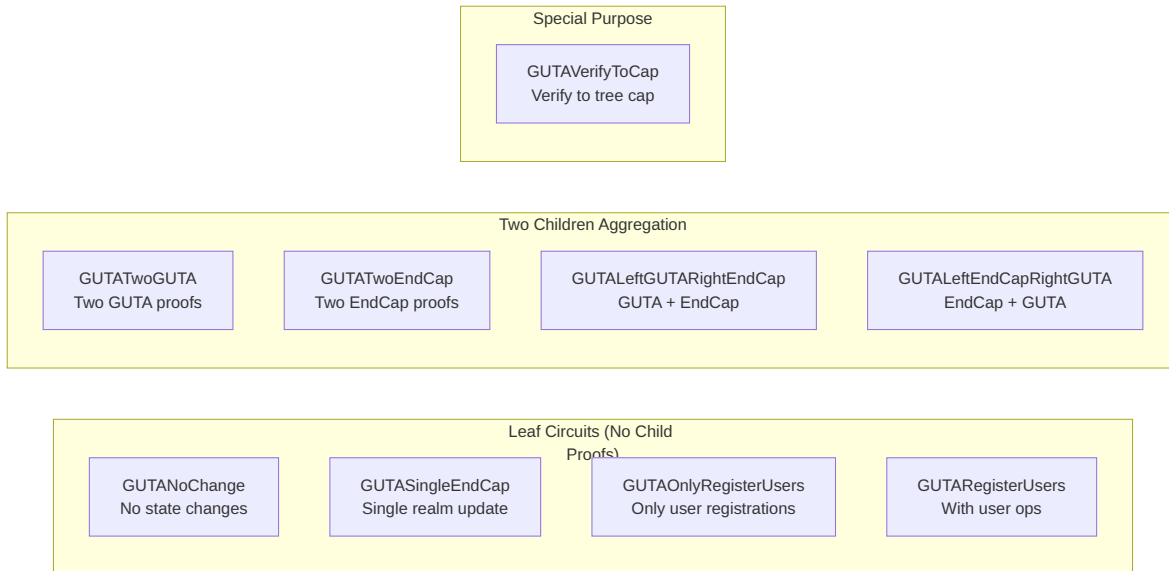
Three Main Trees + Final Aggregation



GUTA Circuit Variants

The GUTA (Global User Tree Aggregator) has multiple circuit variants to handle different scenarios:

GUTA Circuit Types and Usage



GUTA Circuit Details

Circuit	Purpose	Children	Permissions
Leaf Circuits			
GUTANoChange	No state changes in checkpoint	None	[0..4]: c [4..8]: w [8..11]: pm_job [11..15] guta_he
GUTASingleEndCap	Single realm had updates	None	[0..4]: c [4..8]: w [8..11]: pm_job [11..15] guta_he
GUTAOnlyRegisterUsers	Only user registrations, no ops	None	[0..4]: c [4..8]: w [8..11]:

Circuit	Purpose	Children	Primitives
			pm_job [11..15] guta_he
GUTAResisterUsers	User registrations with ops	1 GUTA	[0..4]: c [4..8]: w [8..11]: pm_job [11..15] guta_he
GUTATwoEndCap	Aggregate two EndCap proofs	None	[0..4]: c [4..8]: w [8..11]: pm_job [11..15] guta_he
Two Children Aggregation			
GUTATwoGUTA	Aggregate two GUTA proofs	2 GUTA	[0..4]: c [4..8]: w [8..11]: pm_job [11..15] guta_he
GUTALeftGUTARightEndCap	GUTA on left, EndCap on right	1 GUTA + 1 EndCap	[0..4]: c [4..8]: w [8..11]: pm_job [11..15] guta_he
GUTALeftEndCapRightGUTA	EndCap on left, GUTA on right	1 EndCap + 1 GUTA	[0..4]: c [4..8]: w [8..11]: pm_job [11..15] guta_he

Circuit	Purpose	Children	Public Inputs
Single Child Circuits			
GUTAVerifyToCap	Verify GUTA to tree cap	1 GUTA	[0..4]: commitment [4..8]: worker_public_key [8..11]: pm_jobs_completed_stats [11..15]: guta_header
GUTAVerifyGUTAResults	GUTA with user registrations	1 GUTA	[0..4]: commitment [4..8]: worker_public_key [8..11]: pm_jobs_completed_stats [11..15]: guta_header

State Part 1 (AggUserRegistrationDeployContractsGUTA)

This circuit aggregates the three main trees:

Inputs

- Register Users proof (from aggregation root)
- Deploy Contracts proof (from aggregation root)
- GUTA proof (from aggregation root or GUTAVerifyToCap)

Public Inputs Layout

- **[0..4]:** commitment
- **[4..8]:** worker_public_key
- **[8..11]:** pm_jobs_completed_stats (combined from all three child proofs)

- **[11..15]**: state_transition_hash
- **[15..19]**: register_users_root (directly from register_users_proof[0..4])
- **[19..23]**: deploy_contracts_root (directly from deploy_contracts_proof[0..4])
- **[23..27]**: gutas_root (directly from guta_proof[0..4])

PM Rewards Commitment

The PM (Prover/Miner) Rewards Commitment is calculated from these three roots:

```
PMRewardCommitment {
    register_users_root,
    deploy_contracts_root,
    gutas_root,
}
```

Checkpoint State Transition

The final circuit that creates the checkpoint proof:

Inputs

- State Part 1 proof
- Previous checkpoint proof
- Checkpoint tree merkle proof
- Various metadata (block time, random seed, etc.)

Public Inputs Layout (19 inputs total)

- **[0..4]**: commitment
- **[4..8]**: worker_public_key
- **[8..11]**: pm_jobs_completed_stats (from State Part 1 proof)

- [11..15]: old_checkpoint_tree_root
- [15..19]: new_checkpoint_tree_root

Job Dependencies and Task Graph



Syntax error in text
mermaid version 11.2.0

The dependency graph shows how PM stats flow through the system:

1. **Parallel Trees:** Each tree type accumulates its specific job counts
2. **State Part 1:** Combines PM stats from all three trees
3. **Checkpoint:** Preserves the combined PM stats for final reward calculation
4. **Block Completion:** Uses PM stats to calculate and distribute rewards

Commitment Calculation Rules

The commitment calculation follows a consistent pattern across all circuits:

1. Leaf Circuits (No Child Proofs)

```
commitment = worker_public_key
```

Examples: GUTANoChange, BatchDeployContracts,
AppendUserRegistrationTree

2. Single Child Circuits (One Child Proof)

```
commitment = hash(child.commitment, worker_public_key)
```

Examples: GUTAVerifyToCap, GUTAVerifyGUTAResisterUsers

3. Two Children Circuits (Two Child Proofs)

```
commitment = hash(hash(left.commitment, right.commitment),  
worker_public_key)
```

Examples: GUTATwoGUTA, AggStateTransition, GUTALeftGUTARightEndCap

Why This Design?

- **Leaf nodes:** The commitment IS the worker's identity, proving who did the work
- **Aggregation nodes:** The commitment combines children's work with the aggregator's identity
- **Merkle proof generation:** This forms a proper tree structure for generating proofs of participation

Coordinator Main Circuits

Register Users Tree Circuits

Circuit	Type	Children	Publ
AppendUserRegistrationTree	Leaf	None	[0..4]: com [4..8]: worl [8..11]: pm_jobs_c

Circuit	Type	Children	Public Inputs
			[11..15]: state_transition_hashes
AggStateTransition	Aggregation	2	[0..4]: commitments [4..8]: worker_public_keys [8..11]: pm_jobs_completion [11..15]: state_transition_hashes

Deploy Contracts Tree Circuits

Circuit	Type	Children	Public Inputs
◀ BatchDeployContracts	Leaf	None	[0..4]: commitments [4..8]: worker_public_keys [8..11]: pm_jobs_completion [11..15]: state_transition_hashes
AggStateTransition	Aggregation	2	[0..4]: commitments [4..8]: worker_public_keys [8..11]: pm_jobs_completion [11..15]: state_transition_hashes
▶			

Final Aggregation Circuits

Circuit	Type	Purpose
AggUserRegistrationDeployContractsGUTA	Aggregation	Combines all three trees into

Circuit	Type	Purpose
		State Part 1
CheckpointStateTransition	Root	Creates final checkpoint proof

PM Jobs Completed Stats Tracking

The PM (Proof Miner) jobs completed stats track the number of different types of jobs completed throughout the circuit hierarchy. These stats flow upward through the trees and are combined at aggregation points.

PM Stats Components

- **deploy_contracts_completed**: Number of deploy contract jobs completed in this subtree
- **register_users_completed**: Number of user registration jobs completed in this subtree
- **gutas_completed**: Number of GUTA jobs completed in this subtree

How Stats Flow Through the Hierarchy

Leaf Circuits

Leaf circuits initialize their PM stats based on the work they perform:

- **Deploy Contract leaves** (BatchDeployContracts): `pm_stats = (batch_size, 0, 0)`
- **Register Users leaves** (AppendUserRegistrationTree): `pm_stats = (0, batch_size, 0)`
- **GUTA leaves** (GUTANoChange, GUTASingleEndCap, etc.): `pm_stats = (0, 0, 0)` initially
- **Dummy circuits** (AggStateTransitionDummy): `pm_stats = (0, 0, 0)` (all zeros)

Aggregation Circuits

Aggregation circuits combine PM stats from their children:

```
// Two children aggregation (AggStateTransition, GUTATwoGUTA)
final_pm_stats = PMJobsCompletedStats {
    deploy_contracts_completed: left.pm_stats[0] +
right.pm_stats[0],
    register_users_completed: left.pm_stats[1] + right.pm_stats[1],
    gutas_completed: left.pm_stats[2] + right.pm_stats[2],
}
```

GUTA Circuits Special Handling

GUTA circuits add 1 to their gutas_completed count:

```
// Single child GUTA aggregation (GUTAVerifyToCap)
final_pm_stats = PMJobsCompletedStats {
    deploy_contracts_completed: child.pm_stats[0],
    register_users_completed: child.pm_stats[1],
    gutas_completed: child.pm_stats[2] + 1, // Add 1 GUTA completion
}
```

Final Aggregation

At the State Part 1 level (AggUserRegistrationDeployContractsGUTA), the PM stats from all three trees are combined:

```
final_pm_stats = register_users_proof.pm_stats +  
                  deploy_contracts_proof.pm_stats +  
                  guta_proof.pm_stats
```

This provides a complete count of all work performed in the current checkpoint.

Key Design Principles

1. **Consistent Public Inputs:** All circuits follow the same [commitment, worker_public_key, pm_jobs_completed_stats, data_hash] layout
2. **Tree Aggregation:** Each category (GUTA, Register Users, Deploy Contracts) forms its own tree
3. **Parallel Processing:** The three trees can be processed in parallel
4. **Commitment Chain:** Commitments flow up from leaves to root, enabling reward distribution
5. **Flexibility:** GUTA circuits handle various scenarios (no changes, single realm, multiple realms)
6. **Worker Tracking:** Every circuit includes the worker's public key who computed that proof
7. **PM Stats Tracking:** Job completion counts flow upward through the tree hierarchy for reward calculation

Introduction

This is the documentation for [Psy Smart Contract Language], a new programming language designed for ["safe and efficient smart contract development"]. This book serves as a comprehensive guide for developers interested in learning the language and building applications with it.

[Psy Smart Contract Language] is in active development and a work in progress. If you have feedback or suggestions, feel free to contribute via the [GitHub repository](#).

This book covers the core features of [Psy Smart Contract Language], including its syntax, module system, structs, traits, and storage capabilities.

Before We Begin

[Psy Smart Contract Language] requires a development environment to write and run programs. This chapter covers the prerequisites: setting up your IDE, installing the compiler, and understanding the basic tools.

If you already have the compiler installed (via `git` or another method), you can skip to the next chapter.

Installing the Compiler

Download the latest compiler binary from [<https://github.com/PsyProtocol/psy-v1>]. Binaries are available for macOS, Linux, and Windows.

Using a Package Manager

- **Homebrew (macOS):** `brew install [dargo]`
- **Chocolatey (Windows):** `choco install [dargo]`
- **Cargo (Rust-based, if applicable):** `cargo install --git https://github.com/PsyProtocol/psy-v1 dargo`

Setting up environment

```
set -gx DARGO_STD_PATH "/path-to-psy-lang/psy-std/std.psy"
```

Setting Up Your IDE

Recommended IDEs:

- **VSCode:** Install the [Psy Smart Contract Language] extension for syntax highlighting.
- **IntelliJ IDEA:** Use the [Psy Smart Contract Language] plugin by [PsyProtocol].

Psy LSP Developer Tutorial

Psy is a custom language with a dedicated Language Server Protocol (LSP) service, providing basic features such as `hover`, `goto definition`, `find references`, and `formatting`.

This document introduces how to use the Psy language server `psy-lsp-server` for a better development experience in **VSCode**, **Neovim**, and **RustRover**.

Preparation

1. Clone repository:

```
git clone https://github.com/PsyProtocol/psy-v1.git
cd psy-lang
```

2. Compile `psy-lsp-server` :

```
cd psy-lsp-server
cargo build --release
```

 **Note:** Regardless of which IDE you are using, the `psy-lsp-server` binary is required for the language features to work properly. Please make sure you have built it and **remember its path**.

VSCode Usage Tutorial

Developer debugging mode (recommended for developers)

1. Start VSCode:

```
cd psy-lsp-server/psy-lsp-vscode
code .
```

2. Press F5 to enter plugin debugging mode. VSCode will start a new VSCode window and load the local plugin.
3. In the new window, open a Psy project containing Dargo.toml to enable plugin features, such as:
 - Mouse hover → Show type information
 - Right click → Goto Definition / Find References / Format

💡 Note: In the file `psy-lsp-vscode/src/extension.ts`, the path to the `psy-lsp-server` binary is currently hardcoded:

```
const serverExecutable = path.join(
  // Warning: this path is hardcoded and may not be portable
  // across systems.
  context.extensionPath, '..', '..', 'target', 'release', 'psy-
  lsp-server'
);
```

If you need to change the path (for example, to use a different build directory or binary location), please modify this line accordingly and then rebuild the extension by running:

```
npm run build
```

Neovim usage tutorial (based on Lazy.nvim)

This guide demonstrates how to set up the **Psy language server** (`psy-lsp-server`) in Neovim using the **lazy.nvim** plugin manager.

⚠ **Note:** The `psy-lsp-server` binary must be compiled first and accessible in your system. Ensure the path to the binary is correct.

1 Plugin Installation

In your `init.lua`, make sure to install the necessary plugins:

```
require("lazy").setup({
  { "neovim/nvim-lspconfig" },          -- LSP support
  { "nvim-treesitter/nvim-treesitter", build = ":TSUpdate" }, --
syntax highlighting
  { "hrsh7th/nvim-cmp" },              -- completion
  { "hrsh7th/cmp-nvim-lsp" },          -- LSP completion source
})
```

2 Psy LSP Setup via Lazy

```
local lspconfig = require("lspconfig")
local configs = require("lspconfig.configs")

-- Register custom LSP
if not configs.psy_lsp then
  configs.psy_lsp = {
    default_config = {
      cmd = { "/full path/to/psy-lsp-server" }, -- ⚠ Note to fill in
the full path
      filetypes = { "psy" },
      root_dir = lspconfig.util.root_pattern("Dargo.toml"),
      settings = {},
    },
  },
end

lspconfig.psy_lsp.setup({})

-- Let Neovim recognize the `.psy` file type
vim.filetype.add({
  extension = {
    psy = "psy",
  },
})
```

Instead of manually configuring LSP in `init.lua`, you can configure it via Lazy plugin opts. Create a file like `~/.config/nvim/lua/plugins/lsp.lua` and write:

```

return {
  "neovim/nvim-lspconfig",
  opts = function(_, opts)
    -- Tell Neovim `.psy` files are of type `psy`
    vim.filetype.add({ extension = { psy = "psy" } })

    -- Reuse Rust Tree-sitter highlighting for `.psy` files
    vim.treesitter.language.register("rust", "psy")

    -- Load LSP config
    local lspconfig = require("lspconfig")
    local configs = require("lspconfig.configs")

    -- Register psy_lsp_server only once
    if not configs.psy_lsp_server then
      configs.psy_lsp_server = {
        default_config = {
          cmd = { "/full/path/to/psy-lsp-server" }, --  Replace
with your built binary
          filetypes = { "psy" },
          root_dir = lspconfig.util.root_pattern("Dargo.toml",
".psy"),
          settings = {},
        },
      }
    end

    -- Attach server config to Lazy's LSP handler
    opts.servers.psy_lsp_server = {}
    return opts
  end,
}

```

✓ Once you've saved this config, reopen Neovim and run `:Lazy sync` to apply it.

✓ Additional Notes

- `vim.filetype.add(...)` tells Neovim that `.psy` files should be handled as the `psy` filetype.
- `vim.treesitter.language.register("rust", "psy")` means `.psy` files will reuse Rust's highlighting engine via Tree-sitter.
- Make sure the `psy-lsp-server` binary is either in your `PATH` or referenced using the full path.

3 Navigation & Reference Lookup in Neovim

Once your psy-lsp-server is properly registered and running in Neovim, you can use the following built-in LSP keybindings to navigate your Psy code efficiently.

```
-- Place these in your init.lua or keymap config file if not already
present
vim.keymap.set("n", "gd", vim.lsp.buf.definition, { noremap = true,
desc = "Go to Definition" })
vim.keymap.set("n", "gr", vim.lsp.buf.references, { noremap = true,
desc = "Find References" })
vim.keymap.set("n", "K", vim.lsp.buf.hover, { noremap = true, desc =
"Hover Documentation" })
vim.keymap.set("n", "<leader>rn", vim.lsp.buf.rename, { noremap =
true, desc = "Rename Symbol" })
vim.keymap.set("n", "<leader>f", vim.lsp.buf.format, { noremap =
true, desc = "Format Document" })
```

 Common shortcut keys description

Shortcuts	Functional description
<code>gd</code>	Jump to definition
<code>gr</code>	Find all references
<code>K</code>	Hover type information
<code><leader>rn</code>	Rename symbol
<code><leader>f</code>	Code formatting
<code><C-o></code>	Return to previous position (built-in)

RustRover usage tutorial (based on LSP4IJ plugin)

This tutorial assumes that you have already completed the configuration of Rust in RustRover.

We will use the LSP plugin `LSP Support (lsp4ij)` provided by RedHat to enable custom LSP support for `.psy` files, realizing functions such as jump,

hover, reference lookup, code formatting, etc.

Step 1: Install the LSP4IJ plugin

1. Open RustRover and click `RustRover > Settings` in the upper left corner (or use the shortcut `Cmd + ,`).
 2. Go to `Plugins > Marketplace`.
 3. Search for **lsp4ij**.
 4. Click Install and restart the IDE after the installation is complete.
-

Step 2: Create a Psy file type

1. Open `Settings → Editor → File Types`
2. Click **"+" Add** at the top to create a new File Type
3. Name: `Psy`
4. Description: `Psy language`
5. Configure highlighting (optional) fill in the following fields:

Field name	Suggested value
Line Comment	<code>//</code>
Block Comment Start	<code>/*</code>
Block Comment End	<code>*/</code>
Hex Prefix	<code>0x</code> (optional)
Number Postfixes	<code>u32</code> (optional)
 Support Paired Braces	✓ Check
 Support Paired Brackets	✓ Check
 Support Paired Parentheses	✓ Check
 Support String Escapes	✓ Check

Then click Save.

Step 3: Configure the Language Server of Psy language

Open `Settings > Languages & Frameworks > Language Servers`, we need to add a new configuration, click the plus sign **+**, there are three tabs in the new interface: **Server**, **Mapping** and **Configuration**.

Server tab

Used to register a new LSP service.

- **Name:** Fill in `Psy language server`.
 - **Environment Variables:** Leave it blank (optional, if you need to set `DARGO_STD_PATH`, you can fill it in here).
 - **Command:** Fill in the path of your compiled LSP executable file, for example: `/Users/UserName/bin/psy-lsp-server`
-

Mapping tab

Used to map file types to language services.

- **Language:** Leave it blank.
- **FileType:** Click **+**, select `Psy` on the left, and enter `psy` on the right
 - Note: If there is no `Psy` option, please go back to step 2 and add the file type again.
- **Filename Patterns:** Click **+**, enter `*.psy` on the left, and enter `psy` on the right

 Note:

- The left side is the file type inside the IDE .
 - The right side is the Language ID of the LSP Server, which needs to match your `textDocument.languageId`
-

▶ Configuration (optional)

You can set:

- **Server Trace Level:** Set to `Verbose` to view more debugging information
 - **Client Trace Level:** Set to `Verbose` or `Info`
 - To view LSP communication logs, it is recommended to open the `LSP Console` view
-

✓ Step 4: Verify if it works

1. Open a Psy project with `Dargo.toml`
2. Open the `.psy` file
3. You can try:

- `Hover` to view the type
- `Go to Definition`
- `Find References`
- `Format Document`

You can check the debug log in `View > Tool Windows > LSP Console` to confirm whether LSP is started correctly.

Setting up Shell Completions

Shell completions allow you to use the Tab key to autocomplete commands, options, and arguments when working with the Psy CLI tools. This makes development faster and more convenient.

This guide will help you set up shell completions for the Dargo CLI.

Zsh Completions

You can add the completions script to your Zsh completions directory:

```
# Create the completions directory if it doesn't exist
mkdir -p ~/.zsh/completions

# Generate and save the completion script
dargo complete zsh > ~/.zsh/completions/_dargo

# Add the directory to your fpath in ~/.zshrc if you haven't already
echo 'fpath=(~/.zsh/completions $fpath)' >> ~/.zshrc
echo 'autoload -U compinit && compinit' >> ~/.zshrc

# Reload your shell
source ~/.zshrc
```

Bash Completions

You can add the completions script to your Bash environment in a few ways:

If you have bash-completion installed:

```
# Generate and save the completion script to the bash-completion
directory
dargo complete bash > /usr/local/etc/bash_completion.d/dargo
```

Without bash-completion, you can add it to your profile:

```
# Create a completions directory if you don't have one
mkdir -p ~/.bash_completions

# Generate and save the completion script
dargo complete bash > ~/.bash_completions/dargo.bash

# Add the following line to your ~/.bash_profile or ~/.bashrc
echo 'source ~/.bash_completions/dargo.bash' >> ~/.bashrc

# Reload your shell
source ~/.bashrc
```

Fish Completions

For Fish shell users, you can easily set up completions:

```
# Generate and save the completion script to the Fish completions
directory
dargo complete fish > ~/.config/fish/completions/dargo.fish
```

No additional steps are needed as Fish automatically loads completions from this directory.

Verifying Completions

To verify that completions are working correctly, type `dargo` followed by a space and press Tab. You should see a list of available commands.

Try:

```
dargo <TAB>
```

You should see a list of commands like `new`, `build`, `compile`, etc.

Troubleshooting

If completions aren't working:

1. Make sure you've restarted your terminal or sourced your `.zshrc` file
2. Check that the `dargo complete zsh` command works and produces output
3. Verify that the path to the completion script is correct
4. Ensure that Zsh completions are enabled in your shell configuration

Hello, World!

This chapter walks you through creating your first [Psy Smart Contract Language] program.

Creating a New Program

Create a new project:

```
dargo new hello_world
```

```
fn main(x: Felt, y: Felt) {  
    assert(x == y, "x != y");  
}  
  
#[test]  
fn test_main() {  
    main(1, 1);  
  
    // Uncomment to make test fail  
    // main(1, 2);  
}
```

Compiling

run the program using the compiler:

```
dargo compile
```

You will see a target directory. Open the `hello_world.json` inside and you'll be able to see the target code for psy. The psy smart contract language generates this code for each function in the contract and deploys it to the blockchain.

```
[
  {
    "name": "main",
    "method_id": 680917438,
    "circuit_inputs": [
      0,
      1
    ],
    "circuit_outputs": [],
    "state_commands": [],
    "state_command_resolution_indices": [],
    "assertions": [
      {
        "left": 4294967296,
        "right": 4294967297,
        "message": "x != y"
      }
    ],
    "definitions": [
      {
        "data_type": 0,
        "index": 0,
        "op_type": 0,
        "inputs": [
          0
        ]
      },
      {
        "data_type": 0,
        "index": 1,
        "op_type": 0,
        "inputs": [
          1
        ]
      },
      {
        "data_type": 1,
        "index": 0,
        "op_type": 13,
        "inputs": [
          0,
          1
        ]
      },
      {
        "data_type": 1,
        "index": 1,

```

```
        "op_type": 2,  
        "inputs": [  
            1  
        ]  
    }  
]  
}
```

Running

Basic Syntax

[Psy Smart Contract Language] uses a syntax inspired by [Rust]. Here are the basics:

Variables

Variables are declared with `let` and can be mutable with `mut`:

```
let a: Felt = 1;  
let mut b: Felt = 2;  
b += 1;
```

Comments

Use `//` for single-line comments: Use `/* */` for single-line comments:

```
// This is a comment  
let a: Felt = 1;
```

Types

- `Felt`: A goldilocks field type.
- `bool`: Boolean type.
- `u32`: 32-bit unsigned integer.
- `Array`
- `Tuple`
- `Struct`

Modules and Visibility

Modules organize code and control visibility.

Defining a Module

```
mod math {  
    pub fn max(a: Felt, b: Felt) -> Felt {  
        return a * ((a > b) as Felt) + b * ((a <= b) as Felt);  
    }  
}
```

Using Modules

```
use math::*;  
fn main() -> Felt {  
    max(1, 2)  
}
```

Visibility

`pub` : Makes an item public.

No `modifier` : Private to the module.

Structs and Implementations

Structs define custom data types, and `impl` blocks add methods.

Defining a Struct

```
struct Person {  
    pub age: Felt,  
    male: bool,  
}
```

Implementing Methods

```
impl Person {  
    pub fn get_age(self: Person) -> Felt {  
        return self.age;  
    }  
}
```

Usage

```
fn main() -> Felt {  
    let p: Person = new Person { age: 20, male: true };  
    p.get_age()  
}
```

Traits and Generics

Traits define shared behavior, and generics allow type flexibility.

Defining a Trait

```
pub trait Value {  
    pub fn value() -> Felt;  
}
```

Implementing a Trait

```
struct Two {}  
impl Value for Two {  
    pub fn value() -> Felt {  
        2  
    }  
}  
  
fn get_value<T: Value>(value: T) -> Felt {  
    T::value()  
}
```

Storage and Contracts

[Psy Smart Contract Language] supports storage for persistent state, useful for blockchain applications.

Defining a Contract

```
#[storage]
struct Contract {
}

impl Contract {
    pub fn simple_mint(amount: Felt) -> Felt {
        let self_user_leaf: Hash = get_state_hash_at(get_user_id());
        let current_balance: Felt = self_user_leaf[0];
        let new_balance: Felt = current_balance + amount;
        cset_state_hash_at(get_user_id(), [new_balance,
self_user_leaf[1], self_user_leaf[2], self_user_leaf[3]]);
        new_balance
    }
}
```

Explanation

`#[storage]` : Marks a struct for persistent storage. `get_state_hash_at` :
Retrieves state. `cset_state_hash_at` : Updates state.

Control Flow

If Statements

```
fn min(a: Felt, b: Felt) -> Felt {  
    if a < b {  
        a  
    } else {  
        b  
    }  
}
```

While Loops

```
fn main() -> Felt {  
    let mut res: Felt = 0;  
    let mut i: Felt = 0;  
    while i < 10 {  
        res += i;  
        i += 1;  
    }  
    res  
}
```

Match Expressions

```
fn match_test_case(input: Felt) -> Felt {  
    match input {  
        0 => 10,  
        1 => 20,  
        _ => 30,  
    }  
}
```

Functions and Closures

Functions

```
fn add(a: Felt, b: Felt) -> Felt {  
    a + b  
}
```

Closures

```
fn main() -> Felt {  
    let max = |a: Felt, b: Felt| -> Felt {  
        a * ((a > b) as Felt) + b * ((a <= b) as Felt)  
    };  
    max(1, 2)  
}
```

Arrays and Tuples

Arrays

```
fn main() -> Felt {  
    let arr: [Felt; 2] = [1, 2];  
    arr[0]  
}
```

Tuples

```
fn main() -> Felt {  
    let t: (Felt, Felt) = (1, 2);  
    t.0 + t.1  
}
```


Testing

Tests are written within modules using the `#[test]` attribute.

```
mod math {
    pub fn min(a: Felt, b: Felt) -> Felt {
        a * ((a < b) as Felt) + b * ((a >= b) as Felt)
    }
    mod math_tests {
        use super::*;
        #[test]
        fn test_min() {
            assert(min(2, 3) == 2, "min(2, 3) == 2");
        }
    }
}
```

run tests with `[dargo] test`.

Basic Dargo Commands

Dargo is psy-lang's package manager and build system that handles project creation, dependency management, and development workflows.

Common Commands

dargo new

Creates a new psy-lang project.

```
dargo new my_project
```

dargo init

Initializes a psy-lang project in an current directory.

```
dargo init
```

dargo compile

Compiles your project to zero-knowledge proof (ZKP) circuit.

```
dargo compile --contract-name=<contract_name> \  
  --method-names <method_name_1> <method_name_2> ...
```

dargo execute

Compiles your project to ZKP circuit and verify it

```
dargo execute --contract-name=<contract_name> \  
  --method-names <method_name_1> <method_name_2> ... \  
  --parameters <param_1> <param_2> ...
```

dargo fmt

Formats your psy-lang code.

```
dargo fmt <file_name>
```

User Cli Interface

This documentation organizes the three core data query interfaces implemented by `RpcProvider` (`QTreeDataStoreReaderSync<F>`, `QMetadataStoreReaderSync<F>`, and `PsyComboDataStoreReaderSync<F>`). It includes interface functions, method details, parameter descriptions, and return value types, focusing on core data query capabilities for blockchain scenarios such as users, contracts, and checkpoints.

Basic Information

- **Core Dependent Types:**
 - `F = GoldilocksField`: A prime field type based on Plonky2, used for blockchain data verification and hash calculation.
 - `QHashOut<F>`: Hash output result type, storing raw data after hash calculation.
 - `MerkleProofCore<QHashOut<F>>`: Core Merkle proof type, containing information such as the root, value, and sibling nodes required for proof.

Core Structures

`RpcProvider`

The foundational component for RPC communication, responsible for interacting with **Realm nodes** (user-specific data) and **Coordinator nodes** (global public data, e.g., contracts, checkpoints). It supports cross-environment use (non-WASM like servers, WASM like browser extensions).

Structure Definition

```
#[derive(Debug, Clone)]
pub struct RpcProvider {
    pub client: Arc<Client>,           // HTTP client (for RPC
requests)
    pub realm_configs: HashMap<u64, Vec<String>>, // Realm node
config: {Realm ID → List of RPC URLs}
    pub coordinator_configs: HashMap<u64, Vec<String>>, //
Coordinator node config: {Coordinator ID → List of RPC URLs}
    pub users_per_realm: u64,           // Number of users
assigned to each Realm (for user-to-Realm routing)
    pub current_user_id: u64,           // ID of the currently
active user (for default data requests)
}
```

Key Methods

`RpcProvider` provides methods for node routing, user/contract/data queries, and transaction submission. Core methods are categorized below:

Category	Method Name	Parameter
Node Routing	<code>get_realm_id</code>	<code>user_id: u64</code>
	<code>get_realm_url</code>	<code>user_id: u64</code>
	<code>get_coordinator_url</code>	None

Category	Method Name	Parameter
User Operations	register_user	req: QRegisterUserRPCReq
	get_user_id	public_key: QHash0
Contract Operations	deploy_contract	req: QDeployContractRPCR
Data Queries	get_realm_latest_block_state	None
	get_claim_amount	checkpoint_id: u64 u64, claim_user_id:
	check_tx_is_confirmed	checkpoint_id: u64 u64, tx_hash: QHashOut<Goldilocks
Batch Proof Query	get_job_proofs	job_infos: Vec<Job:

Category	Method Name	Parameter

Auxiliary Structures

RpcConfig & **NetworkConfig**

Configuration structures for node networks and proxy services, loaded from external config files:

```

#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct RpcConfig {
    pub users_per_realm: u64,           // Number of users per
    Realm
    pub global_user_tree_height: u8,    // Height of the global
    user Merkle tree
    pub realm_user_tree_height: u8,    // Height of the Realm-
    specific user Merkle tree
    pub realm_configs: Vec<RealmRpcConfig>, // List of Realm node
    configs
    pub coordinator_configs: Vec<CoordinatorRpcConfig>, // List of
    Coordinator node configs
    pub prove_proxy_url: Vec<String>,   // List of proof proxy
    service URLs
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct NetworkConfig { // Extended config for blockchain
    network
    pub users_per_realm: u64,
    pub global_user_tree_height: u8,
    pub realm_user_tree_height: u8,
    pub realm_configs: Vec<RealmConfig>, // Same as
    RealmRpcConfig
    pub coordinator_configs: Vec<CoordinatorConfig>, // Same as
    CoordinatorRpcConfig
    pub prover_url: Option<String>,      // Optional prover
    service URL
    pub prove_proxy_url: Vec<String>,
    pub native_currency: String,        // Native currency
    symbol (e.g., "PSY")
}

```

Merkle Tree Data Query

This interface focuses on querying roots, leaf hashes, and Merkle proofs of various Merkle trees in the blockchain, covering core scenarios such as users, contracts, checkpoints, deposits, and withdrawals.

1. User-Related Merkle Tree Queries

(1) User Contract State Tree Queries (User-Contract Internal State)

Method Name	Parameter Description	
<code>get_user_contract_state_tree_root</code>	- <code>checkpoint_id</code>: u64 : Unique identifier for the checkpoint - <code>user_id</code>: u64 : Unique identifier for the user - <code>contract_id</code>: u32 : Unique identifier for the contract	Qt
<code>get_user_contract_state_tree_leaf_hash</code>	- Same as above - <code>height</code>: u8 : Height of the state tree - <code>leaf_id</code>: u64 : Unique identifier for the leaf node	Qt

Method Name	Parameter Description	
<code>get_user_contract_state_tree_merkle_proof</code>	Same parameters as above	Me

(2) User Contract Tree Queries (User-Contract Association)

Method Name	Parameter Description	
<code>get_user_contract_tree_root</code>	- <code>checkpoint_id:</code> u64 : Unique identifier for the checkpoint - <code>user_id:</code> u64 : Unique identifier for the user	QHashOut
<code>get_user_contract_tree_leaf_hash</code>	- Same as above - <code>contract_id:</code> u32 : Unique identifier for the contract	QHashOut
<code>get_user_contract_tree_merkle_proof</code>	Same parameters as above	MerklePr

Method Name	Parameter Description	

(3) User Registration Tree Queries (Global User Registration State)

Method Name	Parameter Description	
<code>get_user_registration_tree_root</code>	- <code>checkpoint_id:</code> u64 : Unique identifier for the checkpoint	QHas
<code>get_user_registration_tree_leaf_hash</code>	- Same as above - <code>leaf_index:</code> u64 : Leaf index	QHas
<code>get_user_registration_tree_merkle_proof</code>	Same parameters as above	Merl

(4) User Tree Queries (Global User State)

Method Name	Parameter Description	Return Type
<code>get_user_tree_root</code>	- - <code>checkpoint_id:</code> <u>u64</u> : Unique identifier for the checkpoint	<code>QHashOut<F></code>
<code>get_user_tree_leaf_hash</code>	- Same as above - <code>user_id:</code> <u>u64</u> : Unique identifier for the user	<code>QHashOut<F></code>
<code>get_user_tree_merkle_proof</code>	Same parameters as above	<code>MerkleProofContext</code>
<code>get_user_sub_tree_merkle_proof</code>	- - <code>checkpoint_id:</code> <u>u64</u> : Unique identifier for the checkpoint - <code>root_level:</code> <u>u8</u> : Root level - <code>leaf_level:</code> <u>u8</u> : Leaf level - <code>leaf_index:</code> <u>u64</u> : Leaf index	<code>MerkleProofContext</code>

2. Contract-Related Merkle Tree Queries

Method Name	Parameter Description	
<code>get_contract_function_tree_root</code>	- <code>checkpoint_id: u64</code> : Unique identifier for the checkpoint - <code>contract_id: u32</code> : Unique identifier for the contract	QHas
<code>get_contract_function_tree_leaf_hash</code>	- Same as above - <code>function_id: u32</code> : Unique identifier for the function	QHas
<code>get_contract_function_tree_merkle_proof</code>	Same parameters as above	Merl
<code>get_contract_tree_root</code>	- <code>checkpoint_id: u64</code> : Unique identifier for the checkpoint	QHas
<code>get_contract_tree_leaf_hash</code>	- Same as above - <code>contract_id:</code>	QHas

Method Name	Parameter Description	
	u32 : Unique identifier for the contract	
get_contract_tree_merkle_proof	Same parameters as above	Merk

3. Asset-Related Merkle Tree Queries

(1) Deposit Tree Queries

Method Name	Parameter Description	Return
get_deposit_tree_root	- checkpoint_id: u64 : Unique identifier for the checkpoint	QHashOut<F>
get_deposit_tree_leaf_hash	- Same as above - deposit_id: u32 : Unique identifier for the deposit	QHashOut<F>

Method Name	Parameter Description	Return
<code>get_deposit_tree_merkle_proof</code>	Same parameters as above	<code>MerkleProofCore</code>

(2) Withdrawal Tree Queries

Method Name	Parameter Description	Return
<code>get_withdrawal_tree_root</code>	- <code>checkpoint_id: u64</code> : Unique identifier for the checkpoint	<code>QHashOut<F></code>
<code>get_withdrawal_tree_leaf_hash</code>	- Same as above - <code>withdrawal_id: u32</code> : Unique identifier for the withdrawal	<code>QHashOut<F></code>
<code>get_withdrawal_tree_merkle_proof</code>	Same parameters as above	<code>MerkleProofCore</code>

4. Checkpoint-Related Merkle Tree Queries

Method Name	Parameter Description	
<code>get_latest_checkpoint_tree_root</code>	No parameters	QHashC
<code>get_checkpoint_tree_root</code>	- <code>checkpoint_id:</code> u64 : Unique identifier for the checkpoint	QHashC
<code>get_checkpoint_tree_leaf_hash</code>	- Same as above - <code>leaf_checkpoint_id:</code> u64 : Leaf checkpoint ID	QHashC
<code>get_checkpoint_tree_merkle_proof</code>	Same parameters as above	Merkle

Metadata Query

This interface focuses on querying core blockchain metadata, including complete leaf data for users, contracts, and checkpoints, as well as L2 block states.

1. User Metadata Queries

Method Name	Parameter Description	Return Value	Function Description
<code>get_user_leaf_data</code>	- <code>checkpoint_id:</code> u64 : Unique identifier for the checkpoint - <code>user_id:</code> u64 : Unique identifier for the user	<code>PsyUserLeaf<F></code>	Query contract leaf data for a user under specific checkpoint (includes user's hash,

2. Contract Metadata Queries

Method Name	Parameter Description	Return Value
<code>get_contract_leaf_data</code>	- <code>contract_id:</code> u64 : Unique identifier for the contract	<code>PsyContractLeaf<F></code>
<code>get_contract_code_definition</code>	- <code>contract_id:</code> u64 : Unique identifier for the contract	<code>ContractCodeDefinition</code>

Method Name	Parameter Description	Return Value

3. Checkpoint and L2 Block State Queries

Method Name	Parameter Description	Return Value
<code>get_checkpoint_leaf_data</code>	- <code>checkpoint_id:</code> <code>u64</code> : Unique identifier for the checkpoint	<code>PsyCheckpointLeaf<I</code>
<code>get_latest_block_state</code>	No parameters	<code>PsyBlockState</code>
<code>get_block_state</code>	- <code>checkpoint_id:</code> <code>u64</code> : Unique identifier for the checkpoint	<code>PsyBlockState</code>

Realm Edge RPC Documentation

This document provides comprehensive documentation for all Realm Edge RPC methods defined in the `RealmEdgeRpc` trait.

RPC Namespace: `psy`

Table of Contents

1. [User Management](#)
 2. [User End Cap Submission](#)
 3. [Checkpoint Data Operations](#)
 4. [L2 Block State Operations](#)
 5. [User Registration Tree Operations](#)
 6. [Checkpoint Tree Operations](#)
 7. [User Leaf Data Operations](#)
 8. [User Contract State Tree Operations](#)
 9. [User Contract Tree Operations](#)
 10. [User Tree Operations](#)
 11. [Batch Proof Generation](#)
 12. [GraphViz Export](#)
 13. [Data Structures](#)
-

User Management

1. `check_user_id_in_realm`

Check if a user ID belongs to this realm.

Method Name: psy_check_user_id_in_realm

Request Parameters:

```
{
  "user_id": 12345
}
```

Parameter	Type	Description
user_id	u64	The user ID to check

Response:

```
{
  "result": true
}
```

Field	Type	Description
result	bool	true if the user belongs to this realm, false otherwise

Example:

```
curl -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "psy_check_user_id_in_realm",
  "params": [12345],
  "id": 1
}'
```

User End Cap Submission

2. submit_user_end_cap

Submit a user end cap proof for processing.

Method Name: `psy_submit_user_end_cap`

Request Parameters:

```
{
  "user_ec_input": {
    "core": {
      "checkpoint_id": "123",
      "stats": { ... },
      "state_transition": { ... },
      "new_user_leaf": { ... }
    },
    "contract_state_updates": [ ... ]
  },
  "proof": { ... }
}
```

Parameter	Type	Description
<code>user_ec_input</code>	<code>SubmitUserEndCapNonProofInput<F></code>	End cap input data (see Data Structures)
<code>proof</code>	<code>ProofWithPublicInputs<F, C, D></code>	Zero-knowledge proof

Response:

```
{
  "result": "0x1234567890abcdef..."
}
```

Field	Type	Description
result	String	Transaction hash or job ID

Checkpoint Data Operations

3. get_checkpoint_leaf_data

Get checkpoint leaf data by checkpoint ID (u64 parameter).

Method Name: `psy_get_checkpoint_leaf_data`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [PsyCheckpointLeaf](#)

Example:

```
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_checkpoint_leaf_data",
    "params": [100],
    "id": 1
  }'
```

4. get_checkpoint_leaf_data_f

Get checkpoint leaf data by checkpoint ID (Field parameter).

Method Name: `psy_get_checkpoint_leaf_data_f`

Request Parameters:

```
{
  "checkpoint_id": "100"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	F (Field)	The checkpoint ID as a field element

Response: See [PsyCheckpointLeaf](#)

L2 Block State Operations

5. get_latest_block_state

Get the latest L2 block state.

Method Name: `psy_get_latest_block_state`

Request Parameters: None

Response: See [PsyBlockState](#)

Example:

```
curl -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "psy_get_latest_block_state",
  "params": [],
  "id": 1
}'
```

Response Example:

```
{
  "result": {
    "checkpoint_id": 100,
    "next_add_withdrawal_id": 50,
    "next_process_withdrawal_id": 45,
    "next_deposit_id": 200,
    "total_deposits_claimed_epoch": 180,
    "next_user_id": 1000,
    "end_balance": 5000000,
    "next_contract_id": 25
  }
}
```

6. get_block_state

Get L2 block state at a specific checkpoint (u64 parameter).

Method Name: `psy_get_block_state`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	<code>u64</code>	The checkpoint ID

Response: See [PsyBlockState](#)

7. get_block_state_f

Get L2 block state at a specific checkpoint (Field parameter).

Method Name: `psy_get_block_state_f`

Request Parameters:

```
{  
  "checkpoint_id": "100"  
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	F (Field)	The checkpoint ID as a field element

Response: See [PsyBlockState](#)

User Registration Tree Operations

8. get_user_registration_tree_root

Get the user registration tree root at a specific checkpoint.

Method Name: `psy_get_user_registration_tree_root`

Request Parameters:

```
{  
  "checkpoint_id": 100  
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [QHashOut](#)

Example Response:

```
{
  "result":
    "0x1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef"
}
```

Checkpoint Tree Operations

9. get_latest_checkpoint_tree_root

Get the latest checkpoint tree root.

Method Name: `psy_get_latest_checkpoint_tree_root`

Request Parameters: None

Response: See [QHashOut](#)

10. get_checkpoint_tree_root

Get checkpoint tree root at a specific checkpoint (u64 parameter).

Method Name: `psy_get_checkpoint_tree_root`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [QHashOut](#)

11. get_checkpoint_tree_root_f

Get checkpoint tree root at a specific checkpoint (Field parameter).

Method Name: psy_get_checkpoint_tree_root_f

Request Parameters:

```
{
  "checkpoint_id": "100"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element

Response: See [QHashOut](#)

12. get_checkpoint_tree_leaf_hash

Get a specific checkpoint tree leaf hash (u64 parameters).

Method Name: psy_get_checkpoint_tree_leaf_hash

Request Parameters:

```
{
  "checkpoint_id": 100,
  "leaf_checkpoint_id": 95
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
leaf_checkpoint_id	u64	The leaf checkpoint ID

Response: See [QHashOut](#)

13. get_checkpoint_tree_leaf_hash_f

Get a specific checkpoint tree leaf hash (Field parameters).

Method Name: psy_get_checkpoint_tree_leaf_hash_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "leaf_checkpoint_id": "95"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
leaf_checkpoint_id	F (Field)	The leaf checkpoint ID as a field element

Response: See [QHashOut](#)

14. get_checkpoint_tree_merkle_proof

Get Merkle proof for a checkpoint tree leaf (u64 parameters).

Method Name: `psy_get_checkpoint_tree_merkle_proof`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "leaf_checkpoint_id": 95
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	u64	The checkpoint ID
<code>leaf_checkpoint_id</code>	u64	The leaf checkpoint ID

Response: See [MerkleProofCore](#)

15. get_checkpoint_tree_merkle_proof_f

Get Merkle proof for a checkpoint tree leaf (Field parameters).

Method Name: `psy_get_checkpoint_tree_merkle_proof_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "leaf_checkpoint_id": "95"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	F (Field)	The checkpoint ID as a field element

Parameter	Type	Description
<code>leaf_checkpoint_id</code>	F (Field)	The leaf checkpoint ID as a field element

Response: See [MerkleProofCore](#)

16. get_checkpoint_global_state_roots

Get global state roots at a specific checkpoint.

Method Name: `psy_get_checkpoint_global_state_roots`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	u64	The checkpoint ID

Response: See [PsyCheckpointGlobalStateRoots](#)

Example Response:

```
{
  "result": {
    "contract_tree_root": "0x...",
    "deposit_tree_root": "0x...",
    "user_tree_root": "0x...",
    "withdrawal_tree_root": "0x...",
    "user_registration_tree_root": "0x..."
  }
}
```

User Leaf Data Operations

17. get_user_leaf_data

Get user leaf data at a specific checkpoint (u64 parameters).

Method Name: `psy_get_user_leaf_data`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	u64	The checkpoint ID
<code>user_id</code>	u64	The user ID

Response: See [PsyUserLeaf](#)

Example:

```
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_user_leaf_data",
    "params": [100, 12345],
    "id": 1
  }'
```

18. get_user_leaf_data_f

Get user leaf data at a specific checkpoint (Field parameters).

Method Name: psy_get_user_leaf_data_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "user_id": "12345"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
user_id	F (Field)	The user ID as a field element

Response: See [PsyUserLeaf](#)

User Contract State Tree Operations

19. get_user_contract_state_tree_root

Get user contract state tree root (u64 parameters).

Method Name: psy_get_user_contract_state_tree_root

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345,
  "contract_id": 5
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
user_id	u64	The user ID

Parameter	Type	Description
contract_id	u32	The contract ID

Response: See [QHashOut](#)

20. get_user_contract_state_tree_root_f

Get user contract state tree root (Field parameters).

Method Name: psy_get_user_contract_state_tree_root_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "user_id": "12345",
  "contract_id": "5"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
user_id	F (Field)	The user ID as a field element
contract_id	F (Field)	The contract ID as a field element

Response: See [QHashOut](#)

21. get_user_contract_state_tree_leaf_hash

Get user contract state tree leaf hash (u64 parameters).

Method Name: psy_get_user_contract_state_tree_leaf_hash

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345,
  "contract_id": 5,
  "height": 10,
  "leaf_id": 42
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
user_id	u64	The user ID
contract_id	u32	The contract ID
height	u8	The tree height
leaf_id	u64	The leaf ID

Response: See [QHashOut](#)

22. get_user_contract_state_tree_leaf_hash_f

Get user contract state tree leaf hash (Field parameters).

Method Name: psy_get_user_contract_state_tree_leaf_hash_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "user_id": "12345",
  "contract_id": "5",
  "height": 10,
  "leaf_id": "42"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
user_id	F (Field)	The user ID as a field element

Parameter	Type	Description
contract_id	F (Field)	The contract ID as a field element
height	u8	The tree height
leaf_id	F (Field)	The leaf ID as a field element

Response: See [QHashOut](#)

23. get_user_contract_state_tree_merkle_proof

Get Merkle proof for user contract state tree (u64 parameters).

Method Name: psy_get_user_contract_state_tree_merkle_proof

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345,
  "contract_id": 5,
  "height": 10,
  "leaf_id": 42
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
user_id	u64	The user ID
contract_id	u32	The contract ID
height	u8	The tree height
leaf_id	u64	The leaf ID

Response: See [MerkleProofCore](#)

24. get_user_contract_state_tree_merkle_proof_f

Get Merkle proof for user contract state tree (Field parameters).

Method Name: `psy_get_user_contract_state_tree_merkle_proof_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "user_id": "12345",
  "contract_id": "5",
  "height": 10,
  "leaf_id": "42"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
user_id	F (Field)	The user ID as a field element
contract_id	F (Field)	The contract ID as a field element
height	u8	The tree height
leaf_id	F (Field)	The leaf ID as a field element

Response: See [MerkleProofCore](#)

User Contract Tree Operations

25. get_user_contract_tree_root

Get user contract tree root (u64 parameters).

Method Name: `psy_get_user_contract_tree_root`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
user_id	u64	The user ID

Response: See [QHashOut](#)

26. get_user_contract_tree_root_f

Get user contract tree root (Field parameters).

Method Name: psy_get_user_contract_tree_root_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "user_id": "12345"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
user_id	F (Field)	The user ID as a field element

Response: See [QHashOut](#)

27. get_user_contract_tree_leaf_hash

Get user contract tree leaf hash (u64 parameters).

Method Name: psy_get_user_contract_tree_leaf_hash

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345,
  "contract_id": 5
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
user_id	u64	The user ID
contract_id	u32	The contract ID

Response: See [QHashOut](#)

28. get_user_contract_tree_leaf_hash_f

Get user contract tree leaf hash (Field parameters).

Method Name: psy_get_user_contract_tree_leaf_hash_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "user_id": "12345",
  "contract_id": "5"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
user_id	F (Field)	The user ID as a field element
contract_id	F (Field)	The contract ID as a field element

Response: See [QHashOut](#)

29. get_user_contract_tree_merkle_proof

Get Merkle proof for user contract tree (u64 parameters).

Method Name: `psy_get_user_contract_tree_merkle_proof`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345,
  "contract_id": 5
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	<code>u64</code>	The checkpoint ID
<code>user_id</code>	<code>u64</code>	The user ID
<code>contract_id</code>	<code>u32</code>	The contract ID

Response: See [MerkleProofCore](#)

30. get_user_contract_tree_merkle_proof_f

Get Merkle proof for user contract tree (Field parameters).

Method Name: `psy_get_user_contract_tree_merkle_proof_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "user_id": "12345",
  "contract_id": "5"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
user_id	F (Field)	The user ID as a field element
contract_id	F (Field)	The contract ID as a field element

Response: See [MerkleProofCore](#)

User Tree Operations

31. get_user_tree_root

Get user tree root at a specific checkpoint (u64 parameter).

Method Name: `psy_get_user_tree_root`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [QHashOut](#)

32. get_user_tree_root_f

Get user tree root at a specific checkpoint (Field parameter).

Method Name: `psy_get_user_tree_root_f`

Request Parameters:

```
{
  "checkpoint_id": "100"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	F (Field)	The checkpoint ID as a field element

Response: See [QHashOut](#)

33. get_user_tree_leaf_hash

Get user tree leaf hash (u64 parameters).

Method Name: `psy_get_user_tree_leaf_hash`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	u64	The checkpoint ID
<code>user_id</code>	u64	The user ID

Response: See [QHashOut](#)

34. get_user_tree_leaf_hash_f

Get user tree leaf hash (Field parameters).

Method Name: `psy_get_user_tree_leaf_hash_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "user_id": "12345"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	F (Field)	The checkpoint ID as a field element
<code>user_id</code>	F (Field)	The user ID as a field element

Response: See [QHashOut](#)

35. get_user_bottom_tree_merkle_proof

Get Merkle proof for user bottom tree (u64 parameters).

Method Name: `psy_get_user_bottom_tree_merkle_proof`

Request Parameters:

```
{
  "root_level": 5,
  "checkpoint_id": 100,
  "user_id": 12345
}
```

Parameter	Type	Description
<code>root_level</code>	u8	The root level of the tree
<code>checkpoint_id</code>	u64	The checkpoint ID

Parameter	Type	Description
user_id	u64	The user ID

Response: See [MerkleProofCore](#)

36. get_user_bottom_tree_merkle_proof_f

Get Merkle proof for user bottom tree (Field parameters).

Method Name: psy_get_user_bottom_tree_merkle_proof_f

Request Parameters:

```
{
  "root_level": 5,
  "checkpoint_id": "100",
  "user_id": "12345"
}
```

Parameter	Type	Description
root_level	u8	The root level of the tree
checkpoint_id	F (Field)	The checkpoint ID as a field element
user_id	F (Field)	The user ID as a field element

Response: See [MerkleProofCore](#)

37. get_user_sub_tree_merkle_proof

Get Merkle proof for user sub-tree (u64 parameters).

Method Name: psy_get_user_sub_tree_merkle_proof

Request Parameters:

```
{
  "checkpoint_id": 100,
  "root_level": 5,
  "leaf_level": 2,
  "leaf_index": 42
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
root_level	u8	The root level of the tree
leaf_level	u8	The leaf level of the tree
leaf_index	u64	The leaf index

Response: See [MerkleProofCore](#)

38. get_user_sub_tree_merkle_proof_f

Get Merkle proof for user sub-tree (Field parameters).

Method Name: psy_get_user_sub_tree_merkle_proof_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "root_level": 5,
  "leaf_level": 2,
  "leaf_index": "42"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
root_level	u8	The root level of the tree
leaf_level	u8	The leaf level of the tree
leaf_index	F (Field)	The leaf index as a field element

Response: See [MerkleProofCore](#)

39. get_user_tree_merkle_proof

Get Merkle proof for user tree (u64 parameters).

Method Name: `psy_get_user_tree_merkle_proof`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	u64	The checkpoint ID
<code>user_id</code>	u64	The user ID

Response: See [MerkleProofCore](#)

Example:

```
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_user_tree_merkle_proof",
    "params": [100, 12345],
    "id": 1
  }'
```

40. get_user_tree_merkle_proof_f

Get Merkle proof for user tree (Field parameters).

Method Name: `psy_get_user_tree_merkle_proof_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "user_id": "12345"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	F (Field)	The checkpoint ID as a field element
<code>user_id</code>	F (Field)	The user ID as a field element

Response: See [MerkleProofCore](#)

Note: This method internally converts field parameters to u64 and calls `get_user_tree_merkle_proof`.

Batch Proof Generation

41. `generate_batch_variable_height_reward_proofs`

Generate batch variable height reward Merkle proofs for multiple job IDs.

Method Name: `psy_generate_batch_variable_height_reward_proofs`

Request Parameters:

```

{
  "checkpoint_id": 100,
  "job_ids": [
    {
      "topic": "GenerateStandardProof",
      "goal_id": 100,
      "slot_id": 5,
      "circuit_type": "GUTATwoEndCap",
      "group_id": 1,
      "sub_group_id": 0,
      "task_index": 0,
      "data_type": "InputWitness",
      "data_index": 0
    }
  ]
}

```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
job_ids	Vec<QProvingJobDataID>	Array of proving job data IDs (see QProvingJobDataID)

Response:

```

{
  "result": [
    [
      {
        "top_siblings": [...],
        "sibling_branch": "0x...",
        "reward_leaf": "0x...",
        "proof_height": "5",
        "index": "42"
      },
      {
        "topic": "GenerateStandardProof",
        "goal_id": 100,
        ...
      }
    ]
  ]
}

```

Field	Type	Description
result	Vec<(VariableHeightRewardMerkleProof, QProvingJobDataID)>	Array of tuples containing proofs and job IDs

Example:

```
curl -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "psy_generate_batch_variable_height_reward_proofs",
  "params": [100, [...]],
  "id": 1
}'
```

GraphViz Export

42. get_graphviz

Get GraphViz representation of the Merkle tree at a specific checkpoint.

Method Name: psy_get_graphviz

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response:

```
{
  "result": "digraph G {\n  node1 -> node2;\n  ...\n}"
}
```

Field	Type	Description
result	String	GraphViz DOT format string

Example:

```
curl -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "psy_get_graphviz",
  "params": [100],
  "id": 1
}' | jq -r '.result' | dot -Tpng > tree.png
```

Data Structures

QHashOut

A hash output wrapper for Plonky2 field elements.

Structure:

```
pub struct QHashOut<F: Field>(pub HashOut<F>);
```

JSON Representation:

```
"0x1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef"
```

Description:

- Wraps a Plonky2 `HashOut<F>` containing 4 field elements
 - Serialized as a hexadecimal string (32 bytes)
 - Represents a 256-bit hash value
-

PsyBlockState

L2 block state information at a specific checkpoint.

Structure:

```
pub struct PsyBlockState {  
    pub checkpoint_id: u64,  
    pub next_add_withdrawal_id: u64,  
    pub next_process_withdrawal_id: u64,  
    pub next_deposit_id: u64,  
    pub total_deposits_claimed_epoch: u64,  
    pub next_user_id: u64,  
    pub end_balance: u64,  
    pub next_contract_id: u32,  
}
```

Fields:

Field	Type	Description
checkpoint_id	u64	The checkpoint identifier
next_add_withdrawal_id	u64	Next withdrawal ID to be added
next_process_withdrawal_id	u64	Next withdrawal ID to be processed
next_deposit_id	u64	Next deposit ID
total_deposits_claimed_epoch	u64	Total deposits claimed in current epoch
next_user_id	u64	Next user ID to be assigned
end_balance	u64	Ending balance at this checkpoint

Field	Type	Description
<code>next_contract_id</code>	<code>u32</code>	Next contract ID to be assigned

Example:

```
{
  "checkpoint_id": 100,
  "next_add_withdrawal_id": 50,
  "next_process_withdrawal_id": 45,
  "next_deposit_id": 200,
  "total_deposits_claimed_epoch": 180,
  "next_user_id": 1000,
  "end_balance": 5000000,
  "next_contract_id": 25
}
```

PsyCheckpointLeaf

Checkpoint leaf data containing global chain root and statistics.

Structure:

```
pub struct PsyCheckpointLeaf<F: RichField> {
  pub global_chain_root: QHashOut<F>,
  pub stats: PsyCheckpointLeafStats<F>,
}
```

Fields:

Field	Type	Description
<code>global_chain_root</code>	<code>QHashOut<F></code>	Global chain state root hash
<code>stats</code>	<code>PsyCheckpointLeafStats<F></code>	Checkpoint statistics

PsyCheckpointLeafStats Structure:

```
pub struct PsyCheckpointLeafStats<F: RichField> {
    pub fees_collected: F,
    pub user_ops_processed: F,
    pub total_transactions: F,
    pub slots_modified: F,
    pub pm_jobs_completed: PMJobsCompletedStats<F>,
    pub block_time: F,
    pub random_seed: QHashOut<F>,
    pub pm_rewards_commitment: PMRewardCommitment<F>,
    pub da_challenges_claimed: [F; DA_CHALLENGE_WINDOW],
}
```

Example:

```
{
  "global_chain_root": "0x...",
  "stats": {
    "fees_collected": "1000",
    "user_ops_processed": "50",
    "total_transactions": "75",
    "slots_modified": "30",
    "pm_jobs_completed": {...},
    "block_time": "1234567890",
    "random_seed": "0x...",
    "pm_rewards_commitment": {...},
    "da_challenges_claimed": [...]
  }
}
```

PsyCheckpointGlobalStateRoots

Global state roots at a specific checkpoint.

Structure:

```
pub struct PsyCheckpointGlobalStateRoots<F: RichField> {
    pub contract_tree_root: QHashOut<F>,
    pub deposit_tree_root: QHashOut<F>,
    pub user_tree_root: QHashOut<F>,
    pub withdrawal_tree_root: QHashOut<F>,
    pub user_registration_tree_root: QHashOut<F>,
}
```

Fields:

Field	Type	Description
contract_tree_root	QHashOut<F>	Root of the contract tree
deposit_tree_root	QHashOut<F>	Root of the deposit tree
user_tree_root	QHashOut<F>	Root of the user tree
withdrawal_tree_root	QHashOut<F>	Root of the withdrawal tree
user_registration_tree_root	QHashOut<F>	Root of the user registration tree

Example:

```
{
    "contract_tree_root": "0x1234...",
    "deposit_tree_root": "0x5678...",
    "user_tree_root": "0x9abc...",
    "withdrawal_tree_root": "0xdef0...",
    "user_registration_tree_root": "0x1234..."
}
```

PsyUserLeaf

User leaf data containing user state information.

Structure:

```
pub struct PsyUserLeaf<F: RichField> {
    pub public_key: QHashOut<F>,
    pub user_state_tree_root: QHashOut<F>,
    pub balance: F,
    pub nonce: F,
    pub last_checkpoint_id: F,
    pub event_index: F,
    pub user_id: F,
}
```

Fields:

Field	Type	Description
public_key	QHashOut<F>	User's public key hash
user_state_tree_root	QHashOut<F>	Root of user's state tree
balance	F	User's balance
nonce	F	User's transaction nonce
last_checkpoint_id	F	Last checkpoint ID where user was updated
event_index	F	Event index for this user
user_id	F	User identifier

Example:

```
{
  "public_key": "0x1234...",
  "user_state_tree_root": "0x5678...",
  "balance": "1000000",
  "nonce": "42",
  "last_checkpoint_id": "100",
  "event_index": "15",
  "user_id": "12345"
}
```

MerkleProofCore

Generic Merkle proof structure.

Structure:

```
pub struct MerkleProofCore<Hash: PartialEq + Copy> {  
    pub root: Hash,  
    pub value: Hash,  
    pub index: u64,  
    pub siblings: Vec<Hash>,  
}
```

Fields:

Field	Type	Description
root	Hash	Merkle tree root hash
value	Hash	Leaf value being proven
index	u64	Leaf index in the tree
siblings	Vec<Hash>	Sibling hashes along the path

Example:

```
{  
  "root": "0x1234...",  
  "value": "0x5678...",  
  "index": 42,  
  "siblings": [  
    "0x9abc...",  
    "0xdef0...",  
    "0x1234..."  
  ]  
}
```

Verification: The proof can be verified by hashing the value with siblings along the path according to the index bits.

SubmitUserEndCapNonProofInput

Input data for submitting user end cap (without proof).

Structure:

```

pub struct SubmitUserEndCapNonProofInput<F: RichField> {
    pub core: SubmitUserEndCapNonProofCoreInput<F>,
    pub contract_state_updates:
Vec<PsyContractStateUpdateHistory<F>>,
}

pub struct SubmitUserEndCapNonProofCoreInput<F: RichField> {
    pub checkpoint_id: F,
    pub stats: GUTASStats<F>,
    pub state_transition: UPSEndCapResultCompact<F>,
    pub new_user_leaf: PsyUserLeaf<F>,
}

```

Fields:

Field	Type
core	SubmitUserEndCapNonProofCoreInput<F>
contract_state_updates	Vec<PsyContractStateUpdateHistory<F>>

Core Fields:

Field	Type	Description
checkpoint_id	F	Checkpoint ID
stats	GUTASStats<F>	GUTA statistics
state_transition	UPSEndCapResultCompact<F>	State transition result
new_user_leaf	PsyUserLeaf<F>	New user leaf data

QProvingJobDataID

Proving job data identifier.

Structure:

```
pub struct QProvingJobDataID {  
    pub topic: QJobTopic,  
    pub goal_id: u64,  
    pub slot_id: u64,  
    pub circuit_type: ProvingJobCircuitType,  
    pub group_id: u32,  
    pub sub_group_id: u32,  
    pub task_index: u32,  
    pub data_type: ProvingJobDataType,  
    pub data_index: u8,  
}
```

Fields:

Field	Type	Description
topic	QJobTopic	Job topic (e.g., GenerateStandardProof)
goal_id	u64	Goal identifier (usually checkpoint ID)
slot_id	u64	Slot identifier
circuit_type	ProvingJobCircuitType	Type of circuit (e.g., GUTATwoEndCap)
group_id	u32	Group identifier
sub_group_id	u32	Sub-group identifier
task_index	u32	Task index within the group
data_type	ProvingJobDataType	Data type (e.g., InputWitness)
data_index	u8	Data index

Serialization: Serialized as a 32-byte array.

Example:

```
{
  "topic": "GenerateStandardProof",
  "goal_id": 100,
  "slot_id": 5,
  "circuit_type": "GUTATwoEndCap",
  "group_id": 1,
  "sub_group_id": 0,
  "task_index": 0,
  "data_type": "InputWitness",
  "data_index": 0
}
```

VariableHeightRewardMerkleProof

Variable height Merkle proof for reward distribution.

Structure:

```
pub struct VariableHeightRewardMerkleProof {
  pub top_siblings: Vec<VariableHeightProofSibling>,
  pub sibling_branch: QHashOut<F>,
  pub reward_leaf: QHashOut<F>,
  pub proof_height: F,
  pub index: F,
}

pub struct VariableHeightProofSibling {
  pub sibling_branch: QHashOut<F>,
  pub sibling_reward_leaf: QHashOut<F>,
}
```

Fields:

Field	Type	Description
top_siblings	Vec<VariableHeightProofSibling>	Siblings at each level
sibling_branch	QHashOut<F>	Sibling branch

Field	Type	Description
		hash
<code>reward_leaf</code>	<code>QHashOut<F></code>	Reward leaf hash
<code>proof_height</code>	<code>F</code>	Height of the proof
<code>index</code>	<code>F</code>	Index in the tree

Example:

```
{
  "top_siblings": [
    {
      "sibling_branch": "0x1234...",
      "sibling_reward_leaf": "0x5678..."
    }
  ],
  "sibling_branch": "0x9abc...",
  "reward_leaf": "0xdef0...",
  "proof_height": "5",
  "index": "42"
}
```

Field Type Notes

Throughout this API, `F` represents a field element type (typically `GoldilocksField`).

Field Element Conversion:

- Methods with `_f` suffix accept field elements as strings (e.g., `"12345"`)
- Methods without `_f` suffix accept native types (e.g., `12345`)
- Field elements are internally represented as `u64` values in the Goldilocks field

Best Practices:

- Use u64 variants for better performance when possible
 - Use Field variants when working with circuit inputs/outputs
 - Always validate checkpoint_id exists before querying
 - Handle RPC errors gracefully (missing data, invalid parameters)
-

Error Handling

All RPC methods return `RpcResult<T>` which can contain errors in the following format:

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32000,
    "message": "Error description"
  },
  "id": 1
}
```

Common Error Codes:

- `-32000` : Server error (checkpoint not found, data unavailable)
 - `-32602` : Invalid parameters
 - `-32603` : Internal error
-

Usage Examples

Complete Workflow Example

```
# 1. Check if user belongs to realm
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_check_user_id_in_realm",
    "params": [12345],
    "id": 1
  }'

# 2. Get latest L2 block state
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_latest_block_state",
    "params": [],
    "id": 2
  }'

# 3. Get user leaf data at checkpoint
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_user_leaf_data",
    "params": [100, 12345],
    "id": 3
  }'

# 4. Get Merkle proof for user
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_user_tree_merkle_proof",
    "params": [100, 12345],
    "id": 4
  }'
```

Method Summary

#	Method Name	Parameter
1	<code>check_user_id_in_realm</code>	<code>user_id: u</code>
2	<code>submit_user_end_cap</code>	<code>input, pro</code>
3-4	<code>get_checkpoint_leaf_data[_f]</code>	<code>checkpoint</code>
5-7	<code>get_[latest_]block_state[_f]</code>	<code>[checkpoir</code>
8	<code>get_user_registration_tree_root</code>	<code>checkpoint</code>
9-11	<code>get_[latest_]checkpoint_tree_root[_f]</code>	<code>[checkpoir</code>
12-15	<code>get_checkpoint_tree_[leaf_hash\ merkle_proof] [_f]</code>	<code>checkpoint leaf_id</code>
16	<code>get_checkpoint_global_state_roots</code>	<code>checkpoint</code>
17-18	<code>get_user_leaf_data[_f]</code>	<code>checkpoint user_id</code>
19-24	<code>get_user_contract_state_tree_*[_f]</code>	Various
25-30	<code>get_user_contract_tree_*[_f]</code>	Various
31-40	<code>get_user_tree_*[_f]</code>	Various

#	Method Name	Parameter
41	generate_batch_variable_height_reward_proofs	checkpoint job_ids
42	get_graphviz	checkpoint

Document Version: 1.0
Last Updated: 2025-10-24
Total RPC Methods: 42

Coordinator Edge RPC Documentation

This document provides comprehensive documentation for all Coordinator Edge RPC methods defined in the `CoordinatorEdgeRpc` trait.

RPC Namespace: `psy`

Table of Contents

1. [User Management](#)
 2. [Contract Management](#)
 3. [Block Operations](#)
 4. [GUTA Submission](#)
 5. [Checkpoint Operations](#)
 6. [Checkpoint Sync](#)
 7. [L2 Block State Operations](#)
 8. [User Registration Tree Operations](#)
 9. [User Tree Operations](#)
 10. [Contract Function Tree Operations](#)
 11. [Contract Tree Operations](#)
 12. [Deposit Tree Operations](#)
 13. [Withdrawal Tree Operations](#)
 14. [Checkpoint Tree Operations](#)
 15. [Reward Proofs Generation](#)
 16. [Realm Status Operations](#)
 17. [Data Structures](#)
-

User Management

1. register_user

Register a new user with ZK public key.

Method Name: `psy_register_user`

Request Parameters:

```
{
  "public_key": {
    "fingerprint": "0x1234...",
    "public_key_param": "0x5678..."
  }
}
```

Parameter	Type	Description
<code>public_key</code>	<code>ZKPublicKeyInfo<F></code>	ZK public key information

Response:

```
{
  "result": "ok"
}
```

Field	Type	Description
<code>result</code>	<code>String</code>	"ok" on success

Example:

```
curl -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "psy_register_user",
  "params": [{
    "fingerprint": "0x...",
    "public_key_param": "0x..."
  }],
  "id": 1
}'
```

2. get_user_id

Get user ID by public key hash.

Method Name: `psy_get_user_id`

Request Parameters:

```
{
  "public_key": "0x1234567890abcdef..."
}
```

Parameter	Type	Description
<code>public_key</code>	<code>QHashOut<F></code>	User's public key hash

Response:

```
{
  "result": 12345
}
```

Field	Type	Description
<code>result</code>	<code>u64</code>	User ID

Example:

```
curl -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "psy_get_user_id",
  "params": ["0x1234..."],
  "id": 1
}'
```

Contract Management

3. deploy_contract

Deploy a new smart contract.

Method Name: `psy_deploy_contract`

Request Parameters:

```
{
  "deploy_contract": {
    "deployer": "0x1234...",
    "code_definition": {
      "state_tree_height": 10,
      "functions": [...]
    },
    "function_whitelist": ["0x..."]
  }
}
```

Parameter	Type	Description
<code>deploy_contract</code>	<code>QBCTDeployContract<F></code>	Contract deployment data (see QBCTDeployContract)

Response:

```
{
  "result": "ok"
}
```

Example:

```
curl -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "psy_deploy_contract",
  "params": [{
    "deployer": "0x...",
    "code_definition": {...},
    "function_whitelist": [...]
  }],
  "id": 1
}'
```

4. get_contract_leaf_data

Get contract leaf data by contract ID (u64 parameter).

Method Name: `psy_get_contract_leaf_data`

Request Parameters:

```
{
  "contract_id": 5
}
```

Parameter	Type	Description
<code>contract_id</code>	<code>u64</code>	The contract ID

Response: See [PsyContractLeaf](#)

5. get_contract_leaf_data_f

Get contract leaf data by contract ID (Field parameter).

Method Name: `psy_get_contract_leaf_data_f`

Request Parameters:

```
{
  "contract_id": "5"
}
```

Parameter	Type	Description
<code>contract_id</code>	F (Field)	The contract ID as a field element

Response: See [PsyContractLeaf](#)

6. get_contract_code_definition

Get contract code definition by contract ID (u64 parameter).

Method Name: `psy_get_contract_code_definition`

Request Parameters:

```
{
  "contract_id": 5
}
```

Parameter	Type	Description
<code>contract_id</code>	u64	The contract ID

Response: See [ContractCodeDefinition](#)

7. get_contract_code_definition_f

Get contract code definition by contract ID (Field parameter).

Method Name: `psy_get_contract_code_definition_f`

Request Parameters:

```
{
  "contract_id": "5"
}
```

Parameter	Type	Description
<code>contract_id</code>	F (Field)	The contract ID as a field element

Response: See [ContractCodeDefinition](#)

Block Operations

8. build_block

Trigger building a new block/checkpoint.

Method Name: `psy_build_block`

Request Parameters: None

Response:

```
{
  "result": "ok"
}
```

Example:

```
curl -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "psy_build_block",
  "params": [],
  "id": 1
}'
```

GUTA Submission

9. submit_guta

Submit GUTA (Global User Tree Aggregator) result with proof.

Method Name: `psy_submit_guta`

Request Parameters:

```
{
  "input": {
    "realm_id": 1,
    "checkpoint_id": 100,
    "guta_stats": {...},
    "top_line_proof": {...},
    "checkpoint_tree_root": "0x...",
    "proof_id": {...}
  },
  "proof": {...},
  "realm_id": 1
}
```

Parameter	Type	Descripti
<code>input</code>	<code>SubmitGUTAResultAPINoProofInput<F></code>	GUTA submissic input

Parameter	Type	Description
proof	ProofWithPublicInputs<F, C, D>	Zero-knowledge proof
realm_id	u64	Realm identifier

Response:

```
{
  "result": "ok"
}
```

10. submit_guta_v1

Submit GUTA result with serialized proof (v1 format).

Method Name: psy_submit_guta_v1

Request Parameters:

```
{
  "input": {...},
  "proof": "0x...",
  "realm_id": 1
}
```

Parameter	Type	Description
input	SubmitGUTAREalmResultAPINoProofInput<F>	GUTA submission input
proof	Vec<u8>	Serialized proof bytes
realm_id	u64	Realm identifier



Response:

```
{
  "result": null
}
```

11. submit_realm_result

Submit realm processing result to coordinator.

Method Name: `psy_submit_realm_result`

Request Parameters:

```
{
  "realm_result": {
    "header": {
      "realm_id": 1,
      "checkpoint_id": 100,
      "start_realm_root": "0x...",
      "end_realm_root": "0x...",
      "guta_stats": {...},
      "root_job_id": {...}
    },
    "proof": "0x..."
  }
}
```

Parameter	Type	Description
<code>realm_result</code>	<code>RealmDataForCoordinator<F></code>	Realm result data

Response:

```
{
  "result": null
}
```

Checkpoint Operations

12. get_latest_checkpoint

Get latest checkpoint information.

Method Name: psy_get_latest_checkpoint

Request Parameters: None

Response:

```
{
  "result": {
    "checkpoint_id": 100
  }
}
```

Field	Type	Description
checkpoint_id	u64	Latest checkpoint ID

13. latest_checkpoint

Get latest checkpoint ID (simple version).

Method Name: psy_latest_checkpoint

Request Parameters: None

Response:

```
{
  "result": 100
}
```

14. get_latest_checkpoint_id

Get latest checkpoint ID.

Method Name: psy_get_latest_checkpoint_id

Request Parameters: None

Response:

```
{
  "result": 100
}
```

Example:

```
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_latest_checkpoint_id",
    "params": [],
    "id": 1
  }'
```

15. get_checkpoint_leaf_data

Get checkpoint leaf data by checkpoint ID (u64 parameter).

Method Name: psy_get_checkpoint_leaf_data

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [PsyCheckpointLeaf](#)

16. get_checkpoint_leaf_data_f

Get checkpoint leaf data by checkpoint ID (Field parameter).

Method Name: `psy_get_checkpoint_leaf_data_f`

Request Parameters:

```
{
  "checkpoint_id": "100"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element

Response: See [PsyCheckpointLeaf](#)

17. get_checkpoint_global_state_roots

Get global state roots at a specific checkpoint.

Method Name: `psy_get_checkpoint_global_state_roots`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [PsyCheckpointGlobalStateRoots](#)

Checkpoint Sync

18. get_checkpoint_sync_info

Get checkpoint sync information for a realm.

Method Name: `psy_get_checkpoint_sync_info`

Request Parameters:

```
{
  "realm_id": 1,
  "checkpoint_id": 100
}
```

Parameter	Type	Description
realm_id	u32	Realm identifier
checkpoint_id	u64	Checkpoint ID

Response: See [CheckpointSyncInfo](#)

19. get_checkpoint_sync_info_compact

Get compact checkpoint sync information.

Method Name: `psy_get_checkpoint_sync_info_compact`

Request Parameters:

```
{  
  "checkpoint_id": 100  
}
```

Parameter	Type	Description
checkpoint_id	u64	Checkpoint ID

Response: See [PsyCheckpointSyncInfoCompact](#psycheckpointsyncinfocompact)

L2 Block State Operations

20. get_latest_block_state

Get the latest L2 block state.

Method Name: `psy_get_latest_block_state`

Request Parameters: None

Response: See [PsyBlockState](#)

Example Response:

```
{
  "result": {
    "checkpoint_id": 100,
    "next_add_withdrawal_id": 50,
    "next_process_withdrawal_id": 45,
    "next_deposit_id": 200,
    "total_deposits_claimed_epoch": 180,
    "next_user_id": 1000,
    "end_balance": 5000000,
    "next_contract_id": 25
  }
}
```

21. get_block_state

Get L2 block state at a specific checkpoint (u64 parameter).

Method Name: `psy_get_block_state`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	<code>u64</code>	The checkpoint ID

Response: See [PsyBlockState](#)

22. get_block_state_f

Get L2 block state at a specific checkpoint (Field parameter).

Method Name: `psy_get_block_state_f`

Request Parameters:

```
{  
  "checkpoint_id": "100"  
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element

Response: See [PsyBlockState](#)

User Registration Tree Operations

23. get_user_registration_tree_root

Get user registration tree root at a specific checkpoint (u64 parameter).

Method Name: psy_get_user_registration_tree_root

Request Parameters:

```
{  
  "checkpoint_id": 100  
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [QHashOut](#)

24. get_user_registration_tree_root_f

Get user registration tree root at a specific checkpoint (Field parameter).

Method Name: psy_get_user_registration_tree_root_f

Request Parameters:

```
{
  "checkpoint_id": "100"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element

Response: See [QHashOut](#)

25. get_user_registration_tree_leaf_hash

Get user registration tree leaf hash (u64 parameters).

Method Name: psy_get_user_registration_tree_leaf_hash

Request Parameters:

```
{
  "checkpoint_id": 100,
  "leaf_index": 42
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
leaf_index	u64	The leaf index

Response: See [QHashOut](#)

26. get_user_registration_tree_leaf_hash_f

Get user registration tree leaf hash (Field parameters).

Method Name: psy_get_user_registration_tree_leaf_hash_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "leaf_index": "42"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
leaf_index	F (Field)	The leaf index as a field element

Response: See [QHashOut](#)

27. get_user_registration_tree_merkle_proof

Get Merkle proof for user registration tree (u64 parameters).

Method Name: psy_get_user_registration_tree_merkle_proof

Request Parameters:

```
{
  "checkpoint_id": 100,
  "leaf_index": 42
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
leaf_index	u64	The leaf index

Response: See [MerkleProofCore](#)

28. get_user_registration_tree_merkle_proof_f

Get Merkle proof for user registration tree (Field parameters).

Method Name: `psy_get_user_registration_tree_merkle_proof_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "leaf_index": "42"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	F (Field)	The checkpoint ID as a field element
<code>leaf_index</code>	F (Field)	The leaf index as a field element

Response: See [MerkleProofCore](#)

User Tree Operations

29. get_user_tree_root

Get user tree root at a specific checkpoint (u64 parameter).

Method Name: `psy_get_user_tree_root`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [QHashOut](#)

30. get_user_tree_root_f

Get user tree root at a specific checkpoint (Field parameter).

Method Name: psy_get_user_tree_root_f

Request Parameters:

```
{
  "checkpoint_id": "100"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element

Response: See [QHashOut](#)

31. get_user_sub_tree_merkle_proof

Get Merkle proof for user sub-tree.

Method Name: psy_get_user_sub_tree_merkle_proof

Request Parameters:

```
{
  "checkpoint_id": 100,
  "root_level": 5,
  "leaf_level": 2,
  "leaf_index": 42
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
root_level	u8	Root level of the tree
leaf_level	u8	Leaf level of the tree
leaf_index	u64	The leaf index

Response: See [MerkleProofCore](#)

32. get_user_top_tree_merkle_proof

Get Merkle proof for user top tree.

Method Name: psy_get_user_top_tree_merkle_proof

Request Parameters:

```
{
  "checkpoint_id": 100,
  "leaf_level": 2,
  "leaf_index": 42
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
leaf_level	u8	Leaf level of the tree
leaf_index	u64	The leaf index

Response: See [MerkleProofCore](#)

33. get_user_top_tree_cap_root

Get user top tree cap root.

Method Name: `psy_get_user_top_tree_cap_root`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "cap_level": 3,
  "cap_index": 10
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
cap_level	u8	Cap level
cap_index	u64	Cap index

Response: See [QHashOut](#)

34. get_user_latest_top_tree_cap_root

Get latest user top tree cap root.

Method Name: `psy_get_user_latest_top_tree_cap_root`

Request Parameters:

```
{
  "cap_level": 3,
  "cap_index": 10
}
```

Parameter	Type	Description
cap_level	u8	Cap level

Parameter	Type	Description
cap_index	u64	Cap index

Response: See [QHashOut](#)

35. get_user_leaf_data

Get user leaf data at a specific checkpoint.

Method Name: psy_get_user_leaf_data

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
user_id	u64	The user ID

Response: See [PsyUserLeaf](#)

36. get_user_tree_merkle_proof

Get Merkle proof for user tree (u64 parameters).

Method Name: psy_get_user_tree_merkle_proof

Request Parameters:

```
{
  "checkpoint_id": 100,
  "user_id": 12345
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
user_id	u64	The user ID

Response: See [MerkleProofCore](#)

37. get_user_tree_merkle_proof_f

Get Merkle proof for user tree (Field parameters).

Method Name: psy_get_user_tree_merkle_proof_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "user_id": "12345"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
user_id	F (Field)	The user ID as a field element

Response: See [MerkleProofCore](#)

Contract Function Tree Operations

38. get_contract_function_tree_root

Get contract function tree root (u64 parameters).

Method Name: `psy_get_contract_function_tree_root`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "contract_id": 5
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	<code>u64</code>	The checkpoint ID
<code>contract_id</code>	<code>u32</code>	The contract ID

Response: See [QHashOut](#)

39. get_contract_function_tree_root_f

Get contract function tree root (Field parameters).

Method Name: `psy_get_contract_function_tree_root_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "contract_id": "5"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
contract_id	F (Field)	The contract ID as a field element

Response: See [QHashOut](#)

40. get_contract_function_tree_leaf_hash

Get contract function tree leaf hash (u64 parameters).

Method Name: psy_get_contract_function_tree_leaf_hash

Request Parameters:

```
{
  "checkpoint_id": 100,
  "contract_id": 5,
  "function_id": 3
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
contract_id	u32	The contract ID
function_id	u32	The function ID

Response: See [QHashOut](#)

41. get_contract_function_tree_leaf_hash_f

Get contract function tree leaf hash (Field parameters).

Method Name: psy_get_contract_function_tree_leaf_hash_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "contract_id": "5",
  "function_id": "3"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
contract_id	F (Field)	The contract ID as a field element
function_id	F (Field)	The function ID as a field element

Response: See [QHashOut](#)

42. get_contract_function_tree_merkle_proof

Get Merkle proof for contract function tree (u64 parameters).

Method Name: psy_get_contract_function_tree_merkle_proof

Request Parameters:

```
{
  "checkpoint_id": 100,
  "contract_id": 5,
  "function_id": 3
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
contract_id	u32	The contract ID
function_id	u32	The function ID

Response: See [MerkleProofCore](#)

43. get_contract_function_tree_merkle_proof_f

Get Merkle proof for contract function tree (Field parameters).

Method Name: `psy_get_contract_function_tree_merkle_proof_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "contract_id": "5",
  "function_id": "3"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	F (Field)	The checkpoint ID as a field element
<code>contract_id</code>	F (Field)	The contract ID as a field element
<code>function_id</code>	F (Field)	The function ID as a field element

Response: See [MerkleProofCore](#)

Contract Tree Operations

44. get_contract_tree_root

Get contract tree root at a specific checkpoint (u64 parameter).

Method Name: `psy_get_contract_tree_root`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [QHashOut](#)

45. get_contract_tree_root_f

Get contract tree root at a specific checkpoint (Field parameter).

Method Name: psy_get_contract_tree_root_f

Request Parameters:

```
{
  "checkpoint_id": "100"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element

Response: See [QHashOut](#)

46. get_contract_tree_leaf_hash

Get contract tree leaf hash (u64 parameters).

Method Name: psy_get_contract_tree_leaf_hash

Request Parameters:

```
{
  "checkpoint_id": 100,
  "contract_id": 5
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
contract_id	u32	The contract ID

Response: See [QHashOut](#)

47. get_contract_tree_leaf_hash_f

Get contract tree leaf hash (Field parameters).

Method Name: `psy_get_contract_tree_leaf_hash_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "contract_id": "5"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
contract_id	F (Field)	The contract ID as a field element

Response: See [QHashOut](#)

48. get_contract_tree_merkle_proof

Get Merkle proof for contract tree (u64 parameters).

Method Name: `psy_get_contract_tree_merkle_proof`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "contract_id": 5
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
contract_id	u32	The contract ID

Response: See [MerkleProofCore](#)

49. get_contract_tree_merkle_proof_f

Get Merkle proof for contract tree (Field parameters).

Method Name: psy_get_contract_tree_merkle_proof_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "contract_id": "5"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
contract_id	F (Field)	The contract ID as a field element

Response: See [MerkleProofCore](#)

Deposit Tree Operations

50. get_deposit_tree_root

Get deposit tree root at a specific checkpoint (u64 parameter).

Method Name: `psy_get_deposit_tree_root`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	<code>u64</code>	The checkpoint ID

Response: See [QHashOut](#)

51. get_deposit_tree_root_f

Get deposit tree root at a specific checkpoint (Field parameter).

Method Name: `psy_get_deposit_tree_root_f`

Request Parameters:

```
{
  "checkpoint_id": "100"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	<code>F (Field)</code>	The checkpoint ID as a field element

Response: See [QHashOut](#)

52. get_deposit_tree_leaf_hash

Get deposit tree leaf hash (u64 parameters).

Method Name: `psy_get_deposit_tree_leaf_hash`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "deposit_id": 42
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	<code>u64</code>	The checkpoint ID
<code>deposit_id</code>	<code>u32</code>	The deposit ID

Response: See [QHashOut](#)

53. get_deposit_tree_leaf_hash_f

Get deposit tree leaf hash (Field parameters).

Method Name: `psy_get_deposit_tree_leaf_hash_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "deposit_id": "42"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	<code>F (Field)</code>	The checkpoint ID as a field element
<code>deposit_id</code>	<code>F (Field)</code>	The deposit ID as a field element

Response: See [QHashOut](#)

54. get_deposit_tree_merkle_proof

Get Merkle proof for deposit tree (u64 parameters).

Method Name: `psy_get_deposit_tree_merkle_proof`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "deposit_id": 42
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	<code>u64</code>	The checkpoint ID
<code>deposit_id</code>	<code>u32</code>	The deposit ID

Response: See [MerkleProofCore](#)

55. get_deposit_tree_merkle_proof_f

Get Merkle proof for deposit tree (Field parameters).

Method Name: `psy_get_deposit_tree_merkle_proof_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "deposit_id": "42"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	<code>F (Field)</code>	The checkpoint ID as a field element

Parameter	Type	Description
deposit_id	F (Field)	The deposit ID as a field element

Response: See [MerkleProofCore](#)

Withdrawal Tree Operations

56. get_withdrawal_tree_root

Get withdrawal tree root at a specific checkpoint (u64 parameter).

Method Name: psy_get_withdrawal_tree_root

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [QHashOut](#)

57. get_withdrawal_tree_root_f

Get withdrawal tree root at a specific checkpoint (Field parameter).

Method Name: psy_get_withdrawal_tree_root_f

Request Parameters:

```
{
  "checkpoint_id": "100"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element

Response: See [QHashOut](#)

58. get_withdrawal_tree_leaf_hash

Get withdrawal tree leaf hash (u64 parameters).

Method Name: psy_get_withdrawal_tree_leaf_hash

Request Parameters:

```
{
  "checkpoint_id": 100,
  "withdrawal_id": 42
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
withdrawal_id	u32	The withdrawal ID

Response: See [QHashOut](#)

59. get_withdrawal_tree_leaf_hash_f

Get withdrawal tree leaf hash (Field parameters).

Method Name: psy_get_withdrawal_tree_leaf_hash_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "withdrawal_id": "42"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
withdrawal_id	F (Field)	The withdrawal ID as a field element

Response: See [QHashOut](#)

60. get_withdrawal_tree_merkle_proof

Get Merkle proof for withdrawal tree (u64 parameters).

Method Name: psy_get_withdrawal_tree_merkle_proof

Request Parameters:

```
{
  "checkpoint_id": 100,
  "withdrawal_id": 42
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
withdrawal_id	u32	The withdrawal ID

Response: See [MerkleProofCore](#)

61. get_withdrawal_tree_merkle_proof_f

Get Merkle proof for withdrawal tree (Field parameters).

Method Name: `psy_get_withdrawal_tree_merkle_proof_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "withdrawal_id": "42"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	F (Field)	The checkpoint ID as a field element
<code>withdrawal_id</code>	F (Field)	The withdrawal ID as a field element

Response: See [MerkleProofCore](#)

Checkpoint Tree Operations

62. `get_latest_checkpoint_tree_root`

Get the latest checkpoint tree root.

Method Name: `psy_get_latest_checkpoint_tree_root`

Request Parameters: None

Response: See [QHashOut](#)

63. `get_checkpoint_tree_root`

Get checkpoint tree root at a specific checkpoint (u64 parameter).

Method Name: `psy_get_checkpoint_tree_root`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response: See [QHashOut](#)

64. get_checkpoint_tree_root_f

Get checkpoint tree root at a specific checkpoint (Field parameter).

Method Name: `psy_get_checkpoint_tree_root_f`

Request Parameters:

```
{
  "checkpoint_id": "100"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element

Response: See [QHashOut](#)

65. get_checkpoint_tree_leaf_hash

Get checkpoint tree leaf hash (u64 parameters).

Method Name: `psy_get_checkpoint_tree_leaf_hash`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "leaf_checkpoint_id": 95
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID
leaf_checkpoint_id	u64	The leaf checkpoint ID

Response: See [QHashOut](#)

66. get_checkpoint_tree_leaf_hash_f

Get checkpoint tree leaf hash (Field parameters).

Method Name: psy_get_checkpoint_tree_leaf_hash_f

Request Parameters:

```
{
  "checkpoint_id": "100",
  "leaf_checkpoint_id": "95"
}
```

Parameter	Type	Description
checkpoint_id	F (Field)	The checkpoint ID as a field element
leaf_checkpoint_id	F (Field)	The leaf checkpoint ID as a field element

Response: See [QHashOut](#)

67. get_checkpoint_tree_merkle_proof

Get Merkle proof for checkpoint tree (u64 parameters).

Method Name: `psy_get_checkpoint_tree_merkle_proof`

Request Parameters:

```
{
  "checkpoint_id": 100,
  "leaf_checkpoint_id": 95
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	u64	The checkpoint ID
<code>leaf_checkpoint_id</code>	u64	The leaf checkpoint ID

Response: See [MerkleProofCore](#)

68. get_checkpoint_tree_merkle_proof_f

Get Merkle proof for checkpoint tree (Field parameters).

Method Name: `psy_get_checkpoint_tree_merkle_proof_f`

Request Parameters:

```
{
  "checkpoint_id": "100",
  "leaf_checkpoint_id": "95"
}
```

Parameter	Type	Description
<code>checkpoint_id</code>	F (Field)	The checkpoint ID as a field element

Parameter	Type	Description
leaf_checkpoint_id	F (Field)	The leaf checkpoint ID as a field element

Response: See [MerkleProofCore](#)

Reward Proofs Generation

69. generate_batch_variable_height_reward_proofs

Generate batch variable height reward Merkle proofs for multiple job IDs.

Method Name: psy_generate_batch_variable_height_reward_proofs

Request Parameters:

```
{
  "checkpoint_id": 100,
  "job_ids": [
    {
      "topic": "GenerateStandardProof",
      "goal_id": 100,
      "slot_id": 5,
      "circuit_type": "GUTATwoEndCap",
      "group_id": 1,
      "sub_group_id": 0,
      "task_index": 0,
      "data_type": "InputWitness",
      "data_index": 0
    }
  ]
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Parameter	Type	Description
<code>job_ids</code>	<code>Vec<QProvingJobDataID></code>	Array of proving job data IDs

Response:

```
{
  "result": [
    [
      {
        "top_siblings": [...],
        "sibling_branch": "0x...",
        "reward_leaf": "0x...",
        "proof_height": "5",
        "index": "42"
      },
      {
        "topic": "GenerateStandardProof",
        "goal_id": 100,
        ...
      }
    ]
  ]
}
```

Field	Type	Description
<code>result</code>	<code>Vec<(VariableHeightRewardMerkleProof, QProvingJobDataID)></code>	Array of tuples containing proofs and job IDs

70. get_graphviz

Get GraphViz representation of the job dependency graph at a specific checkpoint.

Method Name: `psy_get_graphviz`

Request Parameters:

```
{
  "checkpoint_id": 100
}
```

Parameter	Type	Description
checkpoint_id	u64	The checkpoint ID

Response:

```
{
  "result": "digraph G {\n  node1 -> node2;\n  ...\n}"
}
```

Field	Type	Description
result	String	GraphViz DOT format string

Example:

```
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_graphviz",
    "params": [100],
    "id": 1
  }' | jq -r '.result' | dot -Tpng > graph.png
```

Realm Status Operations

71. get_current_realm_status_on_coordinator

Get current realm status on the coordinator.

Method Name: `psy_get_current_realm_status_on_coordinator`

Request Parameters:

```
{
  "realm_id": 1
}
```

Parameter	Type	Description
<code>realm_id</code>	<code>u64</code>	Realm identifier

Response: See [BasicRealmStatusOnCoordinator](#)

Example Response:

```
{
  "result": {
    "realm_id": 1,
    "checkpoint_id": 100,
    "realm_root_hash": "0x..."
  }
}
```

72. `get_current_checkpoint_id`

Get current coordinator checkpoint ID.

Method Name: `psy_get_current_checkpoint_id`

Request Parameters: None

Response:

```
{
  "result": 100
}
```

73. get_latest_block_updates_from_coordinator

Get latest block updates from coordinator for a realm within a checkpoint range.

Method Name: `psy_get_latest_block_updates_from_coordinator`

Request Parameters:

```
{
  "realm_id": 1,
  "from_checkpoint": 95,
  "to_checkpoint": 100
}
```

Parameter	Type	Description
<code>realm_id</code>	<code>u32</code>	Realm identifier
<code>from_checkpoint</code>	<code>u64</code>	Starting checkpoint ID (inclusive)
<code>to_checkpoint</code>	<code>u64</code>	Ending checkpoint ID (inclusive)

Response:

```
{
  "result": [
    {
      "latest_checkpoint_id": 100,
      "description": null,
      "source_coordinator_edge_id": null,
      "sync_timestamp": 1234567890,
      "compact": {...},
      "realm_root": "0x..."
    }
  ]
}
```

Field	Type	Description
<code>result</code>	<code>Vec<GlobalBlockUpdateFromCoordinator<F>></code>	Array of block updates

74. wait_until_coordinator_completed

Wait until coordinator completes a specific checkpoint for a realm.

Method Name: `psy_wait_until_coordinator_completed`

Request Parameters:

```
{
  "realm_id": 1,
  "checkpoint_id": 100
}
```

Parameter	Type	Description
<code>realm_id</code>	<code>u64</code>	Realm identifier
<code>checkpoint_id</code>	<code>u64</code>	Checkpoint ID to wait for

Response: See [GlobalBlockUpdateFromCoordinator](#)

Data Structures

QHashOut

A hash output wrapper for Plonky2 field elements.

Structure:

```
pub struct QHashOut<F: Field>(pub HashOut<F>);
```

JSON Representation:

```
"0x1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef"
```

Description:

- Wraps a Plonky2 `HashOut<F>` containing 4 field elements
 - Serialized as a hexadecimal string (32 bytes)
 - Represents a 256-bit hash value
-

PsyBlockState

L2 block state information at a specific checkpoint.

Structure:

```
pub struct PsyBlockState {  
    pub checkpoint_id: u64,  
    pub next_add_withdrawal_id: u64,  
    pub next_process_withdrawal_id: u64,  
    pub next_deposit_id: u64,  
    pub total_deposits_claimed_epoch: u64,  
    pub next_user_id: u64,  
    pub end_balance: u64,  
    pub next_contract_id: u32,  
}
```

Fields:

Field	Type	Description
<code>checkpoint_id</code>	<code>u64</code>	The checkpoint identifier
<code>next_add_withdrawal_id</code>	<code>u64</code>	Next withdrawal ID to be added
<code>next_process_withdrawal_id</code>	<code>u64</code>	Next withdrawal ID to be processed
<code>next_deposit_id</code>	<code>u64</code>	Next deposit ID
<code>total_deposits_claimed_epoch</code>	<code>u64</code>	Total deposits claimed in current epoch
<code>next_user_id</code>	<code>u64</code>	Next user ID to be assigned
<code>end_balance</code>	<code>u64</code>	Ending balance at this checkpoint

Field	Type	Description
<code>next_contract_id</code>	<code>u32</code>	Next contract ID to be assigned

Example:

```
{
  "checkpoint_id": 100,
  "next_add_withdrawal_id": 50,
  "next_process_withdrawal_id": 45,
  "next_deposit_id": 200,
  "total_deposits_claimed_epoch": 180,
  "next_user_id": 1000,
  "end_balance": 5000000,
  "next_contract_id": 25
}
```

PsyCheckpointLeaf

Checkpoint leaf data containing global chain root and statistics.

Structure:

```
pub struct PsyCheckpointLeaf<F: RichField> {
  pub global_chain_root: QHashOut<F>,
  pub stats: PsyCheckpointLeafStats<F>,
}
```

Fields:

Field	Type	Description
<code>global_chain_root</code>	<code>QHashOut<F></code>	Global chain state root hash
<code>stats</code>	<code>PsyCheckpointLeafStats<F></code>	Checkpoint statistics

PsyCheckpointGlobalStateRoots

Global state roots at a specific checkpoint.

Structure:

```
pub struct PsyCheckpointGlobalStateRoots<F: RichField> {  
    pub contract_tree_root: QHashOut<F>,  
    pub deposit_tree_root: QHashOut<F>,  
    pub user_tree_root: QHashOut<F>,  
    pub withdrawal_tree_root: QHashOut<F>,  
    pub user_registration_tree_root: QHashOut<F>,  
}
```

Fields:

Field	Type	Description
contract_tree_root	QHashOut<F>	Root of the contract tree
deposit_tree_root	QHashOut<F>	Root of the deposit tree
user_tree_root	QHashOut<F>	Root of the user tree
withdrawal_tree_root	QHashOut<F>	Root of the withdrawal tree
user_registration_tree_root	QHashOut<F>	Root of the user registration tree

Example:

```
{  
    "contract_tree_root": "0x1234...",  
    "deposit_tree_root": "0x5678...",  
    "user_tree_root": "0x9abc...",  
    "withdrawal_tree_root": "0xdef0...",  
    "user_registration_tree_root": "0x1234..."  
}
```

PsyUserLeaf

User leaf data containing user state information.

Structure:

```
pub struct PsyUserLeaf<F: RichField> {  
    pub public_key: QHashOut<F>,  
    pub user_state_tree_root: QHashOut<F>,  
    pub balance: F,  
    pub nonce: F,  
    pub last_checkpoint_id: F,  
    pub event_index: F,  
    pub user_id: F,  
}
```

Fields:

Field	Type	Description
public_key	QHashOut<F>	User's public key hash
user_state_tree_root	QHashOut<F>	Root of user's state tree
balance	F	User's balance
nonce	F	User's transaction nonce
last_checkpoint_id	F	Last checkpoint ID where user was updated
event_index	F	Event index for this user
user_id	F	User identifier

Example:

```
{  
    "public_key": "0x1234...",  
    "user_state_tree_root": "0x5678...",  
    "balance": "1000000",  
    "nonce": "42",  
    "last_checkpoint_id": "100",  
    "event_index": "15",  
    "user_id": "12345"  
}
```

MerkleProofCore

Generic Merkle proof structure.

Structure:

```
pub struct MerkleProofCore<Hash: PartialEq + Copy> {  
    pub root: Hash,  
    pub value: Hash,  
    pub index: u64,  
    pub siblings: Vec<Hash>,  
}
```

Fields:

Field	Type	Description
root	Hash	Merkle tree root hash
value	Hash	Leaf value being proven
index	u64	Leaf index in the tree
siblings	Vec<Hash>	Sibling hashes along the path

Example:

```
{  
  "root": "0x1234...",  
  "value": "0x5678...",  
  "index": 42,  
  "siblings": [  
    "0x9abc...",  
    "0xdef0...",  
    "0x1234..."  
  ]  
}
```

ZKPublicKeyInfo

Zero-knowledge public key information.

Structure:

```
pub struct ZKPublicKeyInfo<F: RichField> {  
    pub fingerprint: QHashOut<F>,  
    pub public_key_param: QHashOut<F>,  
}
```

Fields:

Field	Type	Description
fingerprint	QHashOut<F>	Public key fingerprint
public_key_param	QHashOut<F>	Public key parameter

Example:

```
{  
    "fingerprint": "0x1234...",  
    "public_key_param": "0x5678..."  
}
```

QBCDeployContract

Contract deployment command.

Structure:

```
pub struct QBCDeployContract<F: RichField> {  
    pub deployer: QHashOut<F>,  
    pub code_definition: ContractCodeDefinition,  
    pub function_whitelist: Vec<QHashOut<F>>,  
}
```

Fields:

Field	Type	Description
<code>deployer</code>	<code>QHashOut<F></code>	Deployer's public key hash
<code>code_definition</code>	<code>ContractCodeDefinition</code>	Contract code definition
<code>function_whitelist</code>	<code>Vec<QHashOut<F>></code>	Function whitelist hashes

Example:

```
{
  "deployer": "0x1234...",
  "code_definition": {
    "state_tree_height": 10,
    "functions": [...]
  },
  "function_whitelist": ["0x5678..."]
}
```

ContractCodeDefinition

Contract code definition structure.

Structure:

```
pub struct ContractCodeDefinition {
    pub state_tree_height: u16,
    pub functions: Vec<ContractFunctionCodeDefinition>,
}

pub struct ContractFunctionCodeDefinition {
    pub method_id: u32,
    pub num_inputs: u32,
    pub num_outputs: u32,
    pub vm_type: u32,
    pub code: Vec<u8>,
}
```

Fields:

Field	Type	Desc
state_tree_height	u16	Height of the state tree
functions	Vec<ContractFunctionCodeDefinition>	Contract function definitions

Function Fields:

Field	Type	Description
method_id	u32	Function method ID
num_inputs	u32	Number of input parameters
num_outputs	u32	Number of output parameters
vm_type	u32	VM type identifier
code	Vec<u8>	Function bytecode

Example:

```
{
  "state_tree_height": 10,
  "functions": [
    {
      "method_id": 12345678,
      "num_inputs": 2,
      "num_outputs": 1,
      "vm_type": 1,
      "code": [0x01, 0x02, ...]
    }
  ]
}
```

PsyContractLeaf

Contract leaf data.

Structure:

```
pub struct PsyContractLeaf<F: RichField> {  
    pub deployer: QHashOut<F>,  
    pub function_tree_root: QHashOut<F>,  
    pub state_tree_height: F,  
}
```

Fields:

Field	Type	Description
deployer	QHashOut<F>	Deployer's public key hash
function_tree_root	QHashOut<F>	Root of the function tree
state_tree_height	F	Height of the state tree

Example:

```
{  
    "deployer": "0x1234...",  
    "function_tree_root": "0x5678...",  
    "state_tree_height": "10"  
}
```

CheckpointSyncInfo

Checkpoint synchronization information.

Structure:

```
pub struct CheckpointSyncInfo<F: RichField> {  
    pub latest_checkpoint_id: u64,  
    pub description: Option<String>,  
    pub source_coordinator_edge_id: Option<String>,  
    pub sync_timestamp: u64,  
    pub compact: PsyCheckpointSyncInfoCompact<F>,  
    pub realm_root: QHashOut<F>,  
}
```


Fields:

Field	Type
latest_checkpoint_id	u64
description	Option<String>
source_coordinator_edge_id	Option<String>
sync_timestamp	u64
compact	PsyCheckpointSyncInfoCompact<F>
realm_root	QHashOut<F>

PsyCheckpointSyncInfoCompact

Compact checkpoint synchronization information.

Structure:

```
pub struct PsyCheckpointSyncInfoCompact<F: RichField> {  
    pub block_state: PsyBlockState,  
    pub stats: PsyCheckpointLeafStats<F>,  
    pub state_roots: PsyCheckpointGlobalStateRoots<F>,  
    pub checkpoint_tree_update_siblings: Vec<QHashOut<F>>,  
    pub regsitered_users_start_pivot_siblings: Vec<QHashOut<F>>,  
    pub registered_users: Vec<ZKPublicKeyInfo<F>>,  
    pub old_checkpoint_leaf_hash: QHashOut<F>,  
    pub slot: u64,  
}
```

Fields:

Field	Type
block_state	PsyBlockState
stats	PsyCheckpointLeafStats<F>
state_roots	PsyCheckpointGlobalStateR
checkpoint_tree_update_siblings	Vec<QHashOut<F>>
regsitered_users_start_pivot_siblings	Vec<QHashOut<F>>
registered_users	Vec<ZKPublicKeyInfo<F>>
old_checkpoint_leaf_hash	QHashOut<F>
slot	u64

SubmitGUTARealmResultAPINoProofInput

GUTA submission input without proof.

Structure:

```
pub struct SubmitGUTRealmResultAPINoProofInput<F: RichField> {
    pub realm_id: u64,
    pub checkpoint_id: u64,
    pub guta_stats: GUTASStats<F>,
    pub top_line_proof: DeltaMerkleProofCore<QHashOut<F>>,
    pub checkpoint_tree_root: QHashOut<F>,
    pub proof_id: QProvingJobDataID,
}
```

Fields:

Field	Type	Des
realm_id	u64	Rea ider
checkpoint_id	u64	Che ID
guta_stats	GUTASStats<F>	GUT stat
top_line_proof	DeltaMerkleProofCore<QHashOut<F>>	Top Mer proc
checkpoint_tree_root	QHashOut<F>	Che tree
proof_id	QProvingJobDataID	Prov ID

QProvingJobDataID

Proving job data identifier.

Structure:

```
pub struct QProvingJobDataID {
    pub topic: QJobTopic,
    pub goal_id: u64,
    pub slot_id: u64,
    pub circuit_type: ProvingJobCircuitType,
    pub group_id: u32,
    pub sub_group_id: u32,
    pub task_index: u32,
    pub data_type: ProvingJobDataType,
    pub data_index: u8,
}
```

Fields:

Field	Type	Description
topic	QJobTopic	Job topic
goal_id	u64	Goal identifier (usually checkpoint ID)
slot_id	u64	Slot identifier
circuit_type	ProvingJobCircuitType	Type of circuit
group_id	u32	Group identifier
sub_group_id	u32	Sub-group identifier
task_index	u32	Task index within the group
data_type	ProvingJobDataType	Data type
data_index	u8	Data index

Serialization: Serialized as a 32-byte array.

VariableHeightRewardMerkleProof

Variable height Merkle proof for reward distribution.

Structure:

```
pub struct VariableHeightRewardMerkleProof {
    pub top_siblings: Vec<VariableHeightProofSibling>,
    pub sibling_branch: QHashOut<F>,
    pub reward_leaf: QHashOut<F>,
    pub proof_height: F,
    pub index: F,
}

pub struct VariableHeightProofSibling {
    pub sibling_branch: QHashOut<F>,
    pub sibling_reward_leaf: QHashOut<F>,
}
```

Fields:

Field	Type	Description
top_siblings	Vec<VariableHeightProofSibling>	Siblings at each level
sibling_branch	QHashOut<F>	Sibling branch hash
reward_leaf	QHashOut<F>	Reward leaf hash
proof_height	F	Height of the proof
index	F	Index in the tree

BasicRealmStatusOnCoordinator

Basic realm status on the coordinator.

Structure:

```
pub struct BasicRealmStatusOnCoordinator<F: RichField> {
    pub realm_id: u64,
    pub checkpoint_id: u64,
    pub realm_root_hash: QHashOut<F>,
}
```

Fields:

Field	Type	Description
realm_id	u64	Realm identifier
checkpoint_id	u64	Current checkpoint ID
realm_root_hash	QHashOut<F>	Realm root hash

Example:

```
{
  "realm_id": 1,
  "checkpoint_id": 100,
  "realm_root_hash": "0x1234..."
}
```

GlobalBlockUpdateFromCoordinator

Type alias for `CheckpointSyncInfo<F>`.

```
pub type GlobalBlockUpdateFromCoordinator<F> =
    CheckpointSyncInfo<F>;
```

See [CheckpointSyncInfo](#) for structure details.

RealmDataForCoordinator

Realm data submitted to coordinator.

Structure:

```
pub struct RealmDataForCoordinator<F: RichField> {
    pub header: RealmDataForCoordinatorHeader<F>,
    pub proof: Vec<u8>,
}

pub struct RealmDataForCoordinatorHeader<F: RichField> {
    pub realm_id: u64,
    pub checkpoint_id: u64,
    pub start_realm_root: QHashOut<F>,
    pub end_realm_root: QHashOut<F>,
    pub guta_stats: GUTASStats<F>,
    pub root_job_id: QProvingJobDataID,
}
```

Fields:

Field	Type	Description
header	RealmDataForCoordinatorHeader<F>	Header data
proof	Vec<u8>	Serialized proof

Header Fields:

Field	Type	Description
realm_id	u64	Realm identifier
checkpoint_id	u64	Checkpoint ID
start_realm_root	QHashOut<F>	Starting realm root
end_realm_root	QHashOut<F>	Ending realm root
guta_stats	GUTASStats<F>	GUTA statistics
root_job_id	QProvingJobDataID	Root job ID

Field Type Notes

Throughout this API, `F` represents a field element type (typically `GoldilocksField`).

Field Element Conversion:

- Methods with `_f` suffix accept field elements as strings (e.g., `"12345"`)
- Methods without `_f` suffix accept native types (e.g., `12345`)
- Field elements are internally represented as `u64` values in the Goldilocks field

Best Practices:

- Use `u64` variants for better performance when possible
 - Use Field variants when working with circuit inputs/outputs
 - Always validate `checkpoint_id` exists before querying
 - Handle RPC errors gracefully (missing data, invalid parameters)
-

Error Handling

All RPC methods return `RpcResult<T>` which can contain errors in the following format:

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32000,
    "message": "Error description"
  },
  "id": 1
}
```

Common Error Codes:

- `-32000` : Server error (checkpoint not found, data unavailable)
 - `-32602` : Invalid parameters
 - `-32603` : Internal error
 - `-32001` : Not found error
-

Usage Examples

Complete User Registration Workflow

```
# 1. Register a new user
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_register_user",
    "params": [{
      "fingerprint": "0x...",
      "public_key_param": "0x..."
    }],
    "id": 1
  }'
```

```
# 2. Get user ID by public key
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_user_id",
    "params": ["0x..."],
    "id": 2
  }'
```

```
# 3. Get user leaf data
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_user_leaf_data",
    "params": [100, 12345],
    "id": 3
  }'
```

Contract Deployment Workflow

```
# 1. Deploy contract
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_deploy_contract",
    "params": [{
      "deployer": "0x...",
      "code_definition": {
        "state_tree_height": 10,
        "functions": [...]
      },
      "function_whitelist": [...]
    }],
    "id": 1
  }'

# 2. Get contract leaf data
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_contract_leaf_data",
    "params": [5],
    "id": 2
  }'

# 3. Get contract code definition
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_contract_code_definition",
    "params": [5],
    "id": 3
  }'
```

Checkpoint Query Workflow

```
# 1. Get latest checkpoint ID
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_latest_checkpoint_id",
    "params": [],
    "id": 1
  }'

# 2. Get checkpoint leaf data
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_checkpoint_leaf_data",
    "params": [100],
    "id": 2
  }'

# 3. Get checkpoint global state roots
curl -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "psy_get_checkpoint_global_state_roots",
    "params": [100],
    "id": 3
  }'
```

Method Summary

#	Method Name	Paramet
1	register_user	public_key
2	get_user_id	public_key

#	Method Name	Paramet
3	<code>deploy_contract</code>	<code>deploy_con</code>
4-5	<code>get_contract_leaf_data[_f]</code>	<code>contract_i</code>
6-7	<code>get_contract_code_definition[_f]</code>	<code>contract_i</code>
8	<code>build_block</code>	None
9-11	<code>submit_guta[_v1]</code>	<code>input, pro</code> <code>realm_id</code>
12-14	<code>[get_]latest_checkpoint[_id]</code>	None
15-16	<code>get_checkpoint_leaf_data[_f]</code>	<code>checkpoint</code>
17	<code>get_checkpoint_global_state_roots</code>	<code>checkpoint</code>
18-19	<code>get_checkpoint_sync_info[_compact]</code>	<code>realm_id?,</code> <code>checkpoint_</code>
20-22	<code>get_[latest_]block_state[_f]</code>	<code>[checkpoin</code>
23-28	<code>get_user_registration_tree_*[_f]</code>	Various
29-37	<code>get_user_tree_*[_f]</code>	Various
38-43	<code>get_contract_function_tree_*[_f]</code>	Various
44-49	<code>get_contract_tree_*[_f]</code>	Various
50-55	<code>get_deposit_tree_*[_f]</code>	Various

#	Method Name	Paramet
56-61	<code>get_withdrawal_tree_*[_f]</code>	Various
62-68	<code>get_[latest_]checkpoint_tree_*[_f]</code>	Various
69	<code>generate_batch_variable_height_reward_proofs</code>	<code>checkpoint,</code> <code>job_ids</code>
70	<code>get_graphviz</code>	<code>checkpoint,</code>
71	<code>get_current_realm_status_on_coordinator</code>	<code>realm_id</code>
72	<code>get_current_checkpoint_id</code>	None
73	<code>get_latest_block_updates_from_coordinator</code>	<code>realm_id,</code> <code>to</code>
74	<code>wait_until_coordinator_completed</code>	<code>realm_id,</code> <code>checkpoint_</code>

Document Version: 1.0

Last Updated: 2025-10-24

Total RPC Methods: 74

Appendix A: Glossary

- **Felt:** A numeric type, likely an integer.
- **Hash:** A fixed-size array used for state storage.
- **Storage:** Persistent state management for contracts.

Appendix B: Reserved Keywords

- `fn`, `pub`, `mod`, `use`, `struct`, `impl`, `trait`, `let`, `mut`, `if`, `else`,
`while`, `match`, `return`, `assert`, `assert_eq`

Appendix C: Publications

Appendix D: Contributing

To contribute, submit a pull request to the [GitHub repository]. The source files are in mdBook format.

Appendix E: Acknowledgements